

HỌC VIỆN KỸ THUẬT MẬT MÃ
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN TỐT NGHIỆP

XÂY DỰNG NHÓM TRỢ LÝ ẢO AI ĐA TÁC TỬ TRONG CÁC TRƯỜNG ĐẠI HỌC

ĐỖ THÁI DƯƠNG CT060310

Ngành: Công nghệ Thông tin

Chuyên ngành: Phát triển phần mềm di động

Người hướng dẫn: TS. Phạm Văn Hưởng – Học viện Kỹ thuật Mật mã
ThS. Ngô Việt Phương – Công ty Công nghệ Thông tin VNPT

Hà Nội, 06-2026

HỌC VIỆN KỸ THUẬT MẬT MÃ
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN TỐT NGHIỆP

XÂY DỰNG NHÓM TRỢ LÝ ẢO AI ĐA TÁC TỬ TRONG CÁC TRƯỜNG ĐẠI HỌC

ĐỖ THÁI DƯƠNG CT060310

Ngành: Công nghệ Thông tin

Chuyên ngành: Phát triển phần mềm di động

Người hướng dẫn: TS. Phạm Văn Hưởng – Học viện Kỹ thuật Mật mã
ThS. Ngô Việt Phương – Công ty Công nghệ Thông tin VNPT

Hà Nội, 06-2026

LỜI CAM ĐOAN

Em xin cam đoan đồ án tốt nghiệp với đề tài **Xây dựng nhóm trợ lý ảo AI đa tác tử hỗ trợ trường đại học** là kết quả của quá trình tìm hiểu, nghiên cứu và thực hiện của bản thân em dưới sự hướng dẫn của TS. Phạm Văn Hưởng và ThS. Ngô Việt Phương.

Các nội dung, số liệu và kết quả trình bày trong đồ án được em tổng hợp, xây dựng và kiểm chứng một cách trung thực, phù hợp với phạm vi đề tài. Để hoàn thành đồ án này, em chỉ sử dụng những tài liệu đã được ghi trong mục Tài liệu tham khảo; các tài liệu, số liệu tham khảo được trích dẫn đầy đủ và rõ ràng theo quy định, ngoài ra không sử dụng bất kỳ tài liệu nào khác mà không được chú thích.

Em xin hoàn toàn chịu trách nhiệm về nội dung của đồ án này. Nếu có bất kỳ sai phạm nào, em xin chịu mọi hình thức kỷ luật theo quy định của Nhà trường.

Hà Nội, ngày 01 tháng 06 năm 2026

SINH VIÊN

ĐỖ THÁI DƯƠNG

LỜI CẢM ƠN

Trước tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến quý thầy cô Học viện Kỹ thuật Mật mã đã tận tình giảng dạy, truyền đạt kiến thức trong suốt quá trình học tập tại trường.

Đặc biệt, em xin bày tỏ lòng biết ơn sâu sắc đến thầy **TS. Phạm Văn Hưởng** và cô **ThS. Ngô Việt Phương** – những người đã trực tiếp hướng dẫn, định hướng nghiên cứu và tận tình chỉ bảo em trong suốt quá trình thực hiện đề án tốt nghiệp. Sự chỉ dẫn tận tâm, những chia sẻ kinh nghiệm quý báu cùng kiến thức chuyên môn sâu rộng của thầy cô đã đóng vai trò quan trọng, giúp em từng bước hoàn thiện đề tài ”Xây dựng nhóm trợ lý ảo AI đa tác tử hỗ trợ các trường đại học”.

Em cũng xin gửi lời cảm ơn chân thành đến gia đình và bạn bè – những người luôn ở bên động viên, khích lệ và là nguồn động lực lớn lao giúp em vượt qua những khó khăn trong suốt thời gian nghiên cứu và hoàn thành đề án.

Mặc dù đã cố gắng hết sức trong quá trình thực hiện, song do kiến thức và kinh nghiệm nghiên cứu còn nhiều hạn chế, báo cáo đề án không tránh khỏi những thiếu sót nhất định. Em rất mong nhận được sự quan tâm, nhận xét và góp ý từ quý thầy cô để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn!

MỤC LỤC

	LỜI CAM ĐOAN	ii
	LỜI CẢM ƠN	iii
	MỤC LỤC	iv
Hình	DANH MỤC HÌNH ẢNH	viii
Bảng	DANH MỤC BẢNG	x
	LỜI MỞ ĐẦU	1
	CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI	3
	1.1 Tổng quan về trợ lý ảo AI trong giáo dục	3
	1.1.1 Khái niệm trợ lý ảo AI	3
	1.1.2 Tiềm năng ứng dụng trong môi trường đại học	4
	1.1.3 Khảo sát một số hệ thống trợ lý ảo hiện có	5
	1.2 Các phương pháp xây dựng hệ thống trợ lý ảo	5
	1.2.1 Hệ thống dựa trên tập luật	6
	1.2.2 Hệ thống dựa trên mô hình ngôn ngữ lớn	6
	1.2.3 Hệ thống dựa trên AI Agent	7
	1.3 Mô hình ngôn ngữ lớn	8
	1.3.1 Khái niệm và kiến trúc	8
	1.3.2 Hiện tượng ảo giác và hướng giải quyết	10
	1.4 Kỹ thuật RAG (Retrieval-Augmented Generation)	10
	1.4.1 Khái niệm và kiến trúc RAG cơ bản	10
	1.4.2 Vector Database và tìm kiếm ngữ nghĩa	12
	1.4.3 Agentic RAG	12
	1.5 AI Agent và hệ thống đa tác tử	13
	1.5.1 Khái niệm AI Agent	13
	1.5.2 Các thành phần của AI Agent	14
	1.5.3 Công cụ trong AI Agent	16
	1.5.4 Lập kế hoạch trong AI Agent	16
	1.5.5 Tự đánh giá và sửa lỗi	17
	1.5.6 Hệ thống đa tác tử và kiến trúc Supervisor–Specialist	18
	1.6 Các công nghệ sử dụng trong đề tài	20
	1.6.1 Ngôn ngữ lập trình Python	20
	1.6.2 FastAPI Framework	21
	1.6.3 OpenAI API	22

1.6.4	<i>Qdrant Vector Database</i>	23
1.6.5	<i>Hệ quản trị cơ sở dữ liệu PostgreSQL</i>	25
1.6.6	<i>Redis</i>	26
1.7	Kết luận chương 1	26
CHƯƠNG 2: PHƯƠNG PHÁP XÂY DỰNG HỆ THỐNG TRỢ LÝ ẢO AI		
ĐA TÁC TỬ		28
2.1	Mô hình kiến trúc tổng thể	28
2.1.1	<i>Đặc tả yêu cầu và phạm vi hệ thống</i>	28
2.1.2	<i>Kiến trúc đa tác tử tổng quan</i>	30
2.2	Cơ chế điều phối truy vấn trong hệ đa tác tử	35
2.2.1	<i>Bộ nhớ phiên trong luồng điều phối</i>	35
2.2.2	<i>Guardrail và kiểm soát phạm vi câu hỏi</i>	37
2.2.3	<i>Query Rewriter và chuẩn hóa câu hỏi</i>	39
2.2.4	<i>Planner và phân rã truy vấn phức tạp</i>	41
2.2.5	<i>Supervisor và định tuyến tác tử</i>	43
2.2.6	<i>Router và lựa chọn pipeline xử lý</i>	46
2.2.7	<i>Aggregator và tổng hợp câu trả lời đa tác tử</i>	47
2.3	Phương pháp đánh giá độ phức tạp truy vấn	49
2.3.1	<i>Bài toán lựa chọn mức xử lý</i>	49
2.3.2	<i>Thiết kế công thức Q_c</i>	49
2.3.3	<i>Các thành phần của chỉ số Q_c</i>	50
2.3.4	<i>Chế độ đánh giá trong triển khai</i>	53
2.3.5	<i>Phân loại intent truy vấn</i>	54
2.3.6	<i>Kết quả đánh giá và tích hợp hệ thống</i>	55
2.4	Phân tầng pipeline truy xuất và sinh câu trả lời	57
2.4.1	<i>Nguyên lý phân tầng pipeline</i>	57
2.4.2	<i>Cơ chế truy xuất nền tảng</i>	58
2.4.3	<i>Pipeline Native RAG ($Q_c < 0,50$)</i>	61
2.4.4	<i>Pipeline Hybrid RAG và cơ chế leo thang ($T_1 \leq Q_c < T_2$)</i>	62
2.4.5	<i>Pipeline Agentic RAG ($Q_c \geq 0,70$ hoặc leo thang)</i>	63
2.4.6	<i>Pipeline Grade Lookup cho tra cứu điểm</i>	65
2.4.7	<i>Đường nhanh cho lịch thi và danh sách thi</i>	67
2.5	Phân rã nghiệp vụ thành các tác tử chuyên biệt	68
2.5.1	<i>Nguyên lý phân chia nghiệp vụ</i>	68
2.5.2	<i>Tổ chức và phân đoạn tài liệu</i>	69

2.5.3	<i>Đặc điểm kỹ thuật của từng tác tử</i>	70
2.5.4	<i>Luồng xử lý của Specialist Runner</i>	73
2.6	Quản lý ngữ cảnh hội thoại đa lượt	73
2.6.1	<i>Bài toán bộ nhớ trong hội thoại đa lượt</i>	73
2.6.2	<i>Kiến trúc lưu trữ nóng–lạnh</i>	74
2.6.3	<i>Trạng thái phiên hội thoại</i>	75
2.6.4	<i>Tóm tắt phiên tự động</i>	76
2.6.5	<i>Phân phối lịch sử theo vai trò xử lý</i>	77
2.6.6	<i>Cache truy vấn trong phiên</i>	77
2.6.7	<i>Tích hợp ngữ cảnh vào luồng xử lý</i>	78
2.7	Tổng kết chương	78
CHƯƠNG 3: PHÂN TÍCH THIẾT KẾ HỆ THỐNG		79
3.1	Yêu cầu hệ thống	79
3.1.1	<i>Yêu cầu về hình thức sản phẩm</i>	79
3.1.2	<i>Yêu cầu chức năng</i>	79
3.1.3	<i>Yêu cầu phi chức năng</i>	81
3.2	Phân tích hệ thống	82
3.2.1	<i>Biểu đồ usecase tổng quát</i>	82
3.2.2	<i>Đặc tả usecase</i>	84
3.2.3	<i>Biểu đồ tuần tự</i>	91
3.3	Thiết kế hệ thống	96
3.3.1	<i>Kiến trúc tổng thể hệ thống</i>	96
3.3.2	<i>Thiết kế cơ sở dữ liệu</i>	98
3.3.3	<i>Thiết kế API</i>	102
3.4	Tổng kết chương 3	106
CHƯƠNG 4: XÂY DỰNG HỆ THỐNG VÀ THỰC NGHIỆM		107
4.1	Mô hình tổng quan của hệ thống	107
4.2	Các chức năng của hệ thống	107
4.2.1	<i>Trang chủ và tin tức thông báo</i>	107
4.2.2	<i>Giao diện hội thoại với chatbot AI</i>	108
4.2.3	<i>Giao diện quản trị</i>	112
4.2.4	<i>Giao diện tra cứu bảng điểm</i>	114
4.2.5	<i>Giao diện theo dõi thời khóa biểu</i>	115
4.3	Đánh giá	116
4.3.1	<i>Mô tả bộ dữ liệu thực nghiệm</i>	116

4.3.2	<i>Kịch bản thực nghiệm</i>	118
4.3.3	<i>Tiến hành thực nghiệm</i>	119
4.3.4	<i>Kết quả thực nghiệm</i>	120
4.4	Tổng kết chương 4	121
KẾT LUẬN		122
Kết quả đạt được		122
Hạn chế còn tồn tại		123
Hướng phát triển		123
TÀI LIỆU THAM KHẢO		125

DANH MỤC HÌNH ẢNH

Hình 1.1	Hệ thống dựa trên tập luật khi đầu vào không nằm trong kịch bản . . .	6
Hình 1.2	Mô hình ngôn ngữ lớn (Large Language Model)	9
Hình 1.3	Mô hình RAG (Retrieval-Augmented Generation)	11
Hình 1.4	Kiến trúc Agentic RAG	13
Hình 1.5	Kiến trúc tổng quát của AI Agent	14
Hình 1.6	Các thành phần trong kiến trúc AI Agent hiện đại	14
Hình 1.7	Quy trình lập kế hoạch và tái lập kế hoạch trong AI Agent	17
Hình 1.8	Khuôn mẫu ReAct trong quá trình suy luận và hành động của Agent	18
Hình 1.9	Kiến trúc Supervisor–Specialist trong hệ thống đa tác tử	19
Hình 1.10	Sơ đồ cấu trúc Collection → Points → (Vector + Payload)	24
Hình 1.11	Sơ đồ HNSW đơn giản minh họa tìm kiếm theo tầng	24
Hình 2.1	Mô hình kiến trúc tổng quan hệ thống trợ lý ảo đa tác tử KMA . . .	31
Hình 2.2	Sơ đồ luồng xử lý tổng quan khi người dùng gửi truy vấn	33
Hình 2.3	Luồng xử lý điều phối đa tác tử	35
Hình 2.4	So sánh ba pipeline RAG theo chỉ số Q_c	57
Hình 2.5	Kiến trúc quản lý phiên Hot–Cold	75
Hình 3.1	Biểu đồ usecase tổng quát của hệ thống	83
Hình 3.2	Usecase gửi câu hỏi và nhận phản hồi từ chatbot	84
Hình 3.3	Usecase tra cứu điểm thi theo mã số sinh viên	85
Hình 3.4	Usecase tra cứu lịch thi học phần	85
Hình 3.5	Usecase tra cứu danh sách sinh viên dự thi	86
Hình 3.6	Usecase hỏi biểu mẫu và thủ tục hành chính	87
Hình 3.7	Usecase đặt câu hỏi đa ý định	88
Hình 3.8	Usecase đăng nhập hệ thống quản trị	89
Hình 3.9	Usecase upload tài liệu nghiệp vụ	90
Hình 3.10	Biểu đồ tuần tự luồng gửi câu hỏi và nhận phản hồi	92
Hình 3.11	Biểu đồ tuần tự luồng tra cứu điểm thi theo MSSV	92
Hình 3.12	Biểu đồ tuần tự luồng xử lý câu hỏi đa ý định	93
Hình 3.13	Biểu đồ tuần tự luồng upload tài liệu nghiệp vụ	94
Hình 3.14	Biểu đồ tuần tự luồng đăng nhập hệ thống quản trị bằng OTP	95
Hình 3.15	Biểu đồ tuần tự luồng xác thực OTP và phát hành JWT	96
Hình 3.16	Sơ đồ kiến trúc tổng thể hệ thống	97
Hình 4.1	Kiến trúc hệ thống	107
Hình 4.2	Trang chủ và danh sách tin tức thông báo	108
Hình 4.3	Giao diện hội thoại với chatbot AI	108
Hình 4.4	Hỏi đáp thông tin nghiệp vụ trên giao diện chatbot	109
Hình 4.5	Kết quả trả lời có kèm tài liệu nguồn tham chiếu	109

Hình 4.6	Tra cứu điểm thi theo mã số sinh viên	110
Hình 4.7	Tra cứu lịch thi và danh sách dự thi	111
Hình 4.8	Hỏi biểu mẫu và thủ tục hành chính	111
Hình 4.9	Xử lý câu hỏi đa ý định	112
Hình 4.10	Giao diện đăng nhập hệ thống quản trị	112
Hình 4.11	Giao diện xác thực OTP khi đăng nhập quản trị	113
Hình 4.12	Giao diện quản lý tài liệu nghiệp vụ	113
Hình 4.13	Giao diện quản lý tin tức thông báo	114
Hình 4.14	Giao diện tra cứu bảng điểm và mô phỏng cải thiện GPA	115
Hình 4.15	Giao diện theo dõi thời khóa biểu học kỳ	116

DANH MỤC BẢNG

Bảng 1.1	So sánh các phương pháp xây dựng hệ thống trợ lý ảo	8
Bảng 2.1	Bảng xác định tính chất môi trường của hệ thống đa tác tử	28
Bảng 2.2	Bảng xác định miền tri thức của hệ thống	29
Bảng 2.3	Danh sách và đặc tả các tác tử chuyên môn	32
Bảng 2.4	Kiến trúc bộ nhớ hai tầng của Session Memory	36
Bảng 2.5	Cấu trúc dữ liệu SessionState	36
Bảng 2.6	Giới hạn lịch sử hội thoại theo mục đích sử dụng	37
Bảng 2.7	Phân loại câu hỏi và hành vi phản hồi của Guardrail	37
Bảng 2.8	Thứ tự kiểm tra rule-based của Guardrail	38
Bảng 2.9	Cấu trúc kết quả ScopeResult của Guardrail	39
Bảng 2.10	Tiêu chí tính điểm heuristic của Query Rewriter	40
Bảng 2.11	Cấu trúc kết quả RewriteResult của Query Rewriter	41
Bảng 2.12	Tiêu chí quyết định phân rã câu hỏi của Planner	41
Bảng 2.13	Cấu trúc kết quả PlanResult của Planner	42
Bảng 2.14	Phân loại intent của Supervisor	43
Bảng 2.15	Các fast-path theo thứ tự ưu tiên của Supervisor	44
Bảng 2.16	Cấu trúc kết quả RoutingDecision của Supervisor	45
Bảng 2.17	Quy tắc định tuyến pipeline theo Q_c	46
Bảng 2.18	Điều chỉnh pipeline theo intent Supervisor	47
Bảng 2.19	Cấu trúc một khối kết quả tác tử đầu vào Aggregator	48
Bảng 2.20	Các thành phần và trọng số trong công thức Q_c	50
Bảng 2.21	Ví dụ tính thành phần E	51
Bảng 2.22	Các dependency tags phát hiện quan hệ phức tạp	52
Bảng 2.23	Điều kiện kích hoạt LLM trong chế độ Hybrid	54
Bảng 2.24	Sáu nhóm intent và ảnh hưởng đến pipeline	55
Bảng 2.25	Cấu trúc đối tượng ComplexityAssessment	56
Bảng 2.26	Ví dụ tính Q_c cho một số câu hỏi học vụ KMA thực tế	56
Bảng 2.27	Ba pipeline truy xuất và sinh câu trả lời theo độ phức tạp truy vấn	57
Bảng 2.28	Ví dụ từ khóa text-index theo từng agent	59
Bảng 2.29	Các tín hiệu điều chỉnh điểm reranking	60
Bảng 2.30	Ba tầng kiểm tra leo thang trong Hybrid RAG	63
Bảng 2.31	Điều kiện thoát vòng lặp Agentic RAG	65
Bảng 2.32	Tổng quan bảy tác tử chuyên biệt	68
Bảng 2.33	Cấu trúc đối tượng SessionState	76
Bảng 2.34	Phân phối lịch sử hội thoại theo vai trò xử lý	77
Bảng 3.1	Đặc tả Usecase gửi câu hỏi và nhận phản hồi từ chatbot	84
Bảng 3.2	Đặc tả Usecase tra cứu điểm thi theo MSSV	85

Bảng 3.3	Đặc tả Usecase tra cứu lịch thi học phần	86
Bảng 3.4	Đặc tả Usecase tra cứu danh sách sinh viên dự thi	87
Bảng 3.5	Đặc tả Usecase hỏi biểu mẫu và thủ tục hành chính	88
Bảng 3.6	Đặc tả Usecase đặt câu hỏi đa ý định	89
Bảng 3.7	Đặc tả Usecase đăng nhập hệ thống quản trị	90
Bảng 3.8	Đặc tả Usecase upload tài liệu nghiệp vụ	91
Bảng 3.9	Cấu trúc bảng users	98
Bảng 3.10	Cấu trúc bảng chat_session_archives	99
Bảng 3.11	Cấu trúc bảng chat_logs	99
Bảng 3.12	Cấu trúc bảng document_records	100
Bảng 3.13	Cấu trúc bảng audit_logs	100
Bảng 3.14	Cấu trúc bảng token_sessions	101
Bảng 3.15	Cấu trúc vector point trong collection kma_docs	102
Bảng 3.16	API tạo phiên hội thoại mới	102
Bảng 3.17	API đóng phiên hội thoại	102
Bảng 3.18	API gửi câu hỏi với phản hồi đầy đủ	103
Bảng 3.19	API gửi câu hỏi với phản hồi streaming SSE	103
Bảng 3.20	API tải biểu mẫu đã điền	103
Bảng 3.21	API đăng nhập bước 1 – xác thực mật khẩu	104
Bảng 3.22	API xác nhận OTP bước 2 – nhận JWT	104
Bảng 3.23	API đổi mật khẩu	104
Bảng 3.24	API lấy danh sách tài liệu	105
Bảng 3.25	API upload tài liệu	105
Bảng 3.26	API xóa tài liệu	105
Bảng 3.27	API re-index tài liệu theo tác tử	105
Bảng 3.28	API upload tin tức	106
Bảng 3.29	API xóa tin tức	106
Bảng 4.1	Bảng mô tả ý nghĩa các trường của bộ dữ liệu thực nghiệm	117
Bảng 4.2	Một số câu hỏi nhóm N1 trong bộ dữ liệu thực nghiệm	117
Bảng 4.3	Một số câu hỏi nhóm N2 trong bộ dữ liệu thực nghiệm	118
Bảng 4.4	Kết quả thực nghiệm bộ dữ liệu N1	120
Bảng 4.5	Kết quả thực nghiệm bộ dữ liệu N2	120

LỜI MỞ ĐẦU

Trong bối cảnh chuyển đổi số giáo dục diễn ra mạnh mẽ, các trường đại học đang mở rộng hệ sinh thái số với cổng thông tin đào tạo, LMS, thư viện điện tử, dịch vụ hành chính trực tuyến và nhiều kênh tương tác đa nền tảng. Tuy nhiên, đi cùng với sự phát triển đó là áp lực ngày càng lớn trong việc cung cấp thông tin học vụ kịp thời, chính xác và nhất quán cho người học. Những câu hỏi lặp lại với tần suất cao về lịch học, lịch thi, chương trình đào tạo, quy chế, biểu mẫu hay thủ tục hành chính không chỉ làm gia tăng khối lượng công việc của các đơn vị chức năng mà còn ảnh hưởng trực tiếp đến trải nghiệm của sinh viên khi cần hỗ trợ nhanh.

Các mô hình chatbot truyền thống đã góp phần tự động hóa bước đầu, nhưng thường gặp hạn chế khi xử lý ngữ cảnh phức tạp, câu hỏi liên ngành, hoặc yêu cầu cần kết hợp nhiều nguồn tri thức chuyên biệt. Sự phát triển của mô hình ngôn ngữ lớn (LLMs) cùng kiến trúc AI đa tác tử mở ra hướng tiếp cận phù hợp hơn cho môi trường đại học: mỗi tác tử đảm nhiệm một vai trò rõ ràng (định tuyến yêu cầu, truy xuất tri thức, đánh giá độ liên quan, tổng hợp phản hồi, kiểm soát an toàn), từ đó phối hợp để tạo ra câu trả lời có độ tin cậy cao hơn, linh hoạt hơn và dễ mở rộng hơn theo từng nghiệp vụ.

Xuất phát từ nhu cầu thực tiễn và định hướng ứng dụng công nghệ AI trong giáo dục đại học, em thực hiện đề tài **Xây dựng nhóm trợ lý ảo AI đa tác tử hỗ trợ các trường đại học**. Đề tài hướng tới xây dựng một hệ thống trợ lý ảo có khả năng vận hành liên tục, hỗ trợ đồng thời nhiều người dùng, giảm tải cho bộ phận quản trị - đào tạo, đồng thời nâng cao chất lượng phục vụ và khả năng tiếp cận thông tin trong nhà trường. Trên phương diện nghiên cứu, đề tài tập trung vào việc thiết kế cơ chế phối hợp giữa các tác tử, tích hợp mô hình ngôn ngữ lớn với truy xuất tri thức theo ngữ cảnh, và tổ chức luồng xử lý nhằm cân bằng giữa tốc độ phản hồi, độ chính xác và tính an toàn nội dung.

Mục tiêu chính của đề án là nghiên cứu, xây dựng hệ thống chatbot thông minh có khả năng hỗ trợ sinh viên và giảng viên trong việc cung cấp thông tin và giải đáp các thắc mắc liên quan đến đào tạo, dựa trên hướng tiếp cận sử dụng trí tuệ nhân tạo và triển khai dưới dạng website có tích hợp chatbot tiện lợi.

Cấu trúc báo cáo gồm các nội dung chính như sau:

Chương 1: Tổng quan đề tài.

Chương 2: Cơ sở lý thuyết và phương pháp xây dựng hệ thống trợ lý ảo AI đa tác tử.

Chương 3: Phân tích thiết kế hệ thống.

Chương 4: Xây dựng hệ thống và thực nghiệm.

Báo cáo được tổ chức theo hướng từ cơ sở lý thuyết đến triển khai và đánh giá thực

nghiệm, nhằm bảo đảm tính chặt chẽ học thuật và khả năng ứng dụng thực tế. Mặc dù đã nỗ lực hoàn thiện trong phạm vi thời gian và nguồn lực cho phép, đề tài khó tránh khỏi những hạn chế nhất định. Em kính mong nhận được ý kiến góp ý của Quý Thầy/Cô và các bạn để tiếp tục hoàn thiện hệ thống, hướng tới khả năng triển khai rộng rãi trong môi trường giáo dục đại học.

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1 Tổng quan về trợ lý ảo AI trong giáo dục

1.1.1 Khái niệm trợ lý ảo AI

Trợ lý ảo AI (AI Virtual Assistant) là một hệ thống phần mềm được thiết kế để mô phỏng cuộc hội thoại với người dùng thông qua giao diện văn bản hoặc giọng nói, có khả năng hiểu ngôn ngữ tự nhiên và tự động đưa ra phản hồi phù hợp với từng yêu cầu. Mục tiêu cốt lõi của trợ lý ảo AI là tự động hóa quy trình cung cấp thông tin và hỗ trợ người dùng, cho phép tiếp cận dịch vụ một cách trực quan, nhanh chóng mà không cần sự can thiệp trực tiếp của con người. Việc ứng dụng trợ lý ảo AI mang lại nhiều lợi ích đáng kể, bao gồm: nâng cao hiệu quả vận hành, giảm chi phí nhân sự cho các tác vụ lặp lại, cung cấp dịch vụ liên tục 24/7 và cải thiện đáng kể trải nghiệm người dùng.

Dựa trên công nghệ lõi, trợ lý ảo AI có thể được phân loại thành hai nhóm chính: trợ lý ảo truyền thống và trợ lý ảo ứng dụng trí tuệ nhân tạo.

Các hệ thống trợ lý ảo truyền thống sử dụng phương pháp tập luật, hoạt động dựa trên một tập hợp các quy tắc và kịch bản hội thoại được lập trình sẵn. Người phát triển phải dự đoán trước các câu hỏi, từ khóa có thể có từ người dùng và xây dựng một cây quyết định logic để trả về câu trả lời tương ứng. Mặc dù phương pháp này đảm bảo tính nhất quán và có thể kiểm soát hoàn toàn luồng hội thoại, nó bộc lộ nhiều hạn chế cố hữu. Hệ thống phụ thuộc hoàn toàn vào cơ sở tri thức do người thiết kế xây dựng và chỉ có thể xử lý các câu hỏi tuân thủ đúng kịch bản. Khi người dùng diễn đạt câu hỏi theo cách khác hoặc đặt câu hỏi nằm ngoài tập kịch bản định sẵn, hệ thống thường không thể hiểu và đưa ra phản hồi phù hợp. Do đó, phạm vi ứng dụng của loại hệ thống này thường bị giới hạn trong các tác vụ đơn giản, lặp lại với cấu trúc câu hỏi-trả lời rõ ràng.

Để khắc phục những hạn chế của trợ lý ảo truyền thống, thế hệ hệ thống hiện đại tích hợp các thành tựu của Trí tuệ nhân tạo, đặc biệt là trong lĩnh vực Xử lý ngôn ngữ tự nhiên (NLP) và các Mô hình ngôn ngữ lớn (LLMs). Thay vì bám vào các quy tắc cứng, trợ lý ảo AI có khả năng "hiểu" ý định đằng sau câu hỏi của người dùng, cho phép xử lý các truy vấn đa dạng và phức tạp hơn, đồng thời tạo ra các phản hồi tự nhiên, linh hoạt như một cuộc đối thoại thực sự. Đặc biệt, khi được xây dựng trên nền tảng kiến trúc AI Agent, trợ lý ảo không chỉ dừng lại ở việc truy xuất và tái hiện thông tin mà còn có khả năng lập kế hoạch, sử dụng các công cụ bên ngoài như cơ sở dữ liệu hay API, và tự đánh giá kết quả để điều chỉnh hành động, mở ra khả năng xử lý các bài toán phức tạp mà các thế hệ trước không thể đáp ứng.

Trong phạm vi đề tài này, hướng tiếp cận được lựa chọn là xây dựng hệ thống trợ lý ảo AI dựa trên kiến trúc đa tác tử, kết hợp với kỹ thuật truy xuất tri thức tăng cường

(RAG), nhằm khai thác tối đa sức mạnh của các mô hình ngôn ngữ lớn đồng thời đảm bảo độ chính xác và tính nhất quán khi phản hồi các câu hỏi nghiệp vụ đặc thù trong môi trường đại học.

1.1.2 Tiềm năng ứng dụng trong môi trường đại học

Môi trường đại học là một trong những lĩnh vực có tiềm năng ứng dụng trợ lý ảo AI cao nhất, xuất phát từ đặc thù vận hành: khối lượng thông tin lớn và đa dạng về nghiệp vụ, nhu cầu tra cứu lặp lại cao và đối tượng người dùng đồng đẳng với các yêu cầu khác nhau. Tại các trường đại học, đặc biệt là các cơ sở đào tạo có quy mô lớn như Học viện Kỹ thuật Mật mã, các câu hỏi liên quan đến lịch học, lịch thi, chương trình đào tạo, học phí, biểu mẫu hành chính hay quy chế tốt nghiệp được gửi đến các phòng ban chức năng với số lượng lớn và tần suất cao, tạo áp lực đáng kể lên đội ngũ cán bộ trong khi chất lượng và tốc độ phản hồi vẫn chưa đáp ứng được kỳ vọng của người học.

1.1.2.1. Lợi ích của chatbot đối với sinh viên

- **Truy cập thông tin mọi lúc, mọi nơi:** Trợ lý ảo AI có khả năng hoạt động 24/7, giúp sinh viên chủ động tra cứu thông tin học vụ mà không phụ thuộc vào giờ làm việc hành chính, đặc biệt hữu ích trong các thời điểm cao điểm như đầu học kỳ hay mùa thi.
- **Giảm thời gian chờ đợi:** Thay vì xếp hàng tại phòng đào tạo hoặc gửi email và chờ phản hồi, sinh viên nhận được câu trả lời ngay lập tức, từ đó chủ động hơn trong việc lên kế hoạch học tập và xử lý thủ tục hành chính.
- **Thông tin cá nhân hóa:** Khi được tích hợp với dữ liệu học tập nội bộ, hệ thống có khả năng cung cấp thông tin phù hợp với tình trạng riêng của từng sinh viên như điểm thi, số tín chỉ tích lũy, các môn còn thiếu hay điều kiện xét tốt nghiệp.
- **Nâng cao tính tự chủ:** Khả năng tự tra cứu và giải quyết thắc mắc một cách độc lập giúp sinh viên rèn luyện tư duy chủ động, một kỹ năng thiết yếu trong môi trường học tập và làm việc hiện đại.

1.1.2.2. Lợi ích của chatbot đối với giảng viên và cán bộ quản lý

Bên cạnh việc hỗ trợ sinh viên, hệ thống trợ lý ảo AI cũng mang lại nhiều lợi ích rõ rệt cho đội ngũ giảng viên và cán bộ quản lý trong quá trình giảng dạy, tư vấn và vận hành học vụ:

- **Giảm tải câu hỏi lặp lại:** Trợ lý ảo AI tự động xử lý phần lớn các câu hỏi thường gặp, giải phóng thời gian của giảng viên và cán bộ để tập trung vào các công việc chuyên môn đòi hỏi năng lực con người như nghiên cứu, tư vấn chuyên sâu hay hỗ trợ tình huống phức tạp.
- **Tăng hiệu quả vận hành:** Khi được tích hợp với hệ thống quản lý đào tạo, trợ

lý ảo có thể hỗ trợ cán bộ theo dõi tiến độ xử lý hồ sơ và cung cấp thông tin tức thời về tình hình học tập của sinh viên, góp phần nâng cao chất lượng quản trị.

- **Cải thiện trải nghiệm phục vụ:** Nhờ hệ thống xử lý các yêu cầu cơ bản một cách tự động và nhất quán, cán bộ có thêm nguồn lực để nâng cao chất lượng tương tác với sinh viên trong các vấn đề phức tạp, từ đó cải thiện mức độ hài lòng của người học.
- **Khả năng mở rộng không giới hạn:** Trong khi năng lực phục vụ của đội ngũ nhân sự bị giới hạn bởi số lượng người và giờ làm việc, hệ thống trợ lý ảo AI có thể phục vụ đồng thời hàng nghìn người dùng mà không ảnh hưởng đến chất lượng phản hồi, đáp ứng nhu cầu tăng trưởng quy mô mà không cần mở rộng tuyến tính nguồn nhân lực.

1.1.3 Khảo sát một số hệ thống trợ lý ảo hiện có

Trên thế giới, xu hướng triển khai trợ lý ảo trong giáo dục đại học đã phát triển mạnh trong khoảng một thập kỷ trở lại đây, từ các chatbot hỗ trợ tuyển sinh đơn giản đến các hệ thống tích hợp sâu với LMS và cổng thông tin sinh viên. Tuy nhiên, tại Việt Nam, ứng dụng công nghệ này trong môi trường đại học vẫn đang ở giai đoạn đầu.

Hiện nay, một số trường đại học trong nước đã thử nghiệm chatbot tích hợp trên website hoặc nền tảng nhắn tin. Tuy nhiên, phần lớn các hệ thống này hoạt động theo cơ chế tập luật hoặc phân loại ý định đơn giản, chỉ có khả năng trả lời các câu hỏi thường gặp được lập trình sẵn. Khi sinh viên đặt câu hỏi theo cách diễn đạt khác, đặt câu hỏi đa ý định hoặc yêu cầu tổng hợp thông tin từ nhiều nghiệp vụ, các hệ thống này thường không xử lý được hoặc trả về kết quả thiếu chính xác. Ngoài ra, hầu hết chưa được tích hợp với dữ liệu nội bộ của nhà trường như hệ thống quản lý đào tạo hay bảng điểm, khiến thông tin cung cấp còn hạn chế về độ chi tiết và tính cập nhật.

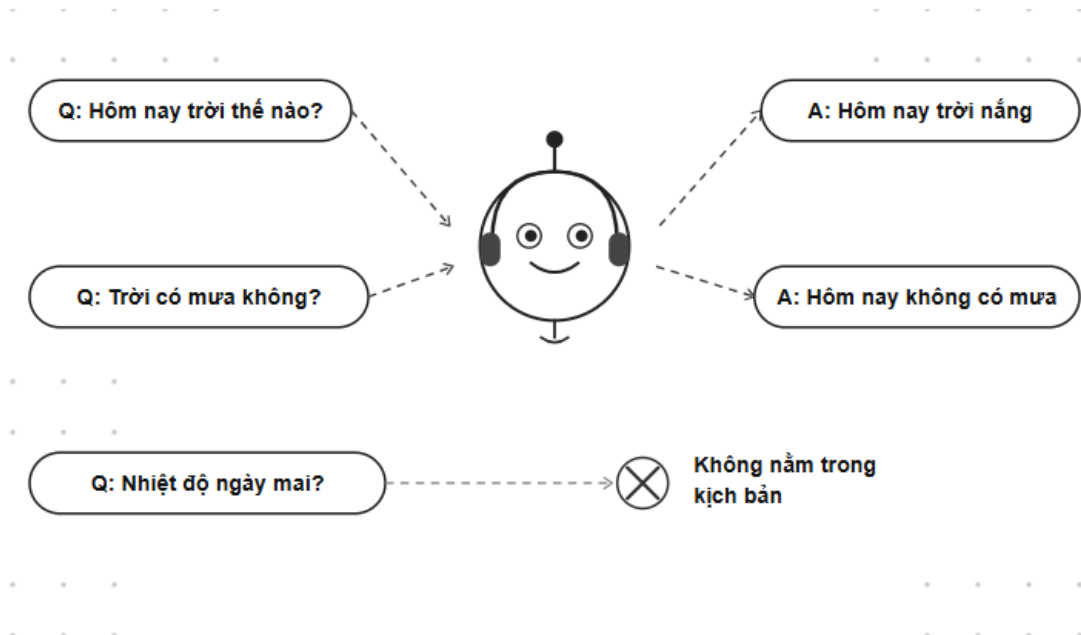
Từ thực trạng đó, có thể xác định rõ khoảng trống bài toán: các trường đại học cần một hệ thống trợ lý ảo có khả năng hiểu ngôn ngữ tự nhiên linh hoạt, truy xuất thông tin chính xác từ dữ liệu nội bộ, xử lý được câu hỏi đa nghiệp vụ phức tạp và vận hành ổn định với kiến trúc có thể mở rộng. Đây là bài toán trung tâm mà đề tài hướng tới giải quyết, với Học viện Kỹ thuật Mật mã là môi trường thực nghiệm điển hình để kiểm chứng tính hiệu quả của kiến trúc đề xuất.

1.2 Các phương pháp xây dựng hệ thống trợ lý ảo

Việc xây dựng hệ thống trợ lý ảo AI có thể được tiếp cận theo nhiều phương pháp khác nhau, mỗi phương pháp phản ánh một giai đoạn phát triển của công nghệ xử lý ngôn ngữ tự nhiên và trí tuệ nhân tạo. Về cơ bản, các phương pháp xây dựng hệ thống trợ lý ảo hiện nay có thể được phân thành ba nhóm chính: hệ thống dựa trên tập luật, hệ thống dựa trên mô hình ngôn ngữ lớn và hệ thống dựa trên AI Agent.

1.2.1 Hệ thống dựa trên tập luật

Phương pháp xây dựng hệ thống trợ lý ảo dựa trên tập luật là cách tiếp cận sơ khai và nền tảng nhất. Hoạt động của các hệ thống này hoàn toàn phụ thuộc vào một kịch bản và bộ quy tắc được lập trình sẵn. Người phát triển phải dự đoán trước các câu hỏi, từ khóa có thể có từ người dùng và xây dựng một cây quyết định logic để đưa ra câu trả lời tương ứng.



Hình 1.1 Hệ thống dựa trên tập luật khi đầu vào không nằm trong kịch bản

Mặc dù phương pháp này đảm bảo tính nhất quán và có thể kiểm soát hoàn toàn luồng hội thoại, nó bộc lộ nhiều hạn chế cố hữu. Hệ thống thiếu đi sự linh hoạt, không có khả năng xử lý các câu hỏi phức tạp, sai chính tả hoặc các truy vấn không nằm trong kịch bản định sẵn. Do đó, phạm vi ứng dụng của hệ thống dựa trên tập luật thường bị giới hạn trong các tác vụ đơn giản như cung cấp thông tin theo mẫu hoặc trả lời các câu hỏi thường gặp có cấu trúc rõ ràng. Khi quy mô nghiệp vụ mở rộng, chi phí bảo trì và cập nhật tập luật cũng tăng theo một cách phi tuyến, trở thành rào cản lớn trong thực tế vận hành.

1.2.2 Hệ thống dựa trên mô hình ngôn ngữ lớn

Để khắc phục những giới hạn của cách tiếp cận dựa trên tập luật, các hệ thống trợ lý ảo thế hệ mới ứng dụng các Mô hình ngôn ngữ lớn (LLMs). Thay vì các quy tắc cứng, hệ thống dựa trên LLMs được huấn luyện trên lượng dữ liệu khổng lồ, có khả năng tự nhận dạng mẫu, hiểu ý định và sinh ra câu trả lời hoàn toàn mới, mang lại các cuộc hội thoại tự nhiên và linh hoạt hơn rất nhiều so với hệ thống truyền thống.

Tuy nhiên, khi áp dụng vào môi trường nghiệp vụ cụ thể như trường đại học, hệ

thống thuần LLMs gặp phải ba vấn đề cốt lõi cần giải quyết:

- **Thiếu kiến thức chuyên biệt theo miền (Non-specific domain):** Các mô hình LLMs được đào tạo trên dữ liệu đại trà, không có thông tin nội bộ đặc thù của từng tổ chức như lịch thi cụ thể, danh sách môn học hay quy chế đào tạo riêng của từng trường.
- **Không tiếp cận được dữ liệu riêng tư (Non-access private data):** Vì lý do bảo mật, dữ liệu nội bộ của tổ chức không thể đưa vào quá trình huấn luyện mô hình. Điều này khiến hệ thống không thể phản hồi chính xác những câu hỏi liên quan đến thông tin nội bộ của nhà trường.
- **Hiện tượng ảo giác (Hallucination):** Khi thiếu thông tin liên quan, các mô hình LLMs có xu hướng tự sinh ra nội dung sai lệch nhưng được trình bày với giọng văn tự tin và mạch lạc. Đây là vấn đề đặc biệt nguy hiểm trong các ứng dụng giáo dục, nơi độ chính xác thông tin ảnh hưởng trực tiếp đến quyết định học tập và hành chính của sinh viên.

Để giải quyết những vấn đề trên, kỹ thuật Truy xuất tạo sinh tăng cường (RAG – Retrieval-Augmented Generation) được đề xuất và nhanh chóng trở thành tiêu chuẩn cho các hệ thống trợ lý ảo nghiệp vụ. RAG cho phép LLMs tiếp cận dữ liệu nội bộ của tổ chức một cách an toàn và giảm thiểu đáng kể hiện tượng ảo giác, tuy nhiên vẫn bị giới hạn ở mức truy vấn-phản hồi một chiều khi xử lý các câu hỏi phức tạp, đa ý định.

1.2.3 Hệ thống dựa trên AI Agent

Kiến trúc AI Agent ra đời như một bước phát triển tất yếu nhằm vượt qua giới hạn của hệ thống RAG đơn thuần. Một AI Agent không chỉ truy xuất thông tin và sinh câu trả lời mà còn có khả năng lập kế hoạch, sử dụng công cụ và tự đánh giá kết quả để quyết định hành động tiếp theo, tạo thành một vòng lặp suy luận – hành động – quan sát cho đến khi bài toán được giải quyết thỏa đáng.

Khi áp dụng vào bài toán trợ lý ảo đại học, kiến trúc Agent cho phép hệ thống xử lý hiệu quả các tình huống mà RAG đơn thuần không đáp ứng được, bao gồm: câu hỏi đa ý định cần phân rã thành nhiều truy vấn con, câu hỏi yêu cầu tổng hợp thông tin từ nhiều nghiệp vụ khác nhau, hoặc câu hỏi cần lặp lại quá trình truy xuất và kiểm chứng nhiều lần trước khi đưa ra kết luận cuối cùng.

Bước phát triển tiếp theo là kiến trúc đa tác tử (Multi-Agent System), trong đó nhiều Agent được tổ chức phối hợp với nhau theo phân công vai trò rõ ràng. Mỗi tác tử chuyên trách một nhiệm vụ cụ thể như kiểm soát phạm vi câu hỏi, viết lại truy vấn, lập kế hoạch, truy xuất tri thức theo từng nghiệp vụ hay tổng hợp kết quả cuối cùng. Cách tổ chức này không chỉ nâng cao chất lượng phản hồi mà còn tăng khả năng mở rộng và bảo trì hệ thống khi nghiệp vụ phát triển.

Trong đề tài này, hướng tiếp cận được lựa chọn là xây dựng hệ thống trợ lý ảo theo kiến trúc đa tác tử kết hợp RAG, với Học viện Kỹ thuật Mật mã là môi trường triển khai thực nghiệm.

Tiêu chí	Tập luật	LLMs + RAG	AI Agent đa tác tử
Xử lý câu hỏi ngoài kịch bản	Không	Có	Có
Sử dụng dữ liệu nội bộ	Có (thủ công)	Có (tự động)	Có (tự động)
Xử lý câu hỏi đa ý định	Không	Hạn chế	Có
Lập kế hoạch đa bước	Không	Không	Có
Khả năng mở rộng nghiệp vụ	Thấp	Trung bình	Cao
Độ phức tạp triển khai	Thấp	Trung bình	Cao

Bảng 1.1 So sánh các phương pháp xây dựng hệ thống trợ lý ảo

1.3 Mô hình ngôn ngữ lớn

1.3.1 Khái niệm và kiến trúc

Mô hình ngôn ngữ lớn (Large Language Models – LLMs) là các mô hình học sâu được huấn luyện trên lượng dữ liệu văn bản khổng lồ thu thập từ nhiều nguồn khác nhau, với mục tiêu học cách hiểu và tạo sinh ngôn ngữ tự nhiên. Nhờ quy mô tham số lên đến hàng chục hoặc hàng trăm tỷ, LLMs có khả năng nắm bắt các mối quan hệ ngữ nghĩa phức tạp, suy luận theo ngữ cảnh và thực hiện đa dạng các tác vụ ngôn ngữ như trả lời câu hỏi, tóm tắt văn bản, dịch thuật hay lập trình mà không cần được tinh chỉnh riêng cho từng nhiệm vụ.



Hình 1.2 Mô hình ngôn ngữ lớn (Large Language Model)

Nền tảng kiến trúc của phần lớn các LLMs hiện đại là Transformer, được giới thiệu lần đầu bởi Vaswani và cộng sự năm 2017. Điểm cốt lõi của kiến trúc này là cơ chế chú ý (Attention Mechanism), cho phép mô hình xác định mức độ liên quan của từng từ trong chuỗi đầu vào khi dự đoán từ tiếp theo, thay vì xử lý tuần tự như các kiến trúc trước đó. Nhờ đó, mô hình có khả năng nắm bắt các phụ thuộc ngữ nghĩa ở khoảng cách xa trong văn bản, tạo ra phản hồi mạch lạc và bám sát ngữ cảnh người dùng.

Quá trình một LLMs tạo sinh câu trả lời trải qua các bước chính sau:

- **Mã hóa đầu vào (Tokenization & Embedding):** Văn bản đầu vào được tách thành các đơn vị nhỏ gọi là token, sau đó mỗi token được chuyển thành vector số học (embedding) mang thông tin ngữ nghĩa và vị trí trong câu.
- **Xử lý qua các lớp Transformer:** Các vector embedding được truyền qua nhiều lớp Transformer xếp chồng nhau. Tại mỗi lớp, cơ chế chú ý tính toán mức độ tương tác giữa các token, trong khi các lớp truyền tiếp (feedforward) thực hiện các biến đổi phi tuyến để mô hình học được các đặc trưng ngôn ngữ phức tạp hơn.
- **Dự đoán đầu ra:** Dựa trên biểu diễn ngữ cảnh đã được xử lý, mô hình dự đoán xác suất của từng token tiếp theo và lặp lại quá trình cho đến khi hoàn thành câu trả lời.

Chất lượng của LLMs phụ thuộc vào hai yếu tố chính: quy mô và chất lượng dữ liệu huấn luyện, cùng với kỹ thuật tinh chỉnh theo phản hồi của con người (RLHF – Reinforcement Learning from Human Feedback) nhằm căn chỉnh hành vi mô hình phù hợp với mục đích sử dụng thực tế. Hiện nay, các LLMs tiêu biểu và được ứng dụng

rộng rãi nhất bao gồm GPT-4o, GPT-4.1 của OpenAI, Gemini của Google và Claude của Anthropic. Trong đề tài này, hệ thống sử dụng các mô hình GPT của OpenAI thông qua API chính thức, tận dụng năng lực ngôn ngữ tiên tiến của các mô hình này mà không cần tự thực hiện quá trình huấn luyện tốn kém về tài nguyên.

Ngoài năng lực sinh ngôn ngữ, một thành phần quan trọng không kém trong hệ thống là mô hình embedding. Embedding là quá trình chuyển đổi đoạn văn bản thành vector số học trong không gian nhiều chiều, sao cho các đoạn văn bản có nghĩa tương đồng sẽ có vector gần nhau theo khoảng cách cosine. Trong đề tài này, mô hình text-embedding-3-small của OpenAI được sử dụng để embedding toàn bộ tài liệu và truy vấn của người dùng, tạo nền tảng cho bước truy xuất ngữ nghĩa trong kiến trúc RAG.

1.3.2 Hiện tượng ảo giác và hướng giải quyết

Mặc dù sở hữu năng lực ngôn ngữ vượt trội, LLMs không tránh khỏi một hạn chế nghiêm trọng được gọi là hiện tượng ảo giác (Hallucination): mô hình có xu hướng tự sinh ra thông tin sai lệch, không có thực hoặc mâu thuẫn với sự thật, nhưng lại trình bày với giọng văn tự tin, mạch lạc khiến người dùng khó nhận ra. Nguyên nhân cốt lõi là do LLMs hoạt động theo cơ chế dự đoán token có xác suất cao nhất dựa trên ngữ cảnh, chứ không có cơ chế kiểm chứng thực tế tích hợp. Khi gặp câu hỏi về thông tin ngoài phạm vi huấn luyện hoặc thông tin nội bộ của tổ chức, mô hình vẫn cố gắng sinh ra câu trả lời trông có vẻ hợp lý thay vì thừa nhận không biết.

Trong môi trường đại học, hiện tượng này đặc biệt nguy hiểm. Một câu trả lời sai về ngày thi, điều kiện đăng ký môn học hay quy trình nộp hồ sơ có thể gây hậu quả trực tiếp đến tiến độ học tập và quyền lợi của sinh viên. Đây là lý do then chốt khiến việc sử dụng LLMs thuần túy không phù hợp cho bài toán trợ lý ảo nghiệp vụ.

Hướng giải quyết được áp dụng phổ biến hiện nay là kỹ thuật RAG (Retrieval-Augmented Generation): thay vì để mô hình tự sinh câu trả lời hoàn toàn từ tri thức nội tại, hệ thống trước tiên truy xuất các đoạn tài liệu liên quan từ cơ sở tri thức bên ngoài, sau đó cung cấp các đoạn này như ngữ cảnh bổ sung để mô hình sinh phản hồi dựa trên đó. Cách tiếp cận này “neo” câu trả lời vào dữ liệu thực tế của tổ chức, giảm thiểu đáng kể hiện tượng ảo giác đồng thời cho phép hệ thống tiếp cận được tri thức nội bộ mà không vi phạm yêu cầu bảo mật.

1.4 Kỹ thuật RAG (Retrieval-Augmented Generation)

1.4.1 Khái niệm và kiến trúc RAG cơ bản

Truy xuất tạo sinh tăng cường (Retrieval-Augmented Generation – RAG) là một kỹ thuật kết hợp giữa hệ thống truy xuất thông tin (Information Retrieval) và mô hình ngôn ngữ lớn, nhằm cung cấp cho LLMs ngữ cảnh bên ngoài phù hợp trước khi sinh câu trả

lời. Thay vì chỉ dựa vào tri thức nội tại được học trong quá trình huấn luyện, hệ thống RAG chủ động truy xuất các đoạn tài liệu liên quan từ cơ sở tri thức riêng của tổ chức và đưa chúng vào prompt như ngữ cảnh bổ sung, qua đó “neo” câu trả lời vào dữ liệu thực tế thay vì để mô hình tự suy diễn.

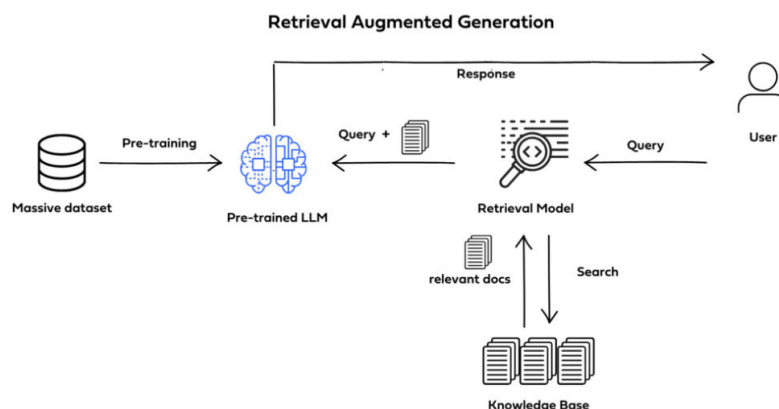
Một hệ thống RAG hoàn chỉnh bao gồm hai giai đoạn chính:

Giai đoạn nạp dữ liệu (Indexing): Toàn bộ tài liệu trong cơ sở tri thức được xử lý và lưu trữ trước để phục vụ truy xuất sau này. Quy trình này bao gồm các bước:

- **Phân đoạn (Chunking):** Tài liệu gốc được chia nhỏ thành các đoạn văn bản có kích thước phù hợp. Việc phân đoạn cần cân bằng giữa hai yếu tố: đoạn quá ngắn sẽ mất ngữ cảnh, đoạn quá dài sẽ làm loãng thông tin liên quan và tốn kém chi phí xử lý.
- **Embedding:** Mỗi đoạn văn bản được chuyển đổi thành vector số học nhiều chiều bằng mô hình embedding. Các vector này mang thông tin ngữ nghĩa của đoạn văn bản tương ứng, sao cho các đoạn có nội dung tương đồng sẽ có vector gần nhau trong không gian véc-tơ.
- **Lưu trữ:** Các vector embedding cùng với nội dung gốc và metadata đi kèm được lưu vào cơ sở dữ liệu vector (Vector Database) để phục vụ tìm kiếm nhanh.

Giai đoạn truy xuất và sinh câu trả lời (Retrieval & Generation): Khi người dùng đặt câu hỏi, hệ thống thực hiện theo trình tự:

- **Embedding truy vấn:** Câu hỏi của người dùng được embedding bằng cùng mô hình đã dùng trong giai đoạn nạp dữ liệu.
- **Tìm kiếm ngữ nghĩa:** Vector truy vấn được so sánh với toàn bộ vector trong cơ sở dữ liệu để xác định các đoạn tài liệu có độ tương đồng ngữ nghĩa cao nhất.
- **Sinh câu trả lời:** Các đoạn tài liệu được truy xuất sẽ được ghép vào prompt cùng với câu hỏi gốc và gửi đến LLM. Mô hình dựa vào ngữ cảnh được cung cấp để sinh ra câu trả lời chính xác, có trích dẫn nguồn thay vì tự suy diễn.



Hình 1.3 Mô hình RAG (Retrieval-Augmented Generation)

1.4.2 Vector Database và tìm kiếm ngữ nghĩa

Vector Database là loại cơ sở dữ liệu chuyên biệt được thiết kế để lưu trữ và tìm kiếm hiệu quả trên không gian vector nhiều chiều. Khác với cơ sở dữ liệu quan hệ truyền thống hoạt động dựa trên so khớp chính xác, Vector Database cho phép tìm kiếm theo độ tương đồng ngữ nghĩa – tức là truy xuất các vector gần nhất với vector truy vấn theo một độ đo khoảng cách nhất định, phổ biến nhất là độ tương đồng cosine.

Độ tương đồng cosine giữa hai vector **a** và **b** được tính theo công thức:

$$\cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|}$$

Giá trị này nằm trong khoảng $[-1, 1]$, trong đó giá trị càng gần 1 cho thấy hai vector càng tương đồng về mặt ngữ nghĩa. Trong thực tế, tìm kiếm vector được tăng tốc bằng các thuật toán tìm kiếm láng giềng gần đúng (Approximate Nearest Neighbor – ANN) như HNSW (Hierarchical Navigable Small World), cho phép truy xuất với độ trễ thấp ngay cả trên tập dữ liệu hàng triệu vector.

Trong đề tài này, Qdrant được lựa chọn làm Vector Database. Qdrant là một vector database mã nguồn mở hiệu năng cao, hỗ trợ lưu trữ vector kết hợp với payload metadata phong phú, cho phép lọc kết hợp giữa tìm kiếm ngữ nghĩa và các điều kiện metadata như loại tài liệu hay nghiệp vụ. Bên cạnh đó, Qdrant cung cấp khả năng tạo chỉ mục payload (payload index) giúp tăng tốc đáng kể các truy vấn có điều kiện lọc, đồng thời hỗ trợ triển khai linh hoạt theo cả chế độ nhúng (embedded) lẫn dịch vụ độc lập — phù hợp với yêu cầu vận hành của hệ thống đề xuất trong đề tài.

1.4.3 Agentic RAG

Kiến trúc RAG cơ bản hoạt động hiệu quả với các câu hỏi đơn giản, có thể trả lời từ một hoặc vài đoạn tài liệu liên quan. Tuy nhiên, trong thực tế, người dùng thường đặt ra những câu hỏi phức tạp hơn đòi hỏi nhiều lượt truy xuất, kiểm chứng và tổng hợp thông tin. Để giải quyết hạn chế này, kiến trúc Agentic RAG được đề xuất nhằm đưa khả năng lập kế hoạch và tự đánh giá của AI Agent vào vòng lặp truy xuất.

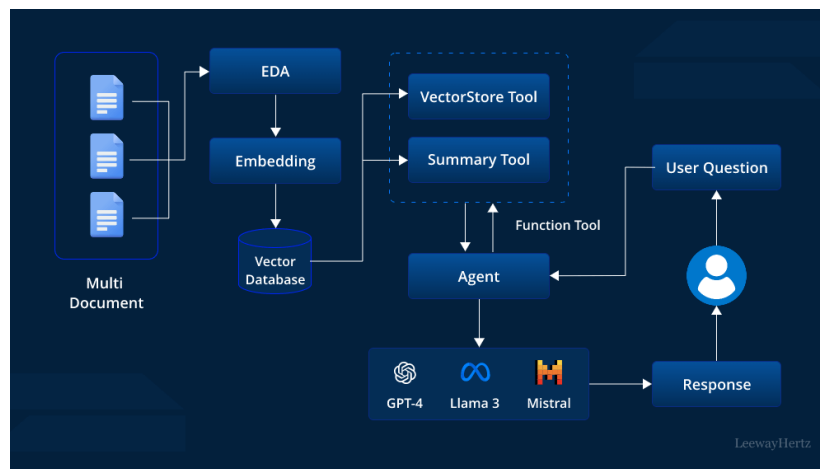
Thay vì thực hiện một lượt truy xuất duy nhất rồi sinh câu trả lời ngay, Agentic RAG tổ chức quá trình xử lý thành một vòng lặp suy luận có kiểm soát bao gồm các bước:

- **Lập kế hoạch truy vấn:** Agent phân tích câu hỏi và xác định cần truy xuất những thông tin gì, có thể phân rã thành nhiều truy vấn con nếu câu hỏi đa ý định.
- **Truy xuất:** Thực hiện tìm kiếm ngữ nghĩa trên Vector Database với từng truy vấn đã lập kế hoạch.
- **Đánh giá độ liên quan (Document Grading):** Mỗi đoạn tài liệu truy xuất được đánh giá xem có thực sự liên quan và đủ để trả lời câu hỏi hay không. Bước này loại

bỏ nhiễu và ngắn mô hình sinh câu trả lời dựa trên tài liệu không phù hợp.

- **Viết lại truy vấn và truy xuất lại (Query Rewriting & Re-retrieval):** Nếu các tài liệu truy xuất được đánh giá là chưa đủ liên quan, Agent tự động cải thiện truy vấn và thực hiện lại bước tìm kiếm, lặp cho đến khi thu thập được đủ ngữ cảnh tin cậy.
- **Tổng hợp và sinh câu trả lời:** Khi ngữ cảnh đã được kiểm chứng là đầy đủ, LLM tổng hợp thông tin và sinh ra câu trả lời cuối cùng.

Trong đề tài này, Agentic RAG được áp dụng cho các câu hỏi được đánh giá có độ phức tạp cao, trong khi các câu hỏi đơn giản hơn vẫn xử lý theo kiến trúc RAG cơ bản nhằm tối ưu hóa thời gian phản hồi. Cơ chế đánh giá độ phức tạp và lựa chọn pipeline phù hợp là một trong những đặc điểm nổi bật của kiến trúc đa tác tử được đề xuất trong đề tài.



Hình 1.4 Kiến trúc Agentic RAG

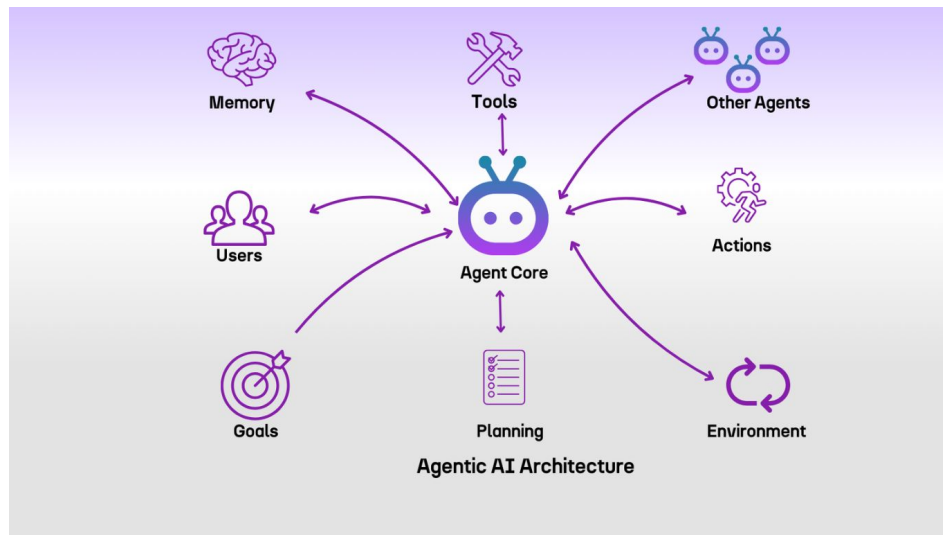
1.5 AI Agent và hệ thống đa tác tử

1.5.1 Khái niệm AI Agent

Một tác tử (Agent) là bất kỳ thực thể nào có khả năng nhận thức môi trường xung quanh và hành động tác động lên môi trường đó để đạt được mục tiêu nhất định. Cuốn sách *Artificial Intelligence: A Modern Approach* (1995) định nghĩa một tác tử là bất cứ thứ gì có thể xem là có khả năng nhận thức môi trường của nó thông qua các cảm biến và tác động lên môi trường đó thông qua các bộ truyền động. Một Agent được đặc trưng bởi môi trường mà nó hoạt động và tập hợp các hành động mà Agent có thể tác động lên môi trường đó.

Tác tử thông minh (AI Agent) là Agent được xây dựng trên nền tảng các kỹ thuật trí tuệ nhân tạo. Môi trường của AI Agent là bối cảnh, yêu cầu và tài liệu được đưa vào bởi người dùng hoặc các tài nguyên hệ thống khác. Các tác động của AI Agent lên môi trường được mở rộng và tăng cường thông qua các công cụ mà nó được trao quyền truy cập.

Điểm khác biệt cốt lõi giữa AI Agent và một mô hình LLMs thông thường nằm ở vòng lặp hành động: thay vì chỉ sinh ra một câu trả lời duy nhất dựa trên đầu vào, AI Agent có khả năng lặp lại chu trình nhận thức – suy luận – hành động – quan sát cho đến khi nhiệm vụ được hoàn thành thỏa đáng. AI Agent không chỉ tận dụng được sức mạnh ngôn ngữ của LLMs mà còn khắc phục được những điểm yếu của chúng, tạo ra một thể hệ thống thông minh có thể xử lý các bài toán phức tạp, đa bước mà một lượt sinh ngôn ngữ đơn lẻ không thể giải quyết.

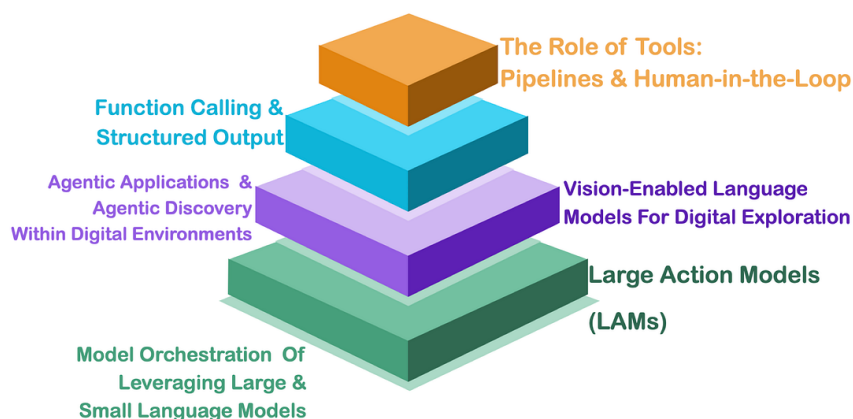


Hình 1.5 Kiến trúc tổng quát của AI Agent

1.5.2 Các thành phần của AI Agent

Kiến trúc của một AI Agent hiện đại bao gồm sự tích hợp tinh vi của nhiều thành phần phối hợp với nhau, cho phép Agent có khả năng nhận thức, tư duy, lập kế hoạch và hành động.

The Emergence Of AI Agent Architecture



www.cobusgreyling.com

Hình 1.6 Các thành phần trong kiến trúc AI Agent hiện đại

1.5.2.1. Cơ chế nhận thức

Cơ chế nhận thức là giao diện cho phép Agent thu thập và xử lý thông tin từ môi trường bên ngoài. Trong các Agent dựa trên ngôn ngữ, nhận thức chủ yếu bao gồm các mô-đun hiểu ngôn ngữ tự nhiên, trích xuất ý định và thực thể liên quan, chuyển đổi văn bản phi cấu trúc thành các đối tượng có cấu trúc để xử lý tiếp. Khả năng nhận thức mạnh mẽ là nền tảng cho hoạt động hiệu quả của toàn bộ Agent, vì mọi quyết định và hành động đều bắt đầu từ chất lượng thông tin được nhận thức.

1.5.2.2. Hệ thống biểu diễn tri thức

Hệ thống biểu diễn tri thức cung cấp các cấu trúc và cơ chế để lưu trữ, tổ chức và truy xuất thông tin bên trong Agent. Một Agent hiệu quả cần phân biệt và quản lý nhiều dạng tri thức khác nhau, bao gồm:

- **Context:** Các sự kiện và thông tin về môi trường, ngữ cảnh hội thoại hiện tại.
- **Task:** Mô tả nhiệm vụ cụ thể Agent cần thực hiện.
- **Steps:** Tri thức thủ tục về cách thực hiện từng bước trong quy trình.
- **Experiments:** Những kinh nghiệm, kết quả quan sát từ các bước đã thực thi trước đó.
- **Meta data:** Siêu tri thức – nhận thức về vai trò, khả năng và giới hạn của chính Agent.

1.5.2.3. Mô-đun suy luận và ra quyết định

Mô-đun suy luận và ra quyết định cho phép Agent xử lý thông tin có sẵn, đánh giá các phương án hành động và lựa chọn bước tiếp theo phù hợp nhất. Đây là thành phần trung tâm quyết định chất lượng của Agent. Trong các hệ thống hiện đại, năng lực suy luận được hiện thực hóa thông qua các mô hình LLMs có khả năng tư duy (thinking), điển hình như GPT-4o hay các mô hình reasoning chuyên biệt, kết hợp với các kỹ thuật prompting như Chain-of-Thought hay ReAct để cấu trúc hóa quá trình suy luận.

1.5.2.4. Thành phần thực thi

Thành phần thực thi chuyển đổi các quyết định thành hành vi cụ thể tác động lên môi trường. Các hành động của Agent có thể bao gồm:

- Sinh câu trả lời văn bản cho người dùng.
- Gọi công cụ tìm kiếm tài liệu trong cơ sở tri thức.
- Truy vấn cơ sở dữ liệu để lấy thông tin cụ thể.
- Gọi API bên ngoài để tích hợp với hệ thống khác.
- Đặt câu hỏi làm rõ thêm thông tin với người dùng.

1.5.2.5. Các thành phần khác

Ngoài các thành phần cốt lõi, AI Agent hiện đại thường bổ sung thêm:

- **Mô-đun quản lý bộ nhớ:** Duy trì thông tin qua nhiều lượt tương tác, cho phép Agent nhớ ngữ cảnh hội thoại và học hỏi từ các bước đã thực thi để đưa ra quyết định tốt hơn ở các lượt sau.
- **Thành phần tự phản chiếu và sửa lỗi:** Cho phép Agent đánh giá hiệu suất của chính mình, nhận ra các giới hạn và điều chỉnh phương pháp tiếp cận khi kết quả trung gian không đạt yêu cầu.
- **Cơ chế an toàn và kiểm soát phạm vi:** Đảm bảo hành vi của Agent vẫn nằm trong giới hạn phù hợp, từ chối xử lý các yêu cầu nằm ngoài phạm vi chức năng hoặc vi phạm chính sách an toàn nội dung.

1.5.3 Công cụ trong AI Agent

Một Agent không nhất thiết cần truy cập vào các công cụ bên ngoài. Tuy nhiên nếu không có công cụ, khả năng của Agent sẽ bị giới hạn hoàn toàn trong phạm vi sinh ngôn ngữ. Chính các công cụ bên ngoài mới là thứ biến Agent thành một thực thể có thể tương tác với thế giới thực và hoàn thành các tác vụ có giá trị thực tiễn.

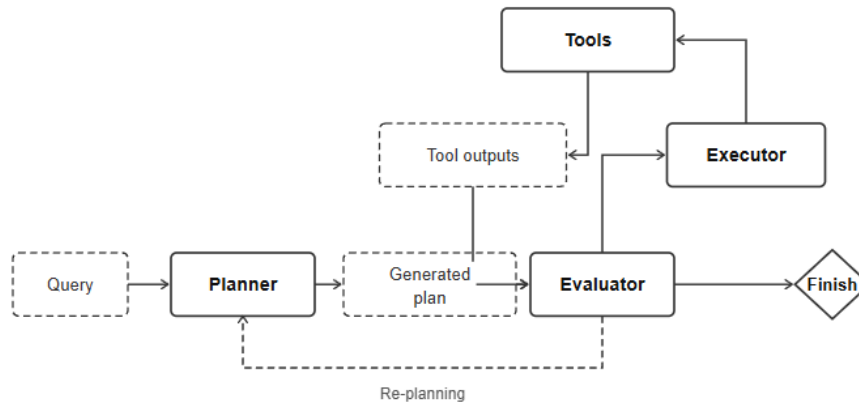
Các công cụ giúp Agent vừa nhận biết môi trường vừa hành động dựa trên nó. Phân theo chiều tác động, công cụ chia thành hai loại:

- **Công cụ đọc:** Cho phép Agent thu thập thông tin từ môi trường, ví dụ: tìm kiếm tài liệu trong Vector Database, tra cứu lịch thi hay truy vấn bảng điểm theo mã số sinh viên.
- **Công cụ ghi:** Cho phép Agent tác động lên môi trường, ví dụ: tạo báo cáo tổng hợp, ghi nhận phiên hội thoại hay kích hoạt quy trình xử lý tự động.

Các Agent thường lựa chọn công cụ phù hợp trong quá trình lập kế hoạch dựa trên mô tả về khả năng và phạm vi của từng công cụ. Ví dụ, một Agent hỗ trợ tra cứu lịch thi sẽ cần công cụ truy vấn cơ sở dữ liệu lịch thi (đọc) thay vì công cụ tìm kiếm tài liệu văn bản thông thường.

1.5.4 Lập kế hoạch trong AI Agent

Lập kế hoạch là năng lực cốt lõi phân biệt AI Agent với chatbot thông thường. Các AI Agent có thể phải giải quyết những nhiệm vụ rất phức tạp – ví dụ: “Liệt kê tất cả các môn học tôi chưa đạt và cho biết lịch thi lại gần nhất của từng môn.” Một nhiệm vụ như vậy đòi hỏi Agent phải thực hiện nhiều bước liên tiếp: tra cứu kết quả học tập, xác định các môn chưa đạt, tìm kiếm lịch thi lại cho từng môn và tổng hợp kết quả.



Hình 1.7 Quy trình lập kế hoạch và tái lập kế hoạch trong AI Agent

Để hoàn thành được các nhiệm vụ phức tạp, Agent thực hiện quá trình lập kế hoạch bao gồm:

- Hiểu rõ nhiệm vụ cần thực hiện và các ràng buộc kèm theo.
- Xem xét các công cụ và tri thức có sẵn.
- Lựa chọn trình tự hành động tối ưu với lượng tài nguyên tiết kiệm nhất.

Tuy nhiên, ngay cả kế hoạch tốt nhất cũng có thể gặp tình huống ngoài dự kiến. Do đó, Agent cần có khả năng tái lập kế hoạch (re-planning): sau mỗi bước thực thi, Agent quan sát kết quả và đánh giá xem kế hoạch hiện tại có còn phù hợp không. Nếu kết quả trung gian không như kỳ vọng, Agent điều chỉnh kế hoạch dựa trên những thông tin mới thu thập được và tiếp tục thực hiện cho đến khi đạt mục tiêu.

1.5.5 Tự đánh giá và sửa lỗi

Như đã trình bày, ngay cả những kế hoạch tốt nhất cũng cần được đánh giá và điều chỉnh liên tục để tối đa hóa độ chính xác của kết quả. Quá trình tự đánh giá này được gọi là Reflection. Tuy không hoàn toàn bắt buộc để Agent hoạt động, Reflection là yếu tố cần thiết để Agent đạt được chất lượng cao nhất trong thực tế.

Có nhiều thời điểm trong quá trình thực thi mà việc tự đánh giá có thể phát huy tác dụng:

- Sau khi nhận câu hỏi của người dùng để xác định yêu cầu có khả thi và nằm trong phạm vi xử lý không.
- Sau khi tạo kế hoạch ban đầu để đánh giá tính hợp lý trước khi thực thi.
- Sau mỗi bước thực thi để kiểm tra kết quả có đi đúng hướng không.
- Sau khi toàn bộ kế hoạch thực thi xong để xác nhận nhiệm vụ đã hoàn thành đầy đủ chưa.

Khuôn mẫu ReAct (Reasoning + Acting) được Yao và cộng sự đề xuất năm 2022 là một trong những cách hiệu quả nhất để cấu trúc hóa vòng lặp này. Trong khuôn mẫu ReAct, Agent xen kẽ liên tục giữa việc diễn giải suy nghĩ (Thought), thực hiện hành động (Action) và quan sát kết quả (Observation) cho đến khi nhiệm vụ hoàn thành. Output của một Agent theo khuôn mẫu ReAct thường có dạng:

Thought 1: [Suy luận về bước cần thực hiện]

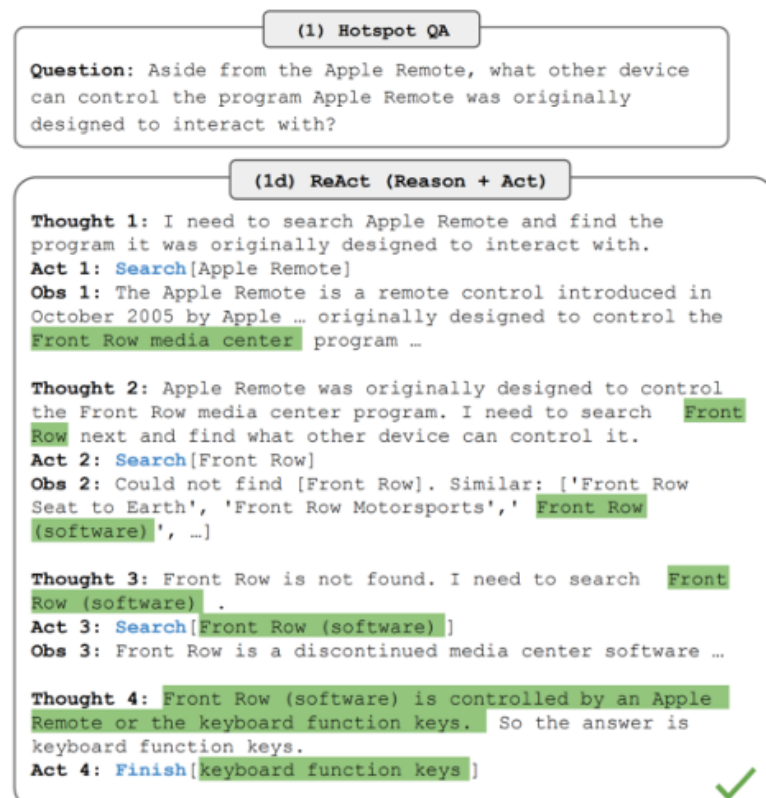
Action 1: [Hành động được chọn và tham số]

Observation 1: [Kết quả quan sát được]

...

Thought N: [Đánh giá kết quả cuối]

Action N: Finish [Câu trả lời hoàn chỉnh]



Hình 1.8 Khuôn mẫu ReAct trong quá trình suy luận và hành động của Agent

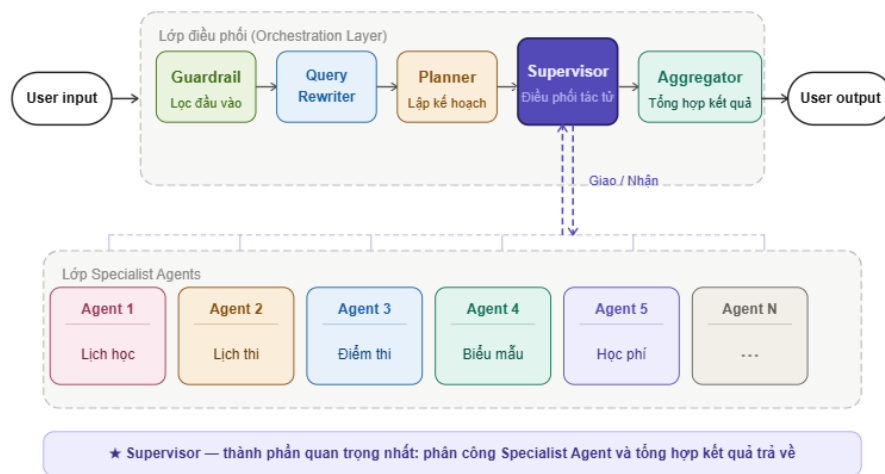
Cấu trúc này giúp quá trình suy luận của Agent trở nên minh bạch, dễ kiểm tra và dễ phát hiện lỗi khi cần gỡ rối.

1.5.6 Hệ thống đa tác tử và kiến trúc Supervisor–Specialist

Hệ thống đa tác tử (Multi-Agent System – MAS) là kiến trúc trong đó nhiều AI Agent phối hợp với nhau để giải quyết các bài toán vượt quá khả năng của một tác tử đơn

lẽ. Thay vì xây dựng một Agent duy nhất phải xử lý tất cả mọi thứ, MAS tổ chức công việc theo nguyên tắc phân công trách nhiệm rõ ràng: mỗi Agent chuyên sâu vào một vai trò hoặc lĩnh vực cụ thể, phối hợp với nhau thông qua một cơ chế điều phối chung.

Cách tiếp cận này mang lại nhiều lợi thế so với kiến trúc đơn tác tử: khả năng xử lý song song nhiều tác vụ độc lập, dễ mở rộng khi thêm nghiệp vụ mới mà không ảnh hưởng đến toàn bộ hệ thống, và tăng tính chuyên biệt của từng Agent giúp nâng cao chất lượng phản hồi.



Hình 1.9 Kiến trúc Supervisor–Specialist trong hệ thống đa tác tử

Trong đề tài này, kiến trúc Supervisor–Specialist được lựa chọn triển khai. Theo kiến trúc này, hệ thống bao gồm hai lớp Agent phân cấp:

- **Lớp điều phối (Supervisor layer):** Gồm các Agent có vai trò quản lý luồng xử lý tổng thể như kiểm soát phạm vi câu hỏi (Guardrail), viết lại truy vấn theo ngữ cảnh hội thoại (Query Rewriter), lập kế hoạch phân rã bài toán (Planner), định tuyến yêu cầu đến đúng Agent chuyên trách (Supervisor) và tổng hợp kết quả cuối cùng (Aggregator).
- **Lớp chuyên gia (Specialist layer):** Gồm các Agent chuyên trách từng nghiệp vụ cụ thể của nhà trường như lịch học, lịch thi, danh sách thi, kết quả học tập hay biểu mẫu hành chính. Mỗi Specialist Agent được trang bị bộ công cụ và cơ sở tri thức phù hợp với lĩnh vực của mình, đảm bảo chất lượng phản hồi chuyên biệt cao hơn so với một Agent tổng quát xử lý tất cả.

Luồng xử lý tổng quát đi theo trình tự: câu hỏi đầu vào được tiền xử lý bởi lớp điều phối, sau đó được định tuyến đến một hoặc nhiều Specialist Agent phù hợp để truy xuất

và xử lý thông tin, cuối cùng lớp điều phối tổng hợp kết quả và trả về câu trả lời hoàn chỉnh cho người dùng. Kiến trúc này là nền tảng thiết kế của toàn bộ hệ thống trợ lý ảo được đề xuất trong đề tài.

1.6 Các công nghệ sử dụng trong đề tài

1.6.1 Ngôn ngữ lập trình Python

Python là ngôn ngữ lập trình bậc cao được Guido van Rossum phát triển và phát hành lần đầu vào năm 1991. Python nổi tiếng với cú pháp rõ ràng, dễ đọc và gần gũi với ngôn ngữ tự nhiên, giúp lập trình viên tập trung vào logic giải quyết bài toán thay vì phải xử lý các vấn đề cú pháp phức tạp. Trong suốt hơn ba thập kỷ phát triển, Python đã trải qua nhiều phiên bản, từ Python 2.x rộng rãi một thời đến Python 3.x hiện là chuẩn mực với nhiều cải tiến đáng kể về hiệu suất, cú pháp và hệ thống kiểu. Ngày nay, Python đã khẳng định vị thế là ngôn ngữ lập trình hàng đầu trong lĩnh vực trí tuệ nhân tạo, khoa học dữ liệu và phát triển hệ thống backend.

Đặc điểm nổi bật của Python:

- **Cú pháp đơn giản và dễ đọc:** Python có cú pháp ngắn gọn, gần gũi với ngôn ngữ tự nhiên. Một đoạn code Python thường ngắn hơn đáng kể so với Java hay C++ cho cùng một chức năng. Đặc điểm này không chỉ giúp lập trình viên viết code nhanh hơn mà còn làm cho code dễ đọc, dễ bảo trì và dễ phát hiện lỗi về sau.
- **Đa nền tảng:** Python có thể chạy trên nhiều hệ điều hành khác nhau như Windows, macOS và Linux mà không cần thay đổi code, làm cho Python trở thành lựa chọn phổ biến cho phát triển phần mềm đa nền tảng.
- **Hệ sinh thái thư viện phong phú:** Python có một hệ thống thư viện đặc biệt phong phú, phục vụ nhiều mục đích khác nhau. Các thư viện nổi bật bao gồm NumPy và Pandas cho xử lý dữ liệu, Matplotlib cho trực quan hóa, Scikit-learn và TensorFlow cho học máy, và đặc biệt là hệ sinh thái AI Agent như OpenAI SDK, Qdrant Client, spaCy. Hầu hết các nhà cung cấp LLM và dịch vụ AI lớn hiện nay đều ưu tiên phát triển và cập nhật SDK Python trước tiên, đảm bảo tích hợp nhanh chóng với các mô hình và dịch vụ mới nhất.
- **Hỗ trợ lập trình bất đồng bộ:** Python cung cấp cú pháp async/await native từ phiên bản 3.5, cho phép xử lý đồng thời nhiều tác vụ I/O mà không bị chặn luồng thực thi. Đây là tính năng đặc biệt quan trọng cho các hệ thống AI cần gọi song song nhiều API bên ngoài như LLM API, Vector Database và các dịch vụ khác trong cùng một chu kỳ xử lý.
- **Cộng đồng lớn mạnh:** Python có một cộng đồng lập trình viên và nhà nghiên cứu đặc biệt đông đảo và năng động. Cộng đồng này liên tục đóng góp thư viện mới, tài liệu hướng dẫn và giải đáp thắc mắc thông qua các kênh như Stack Overflow,

GitHub và các diễn đàn chuyên ngành.

Ứng dụng của Python:

- **Trí tuệ nhân tạo và học máy:** Python là ngôn ngữ không thể thiếu trong lĩnh vực AI nhờ các framework như TensorFlow, PyTorch và Scikit-learn. Các tác vụ từ huấn luyện mô hình học sâu, xử lý ngôn ngữ tự nhiên đến xây dựng hệ thống AI Agent đều có thể thực hiện hiệu quả bằng Python với các thư viện chuyên dụng.
- **Phát triển web và API:** Python có nhiều framework mạnh mẽ như Django, Flask và FastAPI hỗ trợ xây dựng từ các ứng dụng web đơn giản đến các hệ thống API phức tạp với hiệu năng cao và hỗ trợ xử lý bất đồng bộ.
- **Khoa học dữ liệu:** Python là ngôn ngữ hàng đầu trong khoa học dữ liệu nhờ các thư viện NumPy, Pandas và Matplotlib giúp dễ dàng xử lý, phân tích và trực quan hóa dữ liệu quy mô lớn.
- **Tự động hóa:** Python được sử dụng rộng rãi để tự động hóa các tác vụ lặp lại, từ xử lý file, crawl dữ liệu đến tự động kiểm thử phần mềm và quản lý hệ thống.
- **Xử lý văn bản và tài liệu:** Các thư viện như pypdf, pdfplumber và python-docx cho phép Python xử lý và trích xuất nội dung từ nhiều định dạng tài liệu khác nhau, phục vụ hiệu quả cho các bài toán xây dựng cơ sở tri thức và nạp dữ liệu.

Tương lai của Python: Python tiếp tục phát triển mạnh mẽ và được dự đoán sẽ giữ vững vị thế ngôn ngữ hàng đầu trong lĩnh vực AI trong những năm tới. Các phiên bản Python mới liên tục cải thiện hiệu suất thực thi với các dự án như Faster CPython, đồng thời mở rộng thêm tính năng cho kiểu tĩnh và lập trình bất đồng bộ để đáp ứng nhu cầu ngày càng cao của các ứng dụng AI quy mô lớn.

1.6.2 FastAPI Framework

FastAPI là một web framework hiện đại, hiệu suất cao được viết bằng Python (yêu cầu phiên bản 3.7 trở lên), được thiết kế chuyên biệt để xây dựng API một cách nhanh chóng và hiệu quả. Framework này được xây dựng trên nền tảng Starlette cho phần xử lý web và Pydantic cho việc kiểm tra và xác thực dữ liệu, tận dụng tối đa các tính năng hiện đại của Python như gợi ý kiểu (type hints) và lập trình bất đồng bộ.

Ưu điểm:

- **Hiệu năng vượt trội:** Nhờ được xây dựng trên Starlette và hỗ trợ asyncio một cách tự nhiên, FastAPI có hiệu năng rất cao, có thể sánh ngang với các framework trên NodeJS hay Go. Điều này làm cho FastAPI trở thành lựa chọn lý tưởng cho các ứng dụng yêu cầu xử lý đồng thời lượng lớn request, đặc biệt là các hệ thống chatbot cần gọi nhiều API bên ngoài song song trong mỗi lượt xử lý.
- **Hỗ trợ Streaming response:** FastAPI hỗ trợ Server-Sent Events (SSE) cho phép server chủ động đẩy dữ liệu đến client theo từng phần nhỏ liên tục. Tính năng này

cho phép xây dựng trải nghiệm chat trực quan nơi câu trả lời xuất hiện từng từ một ngay khi LLMs sinh ra thay vì người dùng phải chờ đến khi toàn bộ câu trả lời hoàn thành.

- **Tốc độ phát triển cực nhanh:** FastAPI giúp giảm từ 200% đến 300% thời gian phát triển code. Việc sử dụng Pydantic và gợi ý kiểu giúp giảm đáng kể lỗi do con người, đồng thời cung cấp tính năng tự động hoàn thành mạnh mẽ trong các trình soạn thảo code.
- **Tự động sinh tài liệu tương tác:** FastAPI tự động sinh ra tài liệu API theo hai chuẩn mở là OpenAPI (Swagger UI) và ReDoc. Lập trình viên có thể kiểm thử và tương tác với toàn bộ API endpoint một cách trực quan thông qua giao diện web mà không cần viết thêm bất kỳ dòng code tài liệu nào.
- **Xác thực dữ liệu mạnh mẽ:** Với Pydantic, việc định nghĩa và kiểm tra cấu trúc dữ liệu vào ra của API trở nên cực kỳ đơn giản. Lỗi dữ liệu đầu vào được phát hiện tự động và trả về thông báo lỗi chi tiết, chuẩn hóa trước khi đến logic xử lý.
- **Hỗ trợ Middleware linh hoạt:** FastAPI cho phép tích hợp các middleware xử lý cross-cutting concerns như logging, xác thực, giám sát hiệu suất hay CORS một cách gọn gàng. Ví dụ điển hình là tích hợp Prometheus middleware để theo dõi latency và số lượng request theo thời gian thực.
- **Hệ thống Dependency Injection:** FastAPI cung cấp cơ chế dependency injection mạnh mẽ và dễ sử dụng, cho phép tái sử dụng logic như xác thực token, kết nối database hay kiểm tra quyền truy cập một cách gọn gàng trên nhiều endpoint khác nhau.

Nhược điểm:

- **Yêu cầu kiến thức về các tính năng Python hiện đại:** Để tận dụng hết sức mạnh của FastAPI, lập trình viên cần có sự hiểu biết nhất định về các khái niệm như type hints và lập trình bất đồng bộ async/await, điều này có thể là rào cản với người mới làm quen.
- **Cộng đồng nhỏ hơn so với Flask và Django:** Mặc dù đang phát triển rất nhanh, cộng đồng FastAPI vẫn còn non trẻ hơn so với các framework có lịch sử lâu đời. Điều này dẫn đến ít tài liệu hướng dẫn và plugin có sẵn hơn cho một số tình huống đặc thù.

1.6.3 OpenAI API

OpenAI API là nền tảng dịch vụ đám mây cung cấp quyền truy cập vào các mô hình ngôn ngữ lớn tiên tiến nhất của OpenAI thông qua giao diện REST API chuẩn hóa. Được ra mắt lần đầu năm 2020, OpenAI API đã nhanh chóng trở thành nền tảng phổ biến nhất để tích hợp AI vào ứng dụng, với hàng triệu lập trình viên và doanh nghiệp trên toàn thế giới sử dụng.

Thay vì tự xây dựng và vận hành hạ tầng huấn luyện mô hình đòi hỏi chi phí tính toán cực lớn, lập trình viên chỉ cần gọi API và nhận kết quả theo mô hình pay-per-use tính theo số token tiêu thụ. Cách tiếp cận này giúp mọi nhóm phát triển từ cá nhân đến doanh nghiệp đều có thể tích hợp năng lực AI tiên tiến mà không cần đầu tư hạ tầng GPU chuyên dụng.

OpenAI API bao gồm nhiều nhóm dịch vụ, trong đó quan trọng nhất là:

- **Chat Completions API:** Cho phép gửi hội thoại nhiều lượt bao gồm system message (định nghĩa vai trò và hướng dẫn cho mô hình), user message (đầu vào người dùng) và assistant message (phản hồi trước đó) đến các mô hình GPT và nhận về câu trả lời được sinh tự nhiên. Các mô hình GPT được cung cấp qua API bao gồm nhiều phiên bản với mức độ năng lực và chi phí khác nhau, từ các mô hình nhỏ gọn hiệu quả về chi phí như gpt-4o-mini đến các mô hình mạnh nhất như gpt-4o và o3. API hỗ trợ cả chế độ đồng bộ lẫn streaming theo từng token, đồng thời cung cấp tính năng function calling cho phép mô hình gọi các hàm có cấu trúc định nghĩa sẵn – cơ chế nền tảng để xây dựng AI Agent sử dụng công cụ.
- **Embeddings API:** Cung cấp các mô hình chuyên biệt để chuyển đổi đoạn văn bản thành vector số học nhiều chiều, trong đó nổi bật là text-embedding-3-small (1536 chiều) và text-embedding-3-large (3072 chiều). Các vector này mang thông tin ngữ nghĩa của văn bản, cho phép tính toán độ tương đồng giữa các đoạn văn bản – nền tảng của mọi hệ thống tìm kiếm ngữ nghĩa và RAG. Mô hình text-embedding-3-small được đánh giá là cân bằng tốt giữa chất lượng embedding, tốc độ xử lý và chi phí, phù hợp cho các ứng dụng cần nhúng lượng lớn tài liệu và xử lý truy vấn thời gian thực.

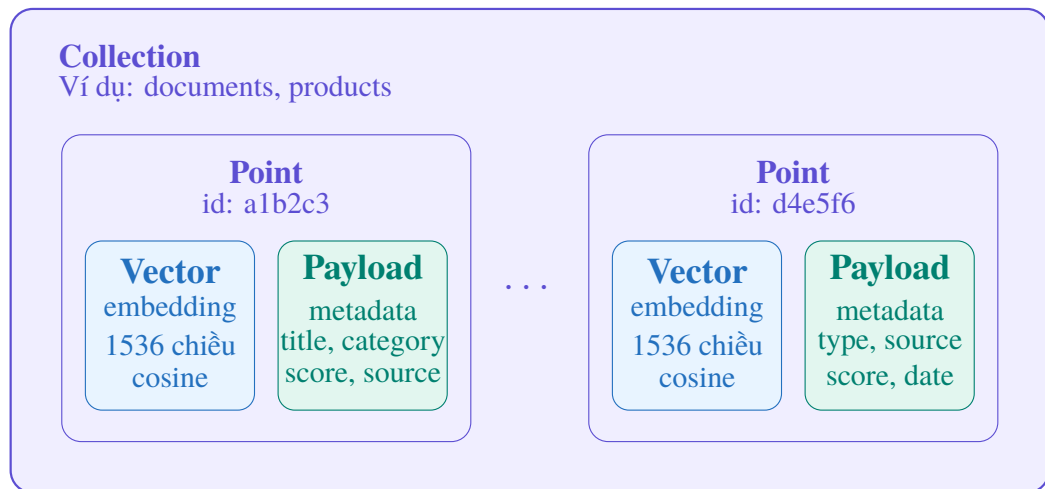
Một điểm mạnh đáng chú ý của OpenAI API là Python SDK chính thức được xây dựng với hỗ trợ đầy đủ async, streaming, retry tự động và xử lý lỗi tốt, giúp tích hợp vào ứng dụng Python một cách ổn định và nhất quán. Các mô hình cũng được OpenAI liên tục cải tiến và nâng cấp định kỳ trong khi vẫn duy trì khả năng tương thích ngược của API.

1.6.4 Qdrant Vector Database

Qdrant là một vector database mã nguồn mở được viết bằng ngôn ngữ Rust, ra đời năm 2021, chuyên thiết kế cho bài toán lưu trữ và tìm kiếm hiệu năng cao trên không gian vector nhiều chiều. Khác với cơ sở dữ liệu quan hệ hoạt động dựa trên so khớp chính xác, Qdrant trả về các vector gần nhất về mặt ngữ nghĩa với vector truy vấn, làm nền tảng cho các hệ thống tìm kiếm ngữ nghĩa và RAG hiện đại.

Kiến trúc của Qdrant được thiết kế xoay quanh khái niệm Collection – đơn vị tổ chức dữ liệu chứa tập hợp các Point. Mỗi Point bao gồm ba thành phần: định danh duy

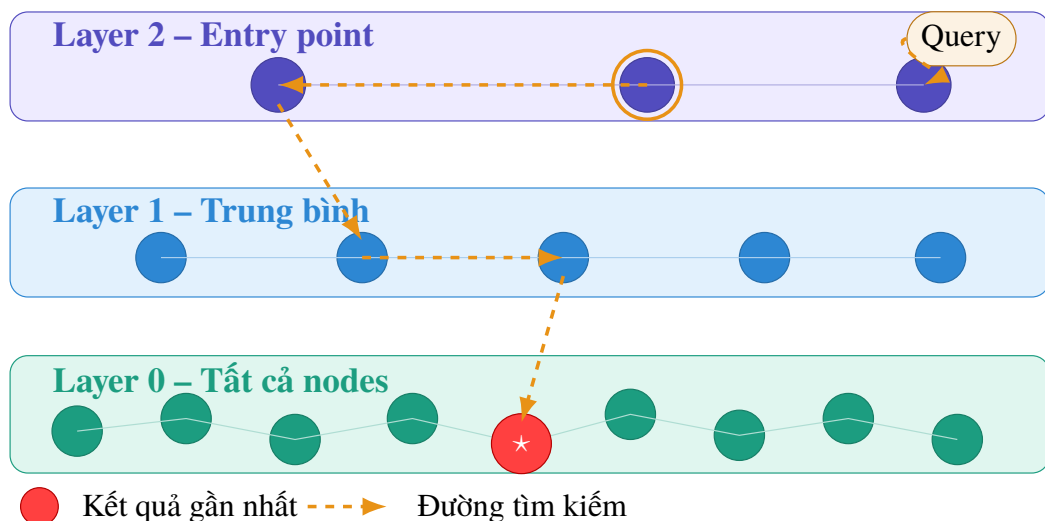
nhất (ID), vector số học nhiều chiều và payload metadata tùy ý dạng JSON. Sự kết hợp này cho phép Qdrant không chỉ thực hiện tìm kiếm ngữ nghĩa đơn thuần mà còn hỗ trợ lọc kết hợp giữa vector similarity và điều kiện metadata trong một truy vấn duy nhất.



Hình 1.10 Sơ đồ cấu trúc Collection → Points → (Vector + Payload)

Các tính năng kỹ thuật nổi bật:

- **Thuật toán HNSW:** Qdrant sử dụng thuật toán Hierarchical Navigable Small World để xây dựng chỉ mục vector. HNSW tổ chức các vector thành một đồ thị phân cấp nhiều tầng, cho phép tìm kiếm láng giềng gần đúng với độ phức tạp logarithmic thay vì tuyến tính. Nhờ đó, Qdrant duy trì độ trễ tìm kiếm thấp ngay cả với tập dữ liệu hàng triệu vector.



Hình 1.11 Sơ đồ HNSW đơn giản minh họa tìm kiếm theo tầng

- **Lọc kết hợp (Filtered Search):** Qdrant cho phép lọc điều kiện metadata trước hoặc sau bước tìm kiếm vector mà vẫn đảm bảo hiệu năng cao. Ví dụ, có thể tìm kiếm “20 tài liệu liên quan nhất về lịch thi, chỉ trong tài liệu thuộc agent lịch thi, có loại

là PDF” trong một truy vấn duy nhất.

- **Payload Index:** Qdrant hỗ trợ tạo chỉ mục trên các trường metadata thường xuyên được dùng để lọc, bao gồm chỉ mục keyword cho trường phân loại và chỉ mục full-text với tokenizer WORD cho trường văn bản. Payload index giúp tăng tốc đáng kể các truy vấn có điều kiện lọc phức tạp mà không làm tăng thời gian tìm kiếm vector.
- **Nhiều độ đo khoảng cách:** Qdrant hỗ trợ Cosine Similarity, Dot Product và Euclidean Distance, linh hoạt cho các bài toán embedding khác nhau. Cosine Similarity thường được sử dụng với các mô hình embedding văn bản vì nó đo góc giữa hai vector, không phụ thuộc vào độ lớn.
- **Linh hoạt triển khai:** Qdrant hỗ trợ chạy embedded trực tiếp trong tiến trình Python mà không cần cài đặt riêng, hoặc triển khai dưới dạng dịch vụ độc lập với REST API và gRPC, phù hợp cho cả môi trường phát triển và sản xuất.
- **Python SDK đầy đủ tính năng:** Qdrant-client cung cấp Python SDK với hỗ trợ async đầy đủ, cho phép tạo collection, upsert điểm, xây dựng payload index và thực hiện tìm kiếm ngữ nghĩa một cách linh hoạt và hiệu quả.

1.6.5 Hệ quản trị cơ sở dữ liệu PostgreSQL

PostgreSQL (thường gọi là Postgres) là hệ quản trị cơ sở dữ liệu quan hệ – đối tượng mã nguồn mở mạnh mẽ và tiên tiến. Với hơn 35 năm phát triển không ngừng từ dự án POSTGRES tại Đại học California, Berkeley, PostgreSQL đã tạo dựng được danh tiếng vững chắc về độ tin cậy, sự ổn định, tính toàn vẹn dữ liệu và khả năng mở rộng vượt trội. Nó được tin dùng cho các ứng dụng từ quy mô nhỏ đến các hệ thống doanh nghiệp phức tạp và yêu cầu cao nhất.

Các tính năng và ưu điểm nổi bật:

- **Tuân thủ ACID nghiêm ngặt:** PostgreSQL tuân thủ chặt chẽ các thuộc tính ACID (Atomicity – nguyên tử, Consistency – nhất quán, Isolation – cô lập, Durability – bền vững). Điều này đảm bảo rằng các giao dịch được xử lý một cách đáng tin cậy, dữ liệu luôn ở trạng thái nhất quán và không bị mất mát ngay cả khi có sự cố xảy ra. Các ràng buộc như khóa chính, khóa ngoại, UNIQUE, CHECK đều được triển khai một cách đầy đủ và mạnh mẽ.
- **Xử lý đồng thời tiên tiến (MVCC):** PostgreSQL sử dụng kiến trúc Multi-Version Concurrency Control, cho phép nhiều tiến trình đọc và ghi hoạt động song song mà không khóa lẫn nhau. Cụ thể, tiến trình đọc không chặn tiến trình ghi và ngược lại, giúp tăng đáng kể hiệu suất và thông lượng cho các ứng dụng có nhiều người dùng truy cập đồng thời.
- **Hỗ trợ đa dạng kiểu dữ liệu nâng cao:** Ngoài các kiểu dữ liệu chuẩn (số, chuỗi, ngày giờ), PostgreSQL hỗ trợ mảng, hstore (key-value), XML và đặc biệt là JSON/JSONB. Với kiểu dữ liệu JSONB được tối ưu hóa và có khả năng đánh chỉ mục,

PostgreSQL có thể xử lý hiệu quả cả dữ liệu có cấu trúc và bán cấu trúc trong cùng một cơ sở dữ liệu.

- **Khả năng mở rộng (Extensibility):** PostgreSQL cho phép người dùng tự định nghĩa các kiểu dữ liệu, toán tử, hàm và ngôn ngữ thủ tục của riêng mình. Ví dụ tiêu biểu nhất là phần mở rộng PostGIS biến PostgreSQL thành một hệ quản trị CSDL không gian địa lý hàng đầu.
- **Tích hợp tốt với Python qua SQLAlchemy:** SQLAlchemy là ORM (Object-Relational Mapping) phổ biến nhất trong hệ sinh thái Python, cung cấp lớp trừu tượng cho phép thao tác với PostgreSQL thông qua các đối tượng Python thay vì viết SQL thuần. Kết hợp với driver psycopg2, SQLAlchemy cho phép định nghĩa schema, thực hiện migration và quản lý session một cách gọn gàng, đồng thời hỗ trợ đầy đủ các tính năng nâng cao của PostgreSQL.
- **Mã nguồn mở và cộng đồng vững mạnh:** Là dự án mã nguồn mở hoàn toàn, PostgreSQL không có chi phí bản quyền, không bị ràng buộc bởi nhà cung cấp và được phát triển bởi cộng đồng toàn cầu. Điều này đảm bảo sự phát triển liên tục, tài liệu phong phú và hệ sinh thái công cụ hỗ trợ đa dạng.

1.6.6 Redis

Redis (Remote Dictionary Server) là một hệ thống lưu trữ cấu trúc dữ liệu trong bộ nhớ (in-memory) mã nguồn mở, hoạt động theo mô hình key-value và hỗ trợ nhiều kiểu dữ liệu phong phú như string, hash, list, set và sorted set. Được phát triển bởi Salvatore Sanfilippo từ năm 2009, Redis đã trở thành lựa chọn hàng đầu cho các bài toán yêu cầu truy xuất dữ liệu với độ trễ cực thấp.

Đặc điểm nổi bật nhất của Redis là tốc độ: do toàn bộ dữ liệu được lưu trữ trực tiếp trong RAM, Redis có thể xử lý hàng triệu thao tác đọc/ghi mỗi giây với độ trễ chỉ tính bằng microsecond. Bên cạnh đó, Redis hỗ trợ cơ chế TTL (Time-To-Live) cho phép dữ liệu tự động hết hạn sau khoảng thời gian định trước, rất phù hợp cho các dữ liệu có vòng đời giới hạn như session người dùng hay cache truy vấn.

Redis được ứng dụng phổ biến trong nhiều lĩnh vực: quản lý session trong các ứng dụng web, cache kết quả truy vấn tốn kém, hàng đợi tin nhắn (message queue), bảng xếp hạng thời gian thực và pub/sub messaging. Trong kiến trúc hệ thống chatbot, Redis đóng vai trò lưu trữ trạng thái phiên hội thoại đang hoạt động với hiệu năng cao, giúp hệ thống truy xuất ngữ cảnh hội thoại nhanh chóng mà không cần truy vấn cơ sở dữ liệu quan hệ cho mỗi lượt tương tác.

1.7 Kết luận chương 1

Chương 1 đã trình bày một cách tổng quan về hệ thống trợ lý ảo AI đa tác tử ứng dụng trong môi trường đại học. Nội dung chính bao gồm phần giới thiệu tổng quan về

trợ lý ảo AI, tiềm năng ứng dụng trong giáo dục và các phương pháp xây dựng hệ thống từ tập luật đến LLMs và AI Agent. Bên cạnh đó, chương 1 cũng trình bày những khái niệm cơ bản về mô hình ngôn ngữ lớn, kỹ thuật RAG, AI Agent và hệ thống đa tác tử theo kiến trúc Supervisor–Specialist — nền tảng kỹ thuật cốt lõi của đề tài. Đồng thời, các công nghệ được sử dụng trong quá trình xây dựng hệ thống cũng được giới thiệu đầy đủ. Trong chương tiếp theo, báo cáo sẽ trình bày chi tiết phương pháp xây dựng hệ thống trợ lý ảo AI đa tác tử.

CHƯƠNG 2: PHƯƠNG PHÁP XÂY DỰNG HỆ THỐNG TRỢ LÝ ẢO AI ĐA TÁC TỬ

2.1 Mô hình kiến trúc tổng thể

2.1.1 Đặc tả yêu cầu và phạm vi hệ thống

Trước khi tiến hành thiết kế kiến trúc, ta cần xác định rõ tính chất môi trường, miền tri thức và tập hành động mà hệ thống cần thực hiện. Việc xác định này đóng vai trò nền tảng, giúp các quyết định thiết kế về sau có căn cứ cụ thể từ bài toán thực tế.

2.1.1.1. Môi trường vận hành

Môi trường hoạt động của hệ thống gắn liền với toàn bộ các hoạt động đào tạo tại Học viện Kỹ thuật Mật mã. Người dùng bao gồm sinh viên, giảng viên và cán bộ nhân viên – những người có nhu cầu tra cứu, hỏi đáp các thông tin học vụ chính thức thường xuyên trong quá trình dạy, học và công tác. Từ đặc điểm đó, ta xác định được các tính chất môi trường của hệ thống như được trình bày trong Bảng 2.1.

Tính chất	Nội dung xác định
Đối tượng sử dụng	Sinh viên, giảng viên, cán bộ/nhân viên Học viện Kỹ thuật Mật mã
Bối cảnh sử dụng	Tra cứu, hỏi đáp thông tin học vụ: tuyển sinh, quy chế đào tạo, chuẩn đầu ra, lịch thi, danh sách thi, điểm học phần, biểu mẫu thủ tục hành chính
Vai trò hệ thống	Nhóm trợ lý ảo đa tác tử, mỗi tác tử phụ trách một miền nghiệp vụ chuyên biệt, phối hợp để tạo câu trả lời thống nhất cho người dùng
Phạm vi phục vụ	Các nội dung có trong tài liệu học vụ KMA đã được nạp vào hệ thống; từ chối câu hỏi ngoài phạm vi, không phỏng đoán
Nguyên tắc phản hồi	Chỉ trả lời dựa trên tài liệu đã được nạp; câu chào hỏi được xử lý ngắn gọn; câu hỏi ngoài phạm vi được từ chối kèm hướng dẫn
Tính chất hội thoại	Hệ thống lưu lịch sử và tóm tắt phiên; hỗ trợ câu hỏi tiếp diễn (follow-up) có tham chiếu đến ngữ cảnh trước

Bảng 2.1 Bảng xác định tính chất môi trường của hệ thống đa tác tử

Đặc biệt, khác với chatbot hỏi đáp thường, hệ thống cần xử lý các câu hỏi đa ý – vừa hỏi về tuyển sinh, vừa hỏi về biểu mẫu nhập học chẳng hạn – đây là yêu cầu thực tế từ sinh viên mới nhập học, đòi hỏi kiến trúc điều phối đa tác tử thay vì một agent đơn lẻ.

2.1.1.2. Miền tri thức và nguồn dữ liệu

Miền tri thức xác định toàn bộ tài nguyên dữ liệu mà hệ thống có thể truy cập để phục vụ người dùng. Toàn bộ tài liệu được thu thập, tiền xử lý theo chiến lược phân mảnh phù hợp với từng loại nội dung, sau đó được vector hóa bằng mô hình embedding `text-embedding-3-small` của OpenAI và lưu trữ trong cơ sở dữ liệu vector Qdrant với tên collection `kma_docs`. Mỗi nhóm tài liệu được gán metadata phân loại theo nghiệp vụ, phục vụ định tuyến truy xuất chính xác tại từng tác tử chuyên môn.

Nguồn dữ liệu	Nghiệp vụ	Tác tử phụ trách	Định dạng
Đề án tuyển sinh, bảng điểm chuẩn theo năm, chỉ tiêu ngành, chương trình đào tạo	Tuyển sinh & CTĐT	tuyen_sinh	PDF, văn bản
Quy chế đào tạo, chuẩn đầu ra ngoại ngữ (TOEIC/VSTEP), quy định thi tốt nghiệp, quy trình khảo thí	Khảo thí & Quy chế	khao_thi	PDF, văn bản
Ma trận đề thi từng môn học (số câu, cấu trúc phần, thang điểm, thời gian)	Ma trận đề thi	ma_tran	PDF, bảng
Bảng điểm thi học phần công bố theo MSSV, kết quả chứng chỉ Anh văn đầu vào	Kết quả học tập	diem_thi	PDF dạng bảng
Đơn từ, biểu mẫu thủ tục hành chính (phúc khảo, nghỉ học, bảo lưu, BHYT, thực tập, ra trường)	Biểu mẫu & Thủ tục	bieu_mau	PDF, DOCX
Danh sách sinh viên dự thi theo ngày thi, ca thi, phòng thi, số báo danh	Danh sách thi	danh_sach_thi	PDF dạng bảng
Lịch thi KTHP theo học kỳ, đợt thi, môn học phần	Lịch thi	lich_thi	PDF dạng bảng

Bảng 2.2 Bảng xác định miền tri thức của hệ thống

Ngoài ra, hệ thống còn sử dụng cơ sở dữ liệu PostgreSQL để lưu trữ lịch sử hội thoại, phiên làm việc, thông tin tài khoản quản trị và nhật ký hệ thống; Redis được dùng để lưu session nóng với thời gian sống ngắn, giúp các thao tác tra cứu lịch sử hội thoại diễn ra nhanh chóng trong phiên làm việc đang hoạt động.

2.1.1.3. Tập hành động của hệ thống

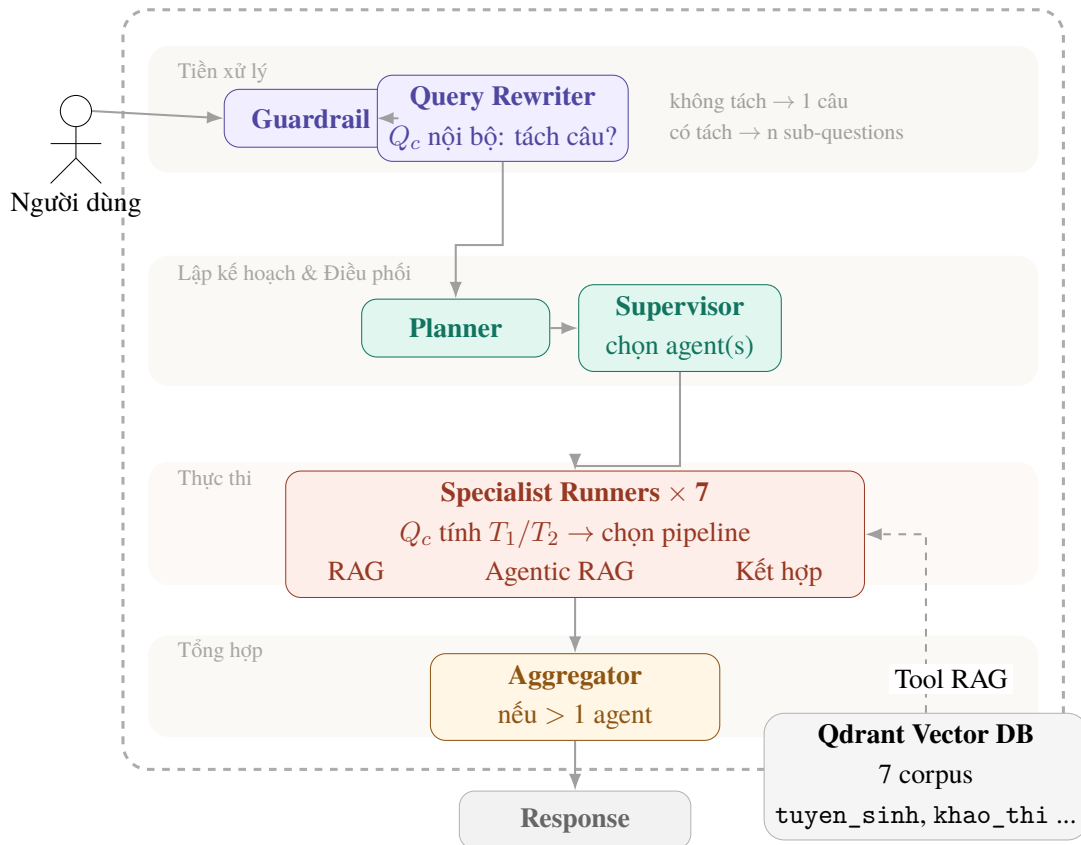
Tập hành động định nghĩa những gì hệ thống có thể thực hiện để đáp ứng yêu cầu người dùng. Đây chính là các tính năng nghiệp vụ cụ thể của toàn bộ nhóm tác tử, được xác định dựa trên miền tri thức và tính chất môi trường đã phân tích:

- **Lọc và phân loại yêu cầu:** Nhận diện câu hỏi thuộc phạm vi KMA, phân biệt với câu hỏi ngoài chủ đề và lời chào hỏi xã giao; phản hồi phù hợp cho từng loại ngay từ bước đầu.
- **Chuẩn hóa truy vấn hội thoại:** Phát hiện câu hỏi follow-up tham chiếu ngữ cảnh cũ (ví dụ: “còn môn đó thì sao?”, “bạn ấy đạt chưa?”) và viết lại thành câu hỏi độc lập, đầy đủ nghĩa.
- **Lập kế hoạch phân rã:** Đánh giá mức độ phức tạp của câu hỏi và tách câu hỏi đa ý thành các câu hỏi con độc lập để từng tác tử có thể xử lý riêng.
- **Điều phối đa tác tử:** Nhận diện ý định (intent) của câu hỏi và định tuyến đến đúng tác tử chuyên môn; hỗ trợ gọi đồng thời nhiều tác tử với câu hỏi đa miền.
- **Truy xuất và hỏi đáp theo miền nghiệp vụ:** Thực hiện tìm kiếm ngữ nghĩa trong corpus tài liệu riêng của từng tác tử, đánh giá độ liên quan và sinh câu trả lời dựa trên tài liệu truy xuất được.
- **Tra cứu điểm thi theo MSSV:** Đặc biệt với agent `diem_thi`, hỗ trợ tra cứu bảng điểm học phần theo mã số sinh viên (MSSV dạng ATxxxxxx/CTxxxxxx), học kỳ và đợt thi.
- **Hỗ trợ điền biểu mẫu:** Hướng dẫn và hỗ trợ người dùng điền vào biểu mẫu/đơn từ hành chính qua hội thoại từng bước.
- **Tổng hợp đa nguồn:** Kết hợp kết quả từ nhiều tác tử chuyên môn thành một câu trả lời mạch lạc, nhất quán, tránh trùng lặp và mâu thuẫn.
- **Phản hồi streaming:** Trả kết quả theo luồng (SSE) giúp người dùng nhận được phản hồi từng phần ngay khi hệ thống đang xử lý, thay vì chờ đến khi hoàn tất.

2.1.2 Kiến trúc đa tác tử tổng quan

2.1.2.1. Các lớp thành phần chính

Dựa trên tính chất môi trường, miền tri thức và tập hành động đã xác định, ta xây dựng được mô hình kiến trúc tổng quan của hệ thống như Hình 2.1. Hệ thống được tổ chức thành bốn lớp xử lý tuần tự.



Hình 2.1 Mô hình kiến trúc tổng quan hệ thống trợ lý ảo đa tác tử KMA

Lớp tiền xử lý yêu cầu bao gồm hai thành phần Guardrail và Query Rewriter. Đây là cổng kiểm soát đầu vào, có nhiệm vụ đảm bảo rằng mọi truy vấn đi đến các tầng xử lý phía sau đều đã được xác nhận thuộc phạm vi hệ thống và được diễn đạt rõ ràng, đầy đủ ngữ cảnh. Guardrail phân loại truy vấn thành ba nhóm: in-scope (học vụ KMA), off-topic (ngoài phạm vi) và chitchat (chào hỏi xã giao); hai nhóm sau được phản hồi ngay mà không đi vào pipeline chuyên môn. Query Rewriter xử lý các câu hỏi phụ thuộc hội thoại, bổ sung chủ thể, tên nghiệp vụ, MSSV hoặc mảng chủ đề từ lịch sử trò chuyện để tạo truy vấn độc lập.

Lớp lập kế hoạch và điều phối bao gồm Planner và Supervisor. Planner đánh giá mức độ phức tạp của truy vấn theo các chỉ số nội bộ; nếu truy vấn có nhiều ý độc lập, Planner tách thành các câu hỏi con. Mỗi câu hỏi con (hoặc câu hỏi gốc nếu không cần tách) được chuyển đến Supervisor. Supervisor là thành phần điều phối trung tâm của toàn hệ thống – phân tích ý định (intent) và chọn tác tử chuyên môn phù hợp; với câu hỏi đa miền, Supervisor có thể gọi song song nhiều tác tử cùng lúc.

Lớp thực thi chuyên môn bao gồm 7 Specialist Runner, tương ứng với 7 miền nghiệp vụ đã xác định. Mỗi runner hoạt động độc lập: tự tính toán chỉ số Q_c cho truy vấn được giao, chọn pipeline xử lý phù hợp (Native RAG, Hybrid RAG hoặc Agentic RAG), truy xuất tài liệu trong corpus nghiệp vụ riêng trên Qdrant và tạo câu trả lời chuyên môn. Thiết kế tách biệt này cho phép từng tác tử tối ưu hóa chiến lược truy xuất theo đặc thù

dữ liệu của mình.

Lớp tổng hợp và phản hồi bao gồm Aggregator. Khi chỉ có một tác tử tham gia xử lý, kết quả được trả trực tiếp. Khi nhiều tác tử cùng tham gia, Aggregator sử dụng LLM để gộp các câu trả lời thành một phản hồi thống nhất, đảm bảo không mất thông tin và không mâu thuẫn giữa các nguồn. Kết quả cuối cùng được trả về người dùng qua cơ chế streaming (SSE).

Về hạ tầng tri thức dùng chung, toàn bộ các tác tử chuyên môn đều khai thác dữ liệu từ cơ sở dữ liệu vector Qdrant, nhưng mỗi tác tử giới hạn truy xuất trong corpus nghiệp vụ của mình thông qua bộ lọc metadata. Điều này vừa đảm bảo độ chính xác định tuyến, vừa tránh nhiễu từ tài liệu của nghiệp vụ khác.

2.1.2.2. Danh sách tác tử chuyên môn

Bảng 2.3 mô tả chi tiết 7 tác tử chuyên môn, bao gồm tên, miền phụ trách và các từ khóa định tuyến chính mà Supervisor sử dụng để phân loại câu hỏi.

Bảng 2.3 Danh sách và đặc tả các tác tử chuyên môn

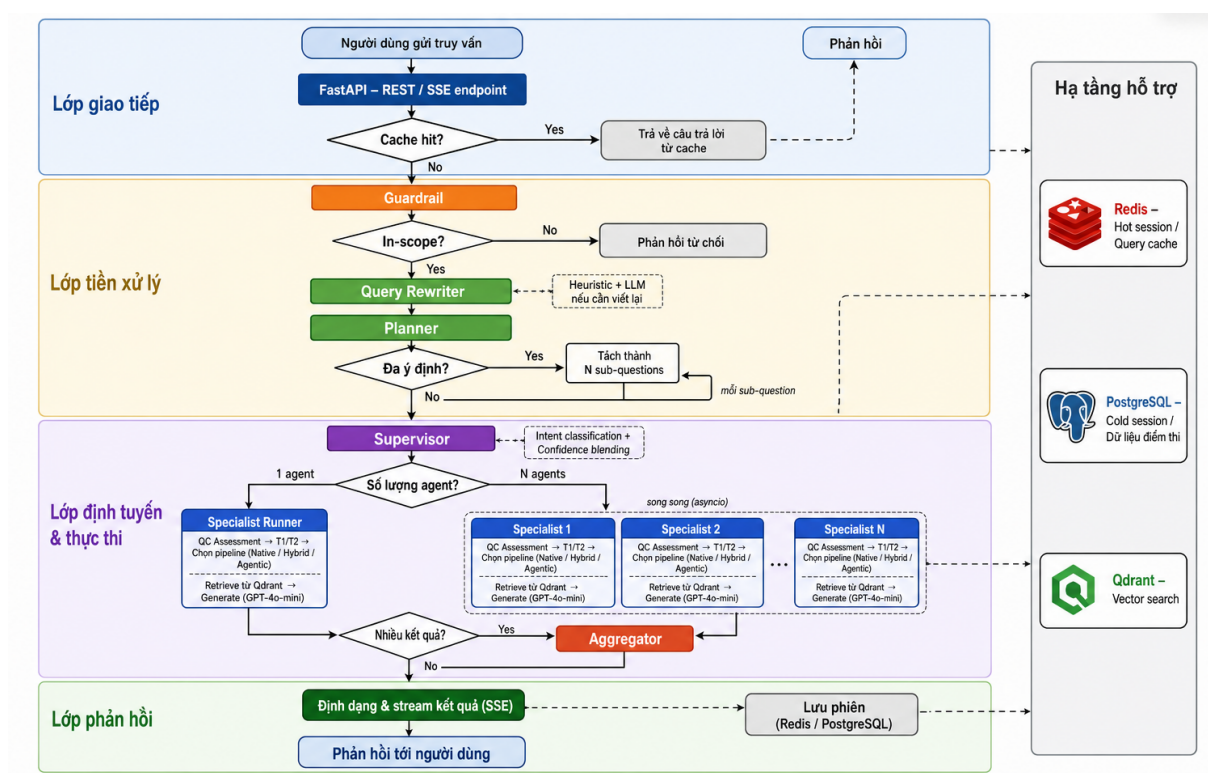
Mã tác tử	Tên tác tử	Phạm vi phụ trách	Từ khóa định tuyến tiêu biểu
tuyen_sinh	Trợ lý Tuyển sinh & CTĐT	Đề án tuyển sinh, điểm chuẩn, chỉ tiêu, ngành học, CTĐT các ngành, nhập học	tuyển sinh, điểm chuẩn, CTĐT, ngành, thạc sĩ, nhập học
khao_thi	Trợ lý Khảo thí & Quy chế	Quy chế đào tạo, chuẩn đầu ra ngoại ngữ, quy định thi, thi tốt nghiệp, sát hạch	quy chế, chuẩn đầu ra, TOEIC, thi tốt nghiệp, khảo thí
ma_tran	Trợ lý Ma trận đề thi	Ma trận đề thi từng môn học, cấu trúc đề, số câu, thang điểm, thời gian thi	ma trận, đề thi, cấu trúc đề, môn thi
diem_thi	Trợ lý Kết quả học tập	Bảng điểm KTHP công bố theo MSSV, kết quả học kỳ, chứng chỉ Anh văn	bảng điểm, MSSV, kết quả thi, học kỳ, CT/AT
bieu_mau	Trợ lý Biểu mẫu & Thủ tục	Đơn từ, biểu mẫu hành chính, thủ tục nghỉ học, phúc khảo, thực tập, ra trường	biểu mẫu, đơn, thủ tục, phúc khảo, nghỉ học, BHYT

Bảng 2.3 (tiếp theo)

Mã tác tử	Tên tác tử	Phạm vi phụ trách	Từ khóa định tuyến tiêu biểu
danh_sach_thi	Trợ lý Danh sách thi	Danh sách dự thi theo ngày/ca/phòng, tra MSSV/SBD trong bảng danh sách thi	danh sách thi, phòng thi, ca thi, số báo danh, ngày thi
lich_thi	Trợ lý Lịch thi	Lịch KTHP theo học kỳ, đợt thi, môn học; lịch thi lại (lần 2)	lịch thi, KTHP, đợt thi, học kỳ, thi lại, địa điểm thi

2.1.2.3. Luồng xử lý truy vấn tổng quát

Hình 2.2 mô tả luồng xử lý tổng quan khi người dùng gửi một truy vấn mới đến hệ thống.



Hình 2.2 Sơ đồ luồng xử lý tổng quan khi người dùng gửi truy vấn

Diễn giải từng bước của luồng như sau:

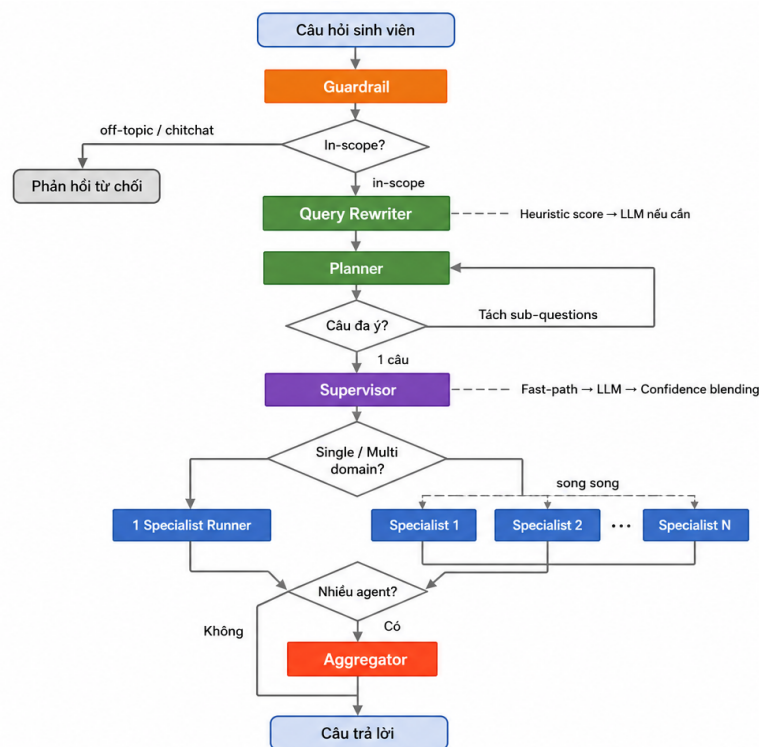
- **Bước 1 – Guardrail phân loại yêu cầu:** Ngay khi tiếp nhận câu hỏi, Guardrail thực hiện phân loại theo ba nhóm: in-scope, off-topic và chitchat. Guardrail kết hợp hai

cơ chế: kiểm tra từ khóa theo luật (rule-based heuristics) để xử lý nhanh các trường hợp rõ ràng, và gọi LLM (gpt-4o-mini) để phán quyết những trường hợp mơ hồ. Với câu hỏi chitchat hoặc off-topic, hệ thống phản hồi theo mẫu định sẵn và kết thúc xử lý mà không đi vào các bước tiếp theo.

- **Bước 2 – Query Rewriter làm rõ truy vấn:** Với truy vấn in-scope, Query Rewriter đánh giá điểm heuristic để xác định xem câu hỏi có phải là câu hỏi phụ thuộc ngữ cảnh (follow-up) hay không. Chỉ khi điểm heuristic vượt ngưỡng 0,65, LLM mới được gọi để viết lại câu hỏi. Cơ chế này giúp tránh chi phí LLM không cần thiết với những câu hỏi đã đủ rõ nghĩa. Kết quả là một câu truy vấn hoàn chỉnh, có thể đứng độc lập để đưa vào các bước xử lý tiếp theo.
- **Bước 3 – Planner đánh giá và lập kế hoạch:** Planner nhận truy vấn đã làm rõ và đánh giá mức độ phức tạp nội bộ. Nếu phát hiện câu hỏi có nhiều ý (nhiều dấu hỏi, từ nối “và/đồng thời/so sánh”, hoặc điểm phức tạp vượt ngưỡng), Planner gọi LLM để tách thành tối đa 3 câu hỏi con độc lập. Với câu hỏi đơn ý, Planner giữ nguyên mà không gọi thêm LLM.
- **Bước 4 – Supervisor định tuyến đến tác tử:** Supervisor nhận câu hỏi (hoặc từng câu hỏi con từ Planner) và thực hiện phân loại ý định thành bốn nhóm: `single_domain` (một miền đơn), `multi_domain` (đa miền), `form_procedure` (biểu mẫu/thủ tục) hoặc `grade_result` (tra điểm). Dựa trên phân loại đó, Supervisor chọn một hoặc nhiều tác tử chuyên môn. Với các câu hỏi rõ ràng về nghiệp vụ (MSSV, điểm chuẩn, danh sách thi, lịch KTHP), Supervisor ưu tiên fast-path bằng rule để giảm độ trễ, chỉ gọi LLM với những câu hỏi mơ hồ.
- **Bước 5 – Specialist Runner thực thi:** Mỗi Specialist Runner được giao câu hỏi và tự quyết định pipeline xử lý dựa trên chỉ số Q_c tính nội bộ: Q_c thấp $\rightarrow$ Native RAG: retrieve và generate một lần, không lặp; Q_c trung bình $\rightarrow$ Hybrid RAG: kết hợp grader đánh giá độ liên quan tài liệu, có thể leo lên Agentic nếu kết quả chưa đủ; Q_c cao $\rightarrow$ Agentic RAG: vòng lặp retrieve–grade–rewrite–generate nhiều lần. Một số trường hợp đặc thù có fast-path riêng: tra điểm theo MSSV dùng pipeline `grade_lookup`; bảng lịch thi/danh sách thi dùng `schedule_extract/exam_list_extract` để parse bảng trực tiếp mà không qua LLM.
- **Bước 6 – Aggregator tổng hợp (khi cần):** Nếu Supervisor gọi nhiều tác tử, các kết quả trả về được Aggregator hợp nhất bằng LLM thành một câu trả lời duy nhất, mạch lạc, giữ nguyên số liệu cụ thể từ từng chuyên gia và nêu rõ nguồn khi có mâu thuẫn.
- **Bước 7 – Cập nhật session và trả phản hồi:** Câu trả lời cuối cùng được trả về người dùng qua SSE streaming. Đồng thời, hệ thống cập nhật lịch sử hội thoại và tóm tắt phiên trong bộ nhớ session để phục vụ các lượt hỏi tiếp theo.

2.2 Cơ chế điều phối truy vấn trong hệ đa tác tử

Cơ chế điều phối là phần cốt lõi phân biệt kiến trúc đa tác tử với một chatbot RAG đơn giản. Thay vì đưa mọi câu hỏi vào cùng một pipeline cứng nhắc, hệ thống ta xây dựng tự động nhận diện phạm vi, làm rõ ngữ cảnh hội thoại, đánh giá độ phức tạp, lập kế hoạch phân rã và định tuyến đến đúng tác tử chuyên môn. Cơ chế này được thực hiện qua sáu thành phần điều phối: Guardrail, Query Rewriter, Planner, Supervisor, Router và Aggregator. Ngoài ra, Session Memory đóng vai trò hạ tầng bộ nhớ xuyên suốt, cung cấp ngữ cảnh hội thoại cho mọi thành phần.



Hình 2.3 Luồng xử lý điều phối đa tác tử

2.2.1 Bộ nhớ phiên trong luồng điều phối

2.2.1.1. Vai trò trong hệ thống

Trong hội thoại nhiều lượt, hệ thống cần ghi nhớ ngữ cảnh để xử lý các câu hỏi tiếp diễn, tránh người dùng phải lặp lại thông tin đã cung cấp và cho phép tóm tắt phiên phục vụ các thành phần điều phối phía sau. Session Memory là thành phần hạ tầng, không tham gia trực tiếp vào luồng xử lý câu hỏi nhưng cung cấp dữ liệu cho tất cả các thành phần khác.

2.2.1.2. Kiến trúc lưu trữ nóng-lạnh

Hệ thống áp dụng kiến trúc bộ nhớ hai tầng để cân bằng giữa tốc độ truy cập và độ bền dữ liệu, như được mô tả trong Bảng 2.4.

Tầng	Công nghệ	Thời gian sống (TTL)	Vai trò
Bộ nhớ nóng (hot)	Redis	2 giờ (7 200 giây)	Lưu session đang hoạt động, truy cập cực nhanh (< 1 ms)
Bộ nhớ lạnh (cold)	PostgreSQL	Vĩnh viễn	Lưu lịch sử hội thoại, phục vụ tra cứu và phân tích
Fallback	RAM (in-process)	Theo vòng đời process	Dùng khi Redis không khả dụng

Bảng 2.4 Kiến trúc bộ nhớ hai tầng của Session Memory

Mỗi phiên được biểu diễn bởi một `SessionState` – đối tượng dữ liệu tập trung lưu trữ toàn bộ thông tin cần thiết cho một lượt hội thoại. Bảng 2.5 mô tả cấu trúc của `SessionState`.

Trường	Kiểu dữ liệu	Ý nghĩa
<code>session_id</code>	<code>str</code>	Mã định danh phiên (UUID)
<code>summary</code>	<code>str</code>	Tóm tắt nội dung phiên, tối đa 1 200 ký tự
<code>turn_count</code>	<code>int</code>	Số lượt hội thoại đã thực hiện trong phiên
<code>hot_messages</code>	<code>list[dict]</code>	Tối đa 6 tin nhắn gần nhất để cung cấp ngữ cảnh nhanh
<code>last_agents</code>	<code>list[str]</code>	Các tác tử được dùng trong lượt trước
<code>last_form_filename</code>	<code>str</code>	Tên biểu mẫu đang xử lý nếu đang trong quy trình điền đơn
<code>student_profile</code>	<code>Student Profile</code>	Hồ sơ sinh viên tích lũy qua hội thoại (MSSV, tên, lớp)
<code>query_cache</code>	<code>dict</code>	Cache kết quả truy vấn trong phiên (TTL 600 giây)

Bảng 2.5 Cấu trúc dữ liệu `SessionState`

Tóm tắt phiên (`summary`) được cập nhật tự động sau mỗi 3 lượt hội thoại bằng LLM, giúp nén lịch sử dài thành một đoạn văn ngắn đủ để Guardrail, Query Rewriter và Supervisor sử dụng mà không cần đọc toàn bộ lịch sử. Toàn bộ lịch sử hội thoại được phân cắt theo từng mục đích sử dụng như Bảng 2.6, đảm bảo mỗi thành phần chỉ nhận lượng ngữ cảnh vừa đủ.

Mục đích	Số lượt tối đa	Thành phần sử dụng
Viết lại câu hỏi	8 lượt	Query Rewriter
Điều phối tác tử	6 lượt	Supervisor, Guardrail
Tạo sinh câu trả lời	10 lượt	Specialist RAG pipeline

Bảng 2.6 Giới hạn lịch sử hội thoại theo mục đích sử dụng

2.2.2 Guardrail và kiểm soát phạm vi câu hỏi

2.2.2.1. Vai trò trong hệ thống

Trong môi trường triển khai thực tế, chatbot học vụ sẽ nhận rất nhiều câu hỏi không liên quan đến KMA – từ các câu hỏi về trường khác, bài tập lập trình, đến lời chào đơn giản. Nếu để những câu hỏi này đi vào pipeline RAG, hệ thống sẽ lãng phí tài nguyên truy xuất vector, tốn chi phí gọi LLM và có nguy cơ sinh ra câu trả lời hallucination vì không có tài liệu KMA liên quan. Guardrail được thiết kế để loại sớm những câu hỏi này ngay từ cổng vào.

2.2.2.2. Phân loại câu hỏi

Guardrail phân loại mỗi câu hỏi vào một trong ba nhóm với hành vi phản hồi tương ứng như mô tả trong Bảng 2.7.

Nhóm	Đặc điểm	Hành vi hệ thống
kma (in-scope)	Liên quan đến học vụ, tài liệu KMA	Cho đi tiếp vào pipeline; answer = None
off_topic	Ngoài phạm vi KMA hoàn toàn	Phản hồi từ chối và liệt kê nghiệp vụ hỗ trợ; kết thúc
chitchat	Chào hỏi, cảm ơn, tạm biệt	Phản hồi thân thiện ngắn; kết thúc

Bảng 2.7 Phân loại câu hỏi và hành vi phản hồi của Guardrail

2.2.2.3. Chiến lược hai tầng: rule-based và LLM

Để vừa đảm bảo tốc độ cho các trường hợp phổ biến, vừa đảm bảo độ chính xác cho các trường hợp mơ hồ, Guardrail áp dụng chiến lược hai tầng xử lý tuần tự.

Tầng 1 – Kiểm tra rule-based (không gọi LLM): Hệ thống kiểm tra một tập từ khóa và mẫu regex được định nghĩa sẵn. Thứ tự ưu tiên kiểm tra từ cao đến thấp như Bảng 2.8.

Thứ tự	Điều kiện	Phán quyết
1	Đang trong phiên điền biểu mẫu (form_fill active)	kma – luôn cho qua
2	Câu hỏi rỗng	chitchat
3	Câu hỏi chứa MSSV (regex AT/CT + 6 số)	kma ngay
4	Câu hỏi chứa từ khóa off-topic mạnh và không có từ khóa KMA	off_topic ngay
5	Câu hỏi hỏi khả năng hỗ trợ và có từ KMA	chitchat (trả danh sách nghiệp vụ)
6	Câu hỏi là lời chào hỏi ngắn (≤ 12 từ) không có từ khóa KMA	chitchat ngay
7	Câu hỏi chứa từ khóa KMA rõ ràng	kma ngay
8	Câu ngắn trong phiên có hội thoại KMA trước đó	kma (theo ngữ cảnh phiên)

Bảng 2.8 Thứ tự kiểm tra rule-based của Guardrail

Tầng 2 – Gọi LLM (chỉ khi rule không kết luận được): Khi câu hỏi không khớp bất kỳ quy tắc nào ở tầng 1, Guardrail gọi LLM với prompt sau để đưa ra phán quyết cuối cùng:

Bạn là bộ lọc phạm vi cho chatbot KMA (Học viện Kỹ thuật Mật mã).
 Phạm vi IN-SCOPE: tuyển sinh, CTĐT, quy chế, khảo thí, chuẩn đầu ra, thi/tốt nghiệp, ma trận đề thi, điểm/kết quả công bố, biểu mẫu/đơn từ
 → KMA.
 OUT-OF-SCOPE: trường khác, kiến thức phổ thông không liên quan KMA, lập trình bài tập, thời tiết, giải trí, tin tức, y tế, chính trị,
 → v.v.
 CHITCHAT: chào hỏi, cảm ơn, tạm biệt (không hỏi nội dung KMA).

Tóm tắt phiên: {session_summary}
 Hội thoại gần đây: {history_text}
 Câu hỏi: {question}

Trả JSON (không markdown):
 {"category": "kma"|"off_topic"|"chitchat", "confidence": 0.0-1.0}

LLM trả về nhãn phân loại và độ tin cậy. Chỉ khi category = "off_topic" và confidence $\geq 0,55$, hệ thống mới từ chối câu hỏi – ngưỡng này được đặt thấp một chút để tránh nhầm lẫn các câu hỏi mơ hồ nhưng có liên quan KMA thành off-topic. Đáng chú ý, trong kiến trúc thực thi, Guardrail được chạy song song với bước chuẩn bị

phiên (`_prepare_turn`) để giảm tổng thời gian chờ. Kết quả phân loại của Guardrail với câu hỏi gốc được dùng làm tham chiếu sớm, trong khi kết quả chính thức được tính lại sau khi Query Rewriter đã làm rõ câu hỏi.

Kết quả của Guardrail là đối tượng `ScopeResult` với cấu trúc như Bảng 2.9.

Trường	Kiểu dữ liệu	Ý nghĩa
<code>in_scope</code>	<code>bool</code>	True nếu câu hỏi được phép đi tiếp; False để kết thúc sớm
<code>category</code>	<code>str</code>	Nhãn: <code>kma</code> / <code>off_topic</code> / <code>chitchat</code>
<code>answer</code>	<code>str</code> <code>None</code>	Câu trả lời mặc định nếu không cần vào pipeline; <code>None</code> nếu in-scope
<code>confidence</code>	<code>float</code>	Độ tin cậy của phán quyết LLM, trong khoảng <code>[0, 1]</code>

Bảng 2.9 Cấu trúc kết quả `ScopeResult` của Guardrail

2.2.3 Query Rewriter và chuẩn hóa câu hỏi

2.2.3.1. Vấn đề cần giải quyết

Trong hội thoại nhiều lượt, người dùng thường đặt câu hỏi phụ thuộc ngữ cảnh. Ví dụ, sau khi hỏi về điểm chuẩn ngành CNTT, người dùng có thể hỏi tiếp “Còn ngành ATTT thì sao?” hay “Thủ tục nhập học bạn ấy như nào?”. Những câu hỏi này nếu được đưa thẳng vào hệ thống truy xuất sẽ cho kết quả kém vì thiếu chủ thể và ngữ cảnh cụ thể. Query Rewriter giải quyết bài toán này bằng cách viết lại câu hỏi thành dạng đầy đủ nghĩa, có thể đứng độc lập.

2.2.3.2. Điểm heuristic và điều kiện gọi LLM

Một thách thức quan trọng là không phải câu hỏi nào cũng cần viết lại. Nếu gọi LLM cho mọi câu hỏi, chi phí sẽ tăng gấp đôi không cần thiết. Query Rewriter giải quyết điều này bằng một bộ tính điểm heuristic – chỉ khi tổng điểm vượt ngưỡng 0,65 mới gọi LLM rewrite. Các tiêu chí tính điểm được mô tả trong Bảng 2.10.

Tiêu chí phát hiện	Thay đổi điểm
Câu chứa từ follow-up mạnh: còn, đó, thế, vậy, sao, nó	+0,55
Câu chứa đại từ ngôi thứ ba: bạn ấy, anh ấy, sinh viên đó	+0,30
Câu rất ngắn (≤ 8 từ)	+0,20
Câu có nhiều dấu hỏi (≥ 2)	+0,15
Câu đã chứa từ khóa KMA cụ thể (tên ngành, môn, MSSV...)	-0,20
Câu chứa MSSV dạng AT/CT (regex)	-0,30
Câu dài (≥ 20 từ) và không có dấu follow-up	-0,15
Chứa nghiệp vụ cụ thể (phức khảo, thực tập, tài chính)	Điểm = 0 (không rewrite)

Bảng 2.10 Tiêu chí tính điểm heuristic của Query Rewriter

Bên cạnh điểm số, hệ thống còn kiểm tra bổ sung: nếu câu hỏi chứa một trong các từ follow-up mạnh nhất (đó, còn, thế, vậy) hoặc câu ngắn (≤ 8 từ) trong phiên đang có lịch sử hội thoại, Query Rewriter sẽ gọi LLM rewrite bất kể điểm heuristic. Cơ chế này đảm bảo không bỏ sót các follow-up rõ ràng.

2.2.3.3. Prompt viết lại câu hỏi

Bạn hỗ trợ chatbot Học viện KMA.

Nhiệm vụ: viết lại câu hỏi MỖI của sinh viên thành câu ĐỦ NGHĨA, đúng một mình.

Ngữ cảnh phiên (tóm tắt trước): {session_summary}

Hội thoại gần đây: {history_text}

Câu hỏi mới (có thể dùng "đó", "còn", "nữa", ...): {question}

Quy tắc:

- Giữ nguyên ý; bổ sung chủ thể/mảng (tuyển sinh, biểu mẫu, quy chế, MSSV, ngành, môn...) nếu thiếu.
- Nếu hội thoại trước có MSSV, ngành, môn, biểu mẫu -- PHẢI đưa vào câu rewrite (ví dụ: "CT060310", "CNTT", "Tiếng Anh 2").
- Không trả lời câu hỏi, chỉ xuất MỘT câu hỏi tiếng Việt (một dòng).
- Nếu câu đã đủ rõ, trả lại gần như nguyên văn.

Query Rewriter sử dụng tối đa 8 lượt hội thoại gần nhất làm ngữ cảnh và giới hạn đầu ra LLM ở 120 token, đảm bảo câu viết lại ngắn gọn, tập trung vào câu hỏi chứ không lan man. Kết quả trả về là một đối tượng RewriteResult như Bảng 2.11.

Trường	Kiểu dữ liệu	Ý nghĩa
original	str	Câu hỏi gốc từ người dùng trước khi xử lý
retrieval_query	str	Câu hỏi sau khi viết lại hoặc giữ nguyên nếu không cần
was_rewritten	bool	Đánh dấu câu hỏi có thực sự được viết lại hay không

Bảng 2.11 Cấu trúc kết quả RewriteResult của Query Rewriter

2.2.4 Planner và phân rã truy vấn phức tạp

2.2.4.1. Vấn đề cần giải quyết

Người dùng trong môi trường học vụ thường đặt câu hỏi có nhiều ý độc lập trong cùng một lượt: vừa hỏi về tuyển sinh, vừa hỏi về thủ tục nhập học; hay câu hỏi có nhiều điểm cần tra cứu từ nhiều miền tài liệu khác nhau. Nếu đưa nguyên câu hỏi đa ý này cho một tác tử đơn, tác tử đó không thể chuyên sâu vào cả hai miền đồng thời. Planner giải quyết điều này bằng cách phân rã câu hỏi đa ý thành các câu hỏi con độc lập – mỗi câu con sau đó được Supervisor gán riêng cho tác tử phù hợp nhất.

2.2.4.2. Cơ chế quyết định phân rã

Planner không phân rã mọi câu hỏi. Việc phân rã câu hỏi đơn giản thành nhiều câu con vừa không cần thiết, vừa tăng chi phí LLM và tăng số lượng tác tử được gọi. Vì vậy, Planner thực hiện kiểm tra heuristic trước; chỉ khi ít nhất một tiêu chí trong Bảng 2.12 được thỏa mãn, LLM mới được gọi để phân rã.

Tiêu chí	Ngưỡng	Giải thích
Số từ trong câu hỏi	≥ 22 từ	Câu hỏi dài thường chứa nhiều ý
Số dấu hỏi trong câu	≥ 2	Dấu hiệu rõ ràng của câu hỏi kép
Chỉ số Q_c cục bộ	$\geq 0,62$	Câu hỏi phức tạp theo đánh giá nội bộ
Từ nối đa ý	và, đồng thời, so sánh, cùng lúc, ngoài ra	Dấu hiệu ngữ nghĩa của câu hỏi đa ý
Câu là dạng bảng lịch/danh sách thi	–	Không phân rã – áp dụng fast-path riêng

Bảng 2.12 Tiêu chí quyết định phân rã câu hỏi của Planner

Khi phân rã được quyết định, Planner gọi LLM với prompt sau:

Bạn là Planner của chatbot KMA.

Tách câu hỏi thành các ý con (sub-question) độc lập để từng chuyên gia trả lời.

Tóm tắt phiên: {session_summary}

Hội thoại gần đây: {history_text}

Câu hỏi (đã làm rõ): {question}

Quy tắc:

- Nếu câu chỉ MỘT ý đơn giản thì trả sub_questions có 1 phần tử (giữ nguyên câu).
- Nếu nhiều ý (so sánh, và, đồng thời, nhiều mảng: tuyển sinh + biểu mẫu + quy chế...) thì tách tối đa {max_n} câu con, mỗi câu đủ nghĩa.
- Không trả lời nội dung, chỉ JSON.

Trả lời ĐÚNG MỘT JSON:

```
{"sub_questions": ["câu 1", "câu 2"], "reason": "giải thích ngắn"}
```

Giới hạn $\text{max_n} = 3$ được thiết lập nhằm kiểm soát số lượng tác tử tối đa được gọi song song. Khi câu hỏi quá dài có thể chứa hơn 3 ý, Planner chỉ giữ 3 câu con quan trọng nhất. Việc phân rã quá nhỏ không làm tăng chất lượng mà chỉ tăng số lần gọi LLM và thời gian chờ. Kết quả của Planner là đối tượng PlanResult như Bảng 2.13.

Trường	Kiểu dữ liệu	Ý nghĩa
sub_questions	list[str]	Danh sách câu hỏi con; 1 phần tử nếu không phân rã
use_decomposition	bool	True nếu đã phân rã thành nhiều câu con; False nếu giữ nguyên
reason	str	Lý do phân rã hoặc giữ nguyên, do LLM hoặc heuristic cung cấp

Bảng 2.13 Cấu trúc kết quả PlanResult của Planner

Kết quả phân rã của Planner được chuyển ngay cho Supervisor. Nếu cờ phân rã có giá trị True, mỗi câu hỏi con được Supervisor định tuyến độc lập; nếu có giá trị False, câu hỏi gốc được định tuyến như bình thường.

2.2.5 Supervisor và định tuyến tác tử

2.2.5.1. Nguyên lý hoạt động

Supervisor là thành phần điều phối trung tâm của toàn hệ thống. Sau khi nhận câu hỏi từ Planner, Supervisor thực hiện hai nhiệm vụ: (1) phân loại ý định nghiệp vụ của câu hỏi và (2) chọn một hoặc nhiều tác tử chuyên môn phù hợp. Supervisor không trả lời câu hỏi – đây là nguyên tắc thiết kế quan trọng đảm bảo phân tách trách nhiệm rõ ràng. Quyết định cuối cùng của Supervisor bao gồm danh sách tác tử được gọi, tác tử chính, intent và độ tin cậy của phán quyết.

2.2.5.2. Phân loại intent nghiệp vụ

Hệ thống định nghĩa bốn loại intent ở mức Supervisor. Việc phân loại intent không chỉ phục vụ định tuyến tác tử mà còn ảnh hưởng đến việc chọn pipeline xử lý phía sau – intent `grade_result` và `multi_domain` sẽ được Router nâng pipeline lên mức cao hơn để đảm bảo độ chính xác. Bảng 2.14 mô tả chi tiết bốn intent này.

Intent	Đặc điểm	Ví dụ câu hỏi	Hành vi định tuyến
<code>single_domain</code>	Câu hỏi thuộc một mảng nghiệp vụ duy nhất	“Chuẩn đầu ra TOEIC là bao nhiêu?”	1 agent
<code>multi_domain</code>	Câu hỏi liên quan hai mảng trở lên	“Điểm chuẩn CNTT 2024 và mẫu đơn nhập học?”	2–3 agent song song
<code>form_procedure</code>	Câu hỏi về đơn từ, biểu mẫu, thủ tục	“Tải mẫu đơn xin nghỉ học ở đâu?”	Ưu tiên <code>bieu_mau</code>
<code>grade_result</code>	Tra cứu điểm, bảng điểm, MSSV	“CT060310 điểm học kỳ 2 năm 2024–2025?”	Chỉ <code>diem_thi</code>

Bảng 2.14 Phân loại intent của Supervisor

2.2.5.3. Chiến lược fast-path và LLM

Supervisor áp dụng chiến lược xử lý nhiều tầng nhằm tối ưu tốc độ phản hồi. Phần lớn các câu hỏi học vụ có tín hiệu nghiệp vụ rõ ràng và có thể được định tuyến bằng rule mà không cần gọi LLM. Các fast-path được áp dụng theo thứ tự ưu tiên như Bảng 2.15.

Thứ tự	Điều kiện phát hiện	Agent được chọn	Intent
1	MSSV (AT/CT + 6 số) + từ khóa danh sách/phòng/ca thi	danh_sach_thi	single_domain
2	Số báo danh (SBD) + từ khóa danh sách/phòng/ca thi	danh_sach_thi	single_domain
3	Từ khóa liệt kê môn thi + thời kỳ học kỳ/đợt	lich_thi	single_domain
4	MSSV + từ khóa điểm/kết quả/học kỳ	diem_thi	grade_result
5	Từ khóa môn + thời gian/ngày thi/địa điểm thi	lich_thi	single_domain
6	Nhiều mảng rõ (tuyển sinh + phí/biểu mẫu)	tuyen_sinh + bieu_mau	multi_domain

Bảng 2.15 Các fast-path theo thứ tự ưu tiên của Supervisor

Chỉ khi không có fast-path nào khớp, Supervisor mới gọi LLM với prompt sau. Supervisor chỉ truyền vào LLM 6 lượt hội thoại gần nhất để giữ token context ngắn gọn.

Bạn là Supervisor -- lớp điều phối đa tác tử của chatbot Học viện
→ KMA.

Nhiệm vụ: phân loại ý định sinh viên và chọn agent chuyên môn
(không trả lời câu hỏi).

Các agent (corpus Qdrant riêng): {agent_list}

Intent nghiệp vụ (chọn một):

single_domain, multi_domain, form_procedure, grade_result.

Quy tắc chọn agent:

- grade_result / MSSV -> chỉ diem_thi.
- tuyển sinh, CTĐT, điểm chuẩn, ngưỡng -> tuyen_sinh.
- quy chế, chuẩn đầu ra ngoại ngữ -> khao_thi.
- ma trận đề, cấu trúc đề thi -> ma_tran.
- danh sách dự thi, phòng/ca thi -> danh_sach_thi.
- lịch KTHP, giờ/ngày/địa điểm thi môn học phần -> lich_thi.
- Tối đa 3 agent; primary = agent trả lời chính.

Tóm tắt phiên: {session_summary}

Hội thoại gần đây: {history_text}

Câu hỏi: {question}

Trả lời ĐÚNG MỘT JSON (không markdown):

```
{
  "intent": "...",
  "agents": ["agent_id"],
  "primary": "agent_id",
  "confidence": 0.0-1.0,
  "reason": "1-2 câu tiếng Việt"
}
```

2.2.5.4. Cơ chế blending khi độ tin cậy thấp

Khi LLM trả về kết quả với độ tin cậy dưới 0,75, Supervisor không tin tưởng hoàn toàn vào quyết định LLM mà kết hợp thêm kết quả từ keyword fallback. Cơ chế blending hoạt động như sau: tính điểm từ khóa riêng cho từng agent dựa trên keywords đã định nghĩa, sau đó hợp nhất danh sách agent từ LLM và từ keyword scoring, đặt agent có điểm keyword cao nhất làm primary nếu nó có confidence cao hơn quyết định LLM. Cơ chế này giúp hệ thống tránh định tuyến sai khi LLM không chắc chắn, đặc biệt quan trọng trong chế độ ACCURACY_MODE.

2.2.5.5. Cache định tuyến theo phiên

Supervisor tích hợp cơ chế cache kết quả định tuyến: nếu cùng một câu hỏi (sau chuẩn hóa) được hỏi lại trong vòng 300 giây, quyết định định tuyến được lấy từ cache mà không gọi LLM. Điều này đặc biệt hữu ích khi người dùng hỏi lại câu hỏi tương tự hoặc trong các kịch bản kiểm thử tải. Kết quả của Supervisor là đối tượng RoutingDecision như Bảng 2.16.

Trường	Kiểu dữ liệu	Ý nghĩa
agents	list[str]	Danh sách agent được chọn (tối đa 3)
primary	str	Agent chính – được ưu tiên kết quả khi tổng hợp
intent	str	Intent: single_domain / multi_domain / form_procedure / grade_result
confidence	float	Độ tin cậy của phán quyết, trong khoảng [0, 1]
reason	str	Giải thích ngắn lý do chọn agent, bằng tiếng Việt

Bảng 2.16 Cấu trúc kết quả RoutingDecision của Supervisor

2.2.6 Router và lựa chọn pipeline xử lý

2.2.6.1. Vị trí trong kiến trúc

Sau khi Supervisor giao việc cho tác tử chuyên môn, mỗi Specialist Runner cần quyết định cách xử lý câu hỏi: sử dụng pipeline đơn giản, trung bình hay phức tạp. Đây là nhiệm vụ của Router – thành phần định tuyến pipeline bên trong mỗi tác tử, hoàn toàn độc lập với các tác tử khác. Thiết kế này cho phép từng tác tử tự tối ưu chiến lược xử lý theo đặc thù câu hỏi và dữ liệu của mình.

Router nhận đầu vào là kết quả đánh giá độ phức tạp (ComplexityAssessment) do QC Calculator cung cấp và chọn một trong ba pipeline theo ngưỡng Q_c , như mô tả trong Bảng 2.17.

Vùng Q_c	Pipeline	Chiến lược xử lý
$Q_c < T_1 = 0,50$	native_rag	Retrieve một lần, generate trực tiếp không lặp
$T_1 \leq Q_c < T_2 = 0,70$	hybrid_rag	Retrieve + Document Grader đánh giá độ liên quan; leo lên agentic nếu kết quả chưa đủ
$Q_c \geq T_2 = 0,70$	agentic_rag	Vòng lặp retrieve–grade–rewrite–generate nhiều vòng đến khi đủ context

Bảng 2.17 Quy tắc định tuyến pipeline theo Q_c

2.2.6.2. Điều chỉnh pipeline theo intent Supervisor

Ngưỡng Q_c là cơ sở chính nhưng không phải yếu tố duy nhất. Router còn điều chỉnh pipeline theo intent từ Supervisor để đảm bảo không thiếu context khi câu hỏi thuộc các loại nghiệp vụ đặc biệt. Bảng 2.18 mô tả các điều chỉnh này.

Intent Supervisor	Pipeline gốc theo Q_c	Pipeline sau điều chỉnh	Lý do
grade_result	native_rag	hybrid_rag	Tra điểm cần nhiều chunk tài liệu
form_procedure	native_rag	hybrid_rag	Biểu mẫu cần đánh giá độ liên quan chặt hơn
multi_domain	native_rag	hybrid_rag	Câu hỏi đa miền cần context phong phú hơn
multi_domain	hybrid_rag	agentic_rag	Đa miền phức tạp cần nhiều vòng truy xuất

Bảng 2.18 Điều chỉnh pipeline theo intent Supervisor

Ngoài ra, khi Supervisor có confidence < 0,75, Router tự động nâng pipeline từ native_rag lên hybrid_rag để bù đắp cho độ không chắc chắn của định tuyến, đảm bảo không bỏ sót context khi agent được giao việc có thể không hoàn toàn phù hợp. Cơ chế leo thang pipeline (escalation) cũng được áp dụng trong quá trình thực thi: nếu native_rag hoặc hybrid_rag cho kết quả “không tìm thấy” hoặc Document Grader đánh giá tài liệu không đủ liên quan, Specialist Runner tự động leo lên agentic_rag để thực hiện thêm vòng truy xuất.

2.2.6.3. Fast-path đặc thù cho dữ liệu bảng

Với hai loại câu hỏi tra cứu từ tài liệu dạng bảng (lịch thi KTHP và danh sách dự thi), hệ thống không đi qua pipeline RAG thông thường mà sử dụng fast-path phân tích bảng trực tiếp (schedule_extract và exam_list_extract). Cơ chế này parse dữ liệu bảng từ các chunk đã retrieve, trích xuất thông tin chính xác (môn, ngày, phòng, ca thi, MSSV/SBD) mà không qua LLM generate – vừa nhanh hơn đáng kể, vừa tránh lỗi hallucination số liệu bảng của LLM.

2.2.7 Aggregator và tổng hợp câu trả lời đa tác tử

2.2.7.1. Điều kiện kích hoạt

Aggregator chỉ được kích hoạt khi Supervisor điều phối từ hai tác tử trở lên. Với câu hỏi đơn miền (một tác tử), kết quả được trả trực tiếp để tiết kiệm một lượt gọi LLM. Khi có nhiều tác tử cùng tham gia, mỗi tác tử trả về một câu trả lời độc lập theo chuyên môn của mình. Nếu không có bước tổng hợp, người dùng sẽ nhận nhiều phản hồi rời rạc

và có thể mâu thuẫn. Aggregator hợp nhất các câu trả lời này thành một phản hồi duy nhất, mạch lạc, không trùng lặp và giữ nguyên độ chính xác của từng nguồn.

2.2.7.2. Cấu trúc đầu vào và prompt tổng hợp

Đầu vào của Aggregator gồm câu hỏi gốc, tóm tắt phiên và danh sách khối kết quả từ các tác tử. Mỗi khối kết quả có cấu trúc như Bảng 2.19.

Trường	Ý nghĩa
agent_id	Mã định danh tác tử
agent_name	Tên hiển thị tác tử
pipeline	Pipeline đã sử dụng (native_rag / hybrid_rag / agentic_rag)
qc	Chỉ số Q_c của câu hỏi tại tác tử này
answer	Câu trả lời chuyên môn của tác tử

Bảng 2.19 Cấu trúc một khối kết quả tác tử đầu vào Aggregator

Prompt tổng hợp sử dụng như sau:

Bạn là Aggregator của hệ thống trợ lý đa tác tử KMA.

Nhiệm vụ: gộp các câu trả lời từ nhiều chuyên gia thành MỘT câu trả lời
 ↪ mạch lạc cho sinh viên.

Tóm tắt phiên (nếu có): {session_summary}

Câu hỏi gốc: {question}

Các chuyên gia đã trả lời: {blocks}

Yêu cầu:

- Giữ đủ thông tin quan trọng từ mỗi chuyên gia, tránh trùng lặp.
- Nêu rõ từng phần nếu câu hỏi đa lĩnh vực.
- Không bịa thêm ngoài nội dung các chuyên gia đã cung cấp.
- Ưu tiên độ chính xác: không gộp làm mất số liệu; nếu hai chuyên gia mâu thuẫn, nêu rõ theo từng nguồn.
- Tiếng Việt, lịch sự, dễ đọc.

Giới hạn đầu ra của Aggregator là 1 400 token trong chế độ ACCURACY_MODE – giới hạn này đủ để tổng hợp đầy đủ câu trả lời từ tối đa 3 tác tử, đồng thời tránh câu trả lời quá dài gây khó đọc cho người dùng. Kết quả cuối cùng được trả về người dùng qua cơ chế SSE streaming, đồng thời được ghi vào lịch sử phiên để phục vụ các lượt hội thoại tiếp theo.

2.3 Phương pháp đánh giá độ phức tạp truy vấn

2.3.1 Bài toán lựa chọn mức xử lý

Trong kiến trúc RAG thông thường, tất cả câu hỏi đều đi qua một pipeline xử lý duy nhất bất kể mức độ phức tạp. Cách tiếp cận này dẫn đến hai hệ quả đối lập: nếu chọn pipeline đơn giản cho toàn bộ câu hỏi, những truy vấn đòi hỏi suy luận đa bước sẽ nhận được câu trả lời thiếu hoặc sai vì ngữ cảnh truy xuất không đủ; nếu chọn pipeline phức tạp cho toàn bộ, những câu hỏi tra cứu đơn giản sẽ tiêu tốn tài nguyên gấp 3–5 lần và tăng độ trễ phản hồi không cần thiết.

Để giải quyết sự đánh đổi này, hệ thống đề xuất chỉ số *Query Complexity* (Q_c) – một giá trị số thực trong khoảng $[0, 1]$ định lượng mức độ phức tạp của từng câu hỏi. Q_c được tính ngay trước khi mỗi Specialist Runner xử lý và được chuyển cho Router để lựa chọn pipeline tối ưu giữa Native RAG, Hybrid RAG và Agentic RAG. Sự phân chia pipeline theo Q_c giúp hệ thống vận hành hiệu quả cả về chất lượng lẫn chi phí vận hành.

Điểm đặc sắc của thiết kế Q_c trong hệ thống này là chỉ số không chỉ phụ thuộc vào đặc điểm ngôn ngữ thuần túy của câu hỏi mà còn kết hợp độ mơ hồ ngữ nghĩa thực tế – tức là mức độ phân tán của kết quả truy xuất vector trên cơ sở dữ liệu đang vận hành. Điều này làm cho Q_c trở thành chỉ số phụ thuộc dữ liệu, phản ánh đặc thù của corpus tài liệu KMA thay vì chỉ đánh giá bề mặt ngôn ngữ.

2.3.2 Thiết kế công thức Q_c

2.3.2.1. Nguyên tắc thiết kế

Công thức Q_c được thiết kế theo ba nguyên tắc cốt lõi. Thứ nhất, đo lường đa chiều: độ phức tạp của một câu hỏi không thể được phản ánh bởi một chiều duy nhất; cần kết hợp chiều thực thể, chiều quan hệ, chiều độ dài và chiều ngữ nghĩa. Thứ hai, không phụ thuộc LLM cho trường hợp thông thường: gọi LLM để tính Q_c cho mọi câu hỏi sẽ nhân đôi chi phí và độ trễ; ba trong bốn thành phần E , R , L được tính cục bộ bằng phân tích từ khóa và thư viện NLP nhẹ. Thứ ba, tích hợp phản hồi thực tế từ hệ thống: thành phần S được tính từ kết quả truy xuất thực tế của Qdrant, cho phép công thức phản ánh đặc thù của cơ sở dữ liệu cụ thể.

2.3.2.2. Công thức tổng quát

$$Q_c = w_E \cdot E + w_R \cdot R + w_L \cdot L + w_S \cdot S, \quad Q_c \in [0, 1]$$

Các thành phần E , R , L , $S \in [0, 1]$ là bốn đặc trưng độc lập của câu hỏi; w_E , w_R , w_L , w_S là trọng số được xác định qua thực nghiệm trên tập câu hỏi học vụ KMA, thỏa mãn:

$$w_E + w_R + w_L + w_S \approx 1$$

Bảng 2.20 Các thành phần và trọng số trong công thức Q_c

Thành phần	Ý nghĩa	Trọng số	Cơ sở lựa chọn trọng số
E – Entity	Mật độ thực thể	$w_E = 0,150$	Thực thể đóng góp vào độ phức tạp nhưng không phải yếu tố quyết định nếu thiếu quan hệ
R – Relation	Mật độ quan hệ logic	$w_R = 0,376$	Quan hệ là chỉ báo mạnh nhất cho câu hỏi đa bước trong nhóm thành phần cục bộ
L – Length	Độ dài câu hỏi	$w_L = 0,089$	Độ dài không tương quan mạnh với độ phức tạp; câu ngắn có thể rất phức tạp và ngược lại
S – Semantic Ambiguity	Độ mơ hồ ngữ nghĩa	$w_S = 0,386$	Phản ánh trực tiếp mức độ khó truy xuất của câu hỏi trên corpus thực tế; là trọng số lớn nhất

Việc S có trọng số lớn nhất xuất phát từ quan sát thực nghiệm: cùng một câu hỏi có thể đơn giản với corpus này nhưng phức tạp với corpus khác, tùy thuộc mức độ tập trung thông tin trong tài liệu. Thành phần S đưa yếu tố phụ thuộc corpus này vào phép tính, làm Q_c phản ánh đặc thù của hệ thống đang triển khai.

2.3.3 Các thành phần của chỉ số Q_c

2.3.3.1. Thành phần E – Mật độ thực thể

$$E = \min \left(\frac{|\mathcal{E}|}{3}, 1, 0 \right)$$

$|\mathcal{E}|$ là số thực thể duy nhất phát hiện được trong câu hỏi. Ngưỡng bão hòa là 3; câu hỏi có từ 3 thực thể trở lên đều đạt $E = 1,0$. Tập hợp $\mathcal{E}$ được xây dựng từ hai nguồn kết hợp.

Nguồn thứ nhất là từ khóa KMA định nghĩa sẵn. Danh sách 20 thuật ngữ đặc thù được kiểm tra theo khớp chuỗi con sau khi chuẩn hóa Unicode NFC và chuyển thường,

bao gồm: kma, học viện, mssv, at20, ctdt, tuyển sinh, khảo thí, quy chế, ma trận, biểu mẫu, bảng điểm, phúc khảo, bảo lưu, attt, cntt, dtvt, phòng đào tạo, phòng khảo thí, học kỳ, đợt thi. Regex `\b(?:ATCT)6\b` nhận diện thêm MSSV dạng ATxxxxxx hoặc CTxxxxxx.

Nguồn thứ hai là spaCy NER. Mô hình ngôn ngữ nhận diện thực thể có nhãn PERSON, ORG, GPE, LOC, FAC, PRODUCT, EVENT và danh từ riêng (POS tag PROP), bổ sung các thực thể không có trong danh sách từ khóa định sẵn. Hệ thống ưu tiên model `vi_core_news_lg`, lùi về `vi_core_news_sm` rồi `en_core_web_sm` tùy theo model nào đã cài đặt.

Câu hỏi	Thực thể phát hiện	$ \mathcal{E} $	E
“Mã trường KMA là gì?”	{kma}	1	0,333
“Quy chế KMA năm 2024 có điểm gì mới?”	{kma, quy chế}	2	0,667
“MSSV CT060310 học kỳ 1 ngành CNTT điểm bao nhiêu?”	{CT060310, học kỳ, cntt}	3	1,000

Bảng 2.21 Ví dụ tính thành phần E

2.3.3.2. Thành phần R – Mật độ quan hệ logic

$$R = \min \left(\frac{|\mathcal{R}|}{3}, 1, 0 \right)$$

$\mathcal{R}$ là tập hợp các quan hệ logic phát hiện được, xây dựng từ hai cơ chế. Cơ chế thứ nhất là từ khóa quan hệ: hệ thống kiểm tra sự xuất hiện của 20 từ khóa quan hệ với khoảng trắng hai bên để tránh khớp nhầm trong từ ghép, gồm: hoặc, hoặc là, nếu, thì, khi, so sánh, khác nhau, giống, hơn, ít hơn, bằng, hay, đồng thời, cùng lúc, nhưng, tuy, vì, quy trình, các bước, điều kiện.

Cơ chế thứ hai là dependency tags từ spaCy. Các nhãn cú pháp đặc trưng của câu phức tạp được trình bày trong Bảng 2.22.

Dependency tag	Ý nghĩa ngữ pháp	Ví dụ
conj	Liên kết ngang hàng	“tuyển sinh và nhập học”
advcl	Mệnh đề trạng ngữ	“Nếu sinh viên thiếu chứng chỉ, thì phải...”
ccomp	Mệnh đề bổ ngữ nội tại	“Biết rằng quy định yêu cầu...”
xcomp	Bổ ngữ mở	“Sinh viên cần đạt TOEIC 450”
relcl	Mệnh đề quan hệ	“Quy định mà khoa ban hành...”

Bảng 2.22 Các dependency tags phát hiện quan hệ phức tạp

R nhận trọng số cao nhất trong nhóm thành phần cục bộ ($w_R = 0,376$) vì quan hệ ngữ pháp là chỉ báo mạnh nhất cho câu hỏi đa bước. Câu chứa nhiều quan hệ điều kiện hoặc so sánh gần như chắc chắn đòi hỏi pipeline phức tạp hơn.

2.3.3.3. Thành phần L – Độ dài câu hỏi

$$L = \min \left(\frac{N_{token}}{30}, 1,0 \right)$$

N_{token} là số token hợp lệ, gồm chữ và số, không tính dấu câu, được đếm bằng biểu thức chính quy `[\wA-ỹ]+` hỗ trợ Unicode tiếng Việt. Câu từ 30 token trở lên đạt $L = 1,0$.

L có trọng số thấp nhất ($w_L = 0,089$) vì độ dài câu hỏi không tương quan chặt với độ phức tạp: câu dài có thể chỉ là diễn giải dài dòng của một ý đơn giản, trong khi câu ngắn như “AT200201 có đủ điều kiện tốt nghiệp không?” đòi hỏi tra cứu đa bước. L chỉ đóng góp đáng kể ở hai cực: câu rất ngắn dưới 5 từ cho L thấp kéo Q_c xuống; câu rất dài trên 30 từ thường chứa nhiều ý, góp phần đẩy Q_c lên.

2.3.3.4. Thành phần S – Độ mơ hồ ngữ nghĩa

Thành phần S là thành phần đặc sắc nhất trong công thức. Thay vì chỉ phân tích đặc điểm ngôn ngữ, S đo lường trực tiếp mức độ mơ hồ của câu hỏi khi chiếu lên không gian vector của cơ sở dữ liệu đang vận hành. Đây là đặc trưng phân biệt thiết kế Q_c của hệ thống này với các phương pháp đánh giá độ phức tạp thuần ngôn ngữ học.

Nguyên lý như sau: nếu câu hỏi có đáp án tập trung trong một vài tài liệu cụ thể, truy xuất vector sẽ trả về một vài tài liệu có điểm tương đồng vượt trội rõ rệt so với phần còn lại; phân phối nhón này cho thấy câu hỏi dễ xử lý. Ngược lại, nếu nhiều tài liệu có

điểm tương đồng gần bằng nhau, phân phối phẳng cho thấy hệ thống không biết tài liệu nào thực sự liên quan, tức câu hỏi mơ hồ và cần pipeline mạnh hơn.

Các bước tính S gồm bốn bước. Bước 1, hệ thống prefetch $K = 5$ tài liệu đầu từ Qdrant cho câu hỏi và lấy danh sách điểm tương đồng cosine $\{s_1, \dots, s_5\}$. Bước 2, hệ thống biến đổi điểm tương đồng thành phân phối xác suất bằng softmax, trừ $s_{\max}$ trước để tránh tràn số:

$$p_i = \frac{e^{s_i - s_{\max}}}{\sum_{j=1}^K e^{s_j - s_{\max}}}, \quad i = 1, \dots, K$$

Bước 3, hệ thống tính Shannon entropy:

$$H = - \sum_{i=1}^K p_i \ln p_i$$

H đạt cực tiểu $H = 0$ khi toàn bộ xác suất tập trung vào một tài liệu duy nhất; H đạt cực đại $H_{\max} = \ln K$ khi phân phối hoàn toàn đều. Bước 4, hệ thống chuẩn hóa entropy:

$$S = \frac{H}{\ln K}, \quad S \in [0, 1]$$

Ví dụ, câu hỏi “Mã trường KMA là gì?” trả về 5 điểm tương đồng $\{0,92; 0,45; 0,41; 0,38; 0,35\}$. Điểm đầu vượt trội rõ rệt; sau softmax phân phối là $\{0,82; 0,05; 0,04; 0,05; 0,04\}$, entropy $H = 0,63$, suy ra $S = 0,39$ – thấp, đúng với kỳ vọng câu hỏi đơn giản. Với câu hỏi mơ hồ “Sinh viên cần chuẩn bị những gì?”, 5 điểm là $\{0,71; 0,68; 0,65; 0,63; 0,61\}$; phân phối gần đều, entropy $H = 1,60$, suy ra $S = 0,99$, phản ánh đúng tính mơ hồ cao.

2.3.4 Chế độ đánh giá trong triển khai

Hệ thống cung cấp hai chế độ đánh giá Q_c cho môi trường production, lựa chọn theo yêu cầu về chi phí và độ chính xác.

2.3.4.1. Chế độ Local QC

Khi USE_LOCAL_QC=1, hệ thống tính Q_c hoàn toàn theo công thức bốn thành phần $E/R/L/S$ mà không gọi LLM. Thành phần S vẫn được tính qua prefetch Qdrant với $K = 5$ kết quả. Mỗi lượt đánh giá chỉ tốn một truy vấn Qdrant nhỏ, không phát sinh chi phí API LLM, độ trễ thường dưới 50 ms.

Hạn chế chính là các câu hỏi nằm trong vùng biên, tức Q_c gần ngưỡng T_1 hoặc T_2 , dễ bị phân loại sai pipeline do thiếu ngữ cảnh ngữ nghĩa sâu. Chế độ này phù hợp cho triển khai cần tối ưu chi phí vận hành.

2.3.4.2. Chế độ Hybrid (mặc định trong Accuracy Mode)

Đây là chế độ mặc định khi `ACCURACY_MODE=1`, kết hợp tính cục bộ làm nền và gọi LLM bổ sung chỉ khi thực sự cần thiết.

Bước 1, hệ thống tính Q_c cục bộ theo $E/R/L/S$. Bước 2, hệ thống quyết định gọi LLM khi thỏa một trong ba điều kiện trong Bảng 2.23.

Điều kiện	Ngưỡng	Giải thích
Câu hỏi đa ý	≥ 2 dấu hỏi, ≥ 24 từ, hoặc chứa từ nối đa ý	Câu hỏi chứa nhiều câu con cần LLM phân tích cấu trúc
Intent cục bộ thuộc nhóm phức tạp	<code>compare</code> , <code>multi_hop</code> , <code>procedural</code> , <code>list_enumerate</code>	LLM xác nhận và cho điểm chính xác hơn
Q_c cục bộ nằm trong vùng biên Router	$Q_c \in [0,43; 0,77]$	Sai một bên ngưỡng T_1/T_2 là chọn sai pipeline

Bảng 2.23 Điều kiện kích hoạt LLM trong chế độ Hybrid

Bước 3, hệ thống gọi LLM với prompt đánh giá độ phức tạp, `temperature = 0` và tối đa 120 token đầu ra. LLM trả về điểm 0–10, intent và cờ `needs_multi_doc`. Bước 4, hệ thống tổng hợp kết quả cục bộ và LLM theo nguyên tắc `accuracy-first`:

$$Q_c^{final} = \max(Q_c^{local}, Q_c^{LLM})$$

Việc lấy giá trị lớn hơn thay vì trung bình phản ánh triết lý thiết kế “thiên về leo pipeline khi nghi ngờ”: xử lý câu hỏi đơn giản bằng pipeline cao hơn cần thiết chỉ lãng phí tài nguyên, nhưng xử lý câu hỏi phức tạp bằng pipeline thấp hơn sẽ cho câu trả lời thiếu hoặc sai, đây là hậu quả nghiêm trọng hơn.

Intent cuối được chọn theo quy tắc ưu tiên: giữ `grade_lookup` nếu phát hiện MSSV; ưu tiên intent LLM nếu thuộc nhóm phức tạp; nếu $Q_c^{LLM} > Q_c^{local} + 0,12$ thì lấy intent LLM; còn lại giữ intent cục bộ.

2.3.5 Phân loại intent truy vấn

Song song với tính Q_c , bộ đánh giá phân loại câu hỏi vào sáu nhóm intent. Intent ảnh hưởng đến hai quyết định phía sau: Router có điều chỉnh lựa chọn pipeline hay không, và Specialist Runner nên chọn chiến lược tạo sinh câu trả lời nào.

Intent	Mô tả	Ví dụ đại diện	Ảnh hưởng pipeline
fact_lookup	Tra cứu một thông tin xác định	“Mã trường KMA là gì?”	Giữ nguyên theo Q_c
list_enumerate	Liệt kê nhiều mục	“Những môn KTHP học kỳ 1 gồm những môn nào?”	Nâng lên ít nhất hybrid_rag
compare	So sánh hai đối tượng	“Điểm chuẩn CNTT 2023 và 2024 khác gì?”	Nâng lên agentric_rag nếu đang ở hybrid_rag
multi_hop	Suy luận nhiều bước	“AT200201 có đủ điều kiện tốt nghiệp không?”	Ép về agentric_rag
procedural	Quy trình, các bước thủ tục	“Các bước làm thủ tục bảo lưu?”	Nâng lên ít nhất hybrid_rag
grade_lookup	Tra cứu điểm theo MSSV	“CT060310 điểm học kỳ 2 năm 2024–2025?”	Ép về pipeline grade_lookup chuyên biệt

Bảng 2.24 Sáu nhóm intent và ảnh hưởng đến pipeline

2.3.6 Kết quả đánh giá và tích hợp hệ thống

Kết quả đánh giá được đóng gói trong đối tượng ComplexityAssessment, chuyển cho Router để chọn pipeline và ghi vào metadata phản hồi phục vụ giám sát.

Trường	Kiểu	Ý nghĩa
qc	float	Chỉ số Q_c tổng hợp trong khoảng $[0, 1]$
score_0_10	int	Quy đổi $round(Q_c \times 10)$, dùng trong log
intent	str	Nhãn intent theo Mục 2.3.5
needs_multi_doc	bool	True nếu cần truy xuất từ nhiều tài liệu
reason	str	Chuỗi giải thích breakdown $E/R/L/S$ hoặc lý do LLM

Bảng 2.25 Cấu trúc đối tượng ComplexityAssessment

Nếu quá trình tính Q_c gặp lỗi ở bất kỳ bước nào như spaCy không khởi động, Qdrant timeout hoặc LLM lỗi, hệ thống trả về giá trị mặc định $QC_FALLBACK = 0,55$. Giá trị này nằm trong vùng hybrid_rag ($T_1 = 0,50 \leq 0,55 < T_2 = 0,70$), giúp hệ thống vẫn chọn được pipeline an toàn khi thành phần phụ trợ gặp sự cố.

Câu hỏi	E	R	L	S	Q_c	Intent	Pipeline
“Mã trường KMA là gì?”	0,33	0,00	0,17	0,39	0,22	fact_lookup	native_rag
“Chuẩn đầu ra TOEIC là bao nhiêu?”	0,67	0,00	0,27	0,50	0,34	fact_lookup	native_rag
“CT060310 điểm học kỳ 2 năm 2024–2025?”	1,00	0,00	0,37	0,62	0,54	grade_lookup	grade_lookup
“So sánh điểm chuẩn CNTT và ATTT 2024?”	0,67	0,33	0,30	0,68	0,63	compare	hybrid_rag
“Các bước thủ tục bảo lưu và điều kiện?”	0,33	0,67	0,33	0,72	0,69	procedural	hybrid_rag
“AT200201 đủ điều kiện tốt nghiệp không, điểm rèn luyện và TA đầu vào?”	1,00	0,67	0,60	0,74	0,78	multi_hop	agentic_rag

Bảng 2.26 Ví dụ tính Q_c cho một số câu hỏi học vụ KMA thực tế

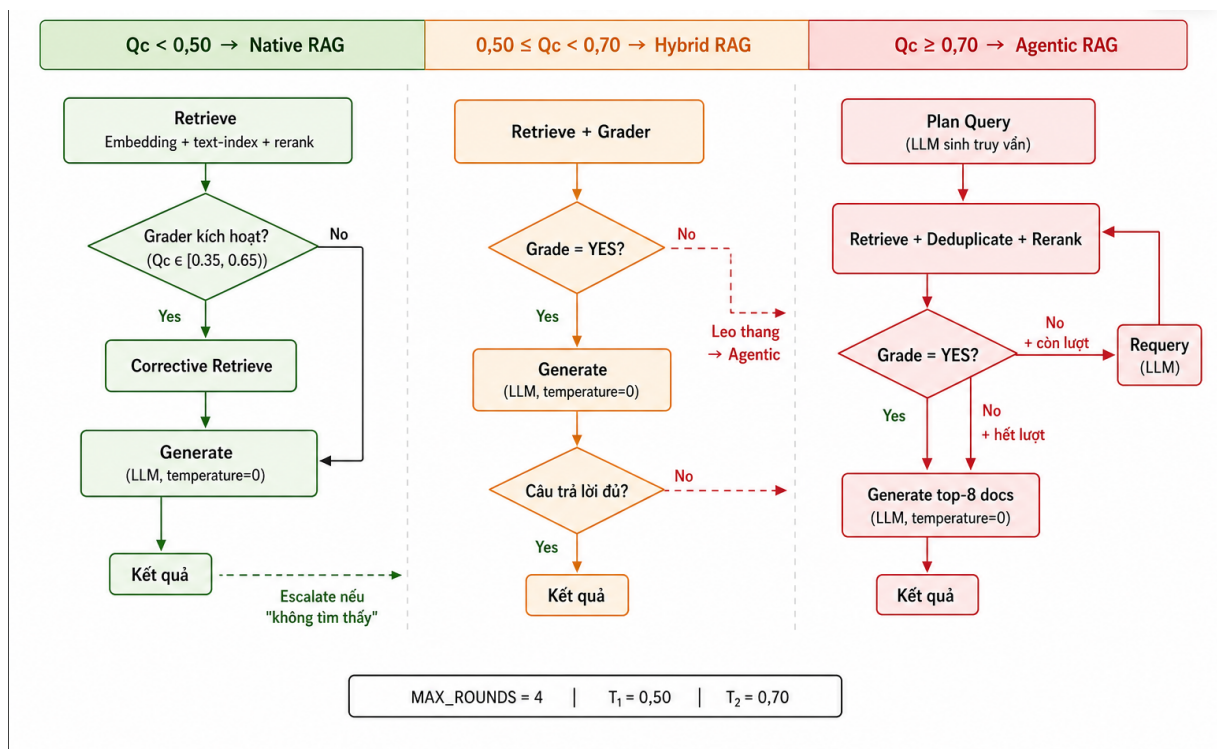
2.4 Phân tầng pipeline truy xuất và sinh câu trả lời

2.4.1 Nguyên lý phân tầng pipeline

Trong hệ thống RAG thông thường, một pipeline duy nhất xử lý mọi loại câu hỏi. Cách tiếp cận này buộc người thiết kế phải chọn giữa hai thái cực: pipeline đơn giản, nhanh nhưng thiếu context cho câu hỏi phức tạp; hoặc pipeline đầy đủ, chính xác hơn nhưng lãng phí tài nguyên cho câu hỏi đơn giản. Hệ thống này giải quyết sự đánh đổi đó bằng cách xây dựng ba pipeline có độ phức tạp leo thang, được chọn động theo chỉ số Q_c đã tính ở bước trước.

Pipeline	Điều kiện Q_c	Đặc điểm	Số lần gọi LLM tối thiểu
Native RAG	$Q_c < 0,50$	Retrieve $\rightarrow$ Generate	1 (Generate)
Hybrid RAG	$0,50 \leq Q_c < 0,70$	Native + Grader + leo thang tự động	2 (Grader + Generate)
Agentic RAG	$Q_c \geq 0,70$	Plan $\rightarrow$ Retrieve $\times N \rightarrow$ Grade $\times N \rightarrow$ Generate	3+ (Plan + Grade/Requery + Generate)

Bảng 2.27 Ba pipeline truy xuất và sinh câu trả lời theo độ phức tạp truy vấn



Hình 2.4 So sánh ba pipeline RAG theo chỉ số Q_c

Bên cạnh phân tầng theo Q_c , Router còn áp dụng hai lớp điều chỉnh bổ sung. Lớp *Supervisor Hint* nâng pipeline tối thiểu theo intent Supervisor: intent grade_result hoặc form_procedure nâng từ native_rag lên ít nhất hybrid_rag; intent multi_domain nâng thêm một bậc (native $\rightarrow$ hybrid, hybrid $\rightarrow$ agentic); nếu độ tin cậy phân loại Supervisor thấp hơn ngưỡng SUPERVISOR_LOW_CONFIDENCE = 0.75, pipeline cũng tự động lên hybrid_rag.

Lớp *Accuracy Policy* khi ACCURACY_MODE=1 ép pipeline theo intent của Q_c : multi_hop luôn về agentic_rag; compare, procedural, list_enumerate lên ít nhất hybrid_rag; câu hỏi đã qua Planner, tức có nhiều ý con, lên agentic_rag bất kể Q_c .

Ngoài ba pipeline chính, hệ thống còn có hai đường nhanh đặc biệt: *Grade Lookup* cho tra cứu điểm theo MSSV và *Schedule/Exam Fast-Path* cho lịch thi, danh sách thi dạng bảng. Các đường nhanh này được kiểm tra trước toàn bộ bước routing; khi thỏa điều kiện, hệ thống bỏ qua hoàn toàn việc tính Q_c và chọn pipeline.

2.4.2 Cơ chế truy xuất nền tảng

Tất cả pipeline đều dùng chung lớp QdrantRetriever cho việc truy xuất tài liệu. Quá trình này gồm bốn giai đoạn kế tiếp, mỗi giai đoạn giải quyết một điểm yếu cụ thể của phương pháp trước.

2.4.2.1. Tìm kiếm vector có lọc tác tử

Câu hỏi được chuyển thành vector embedding bằng model text-embedding-3-small của OpenAI, kích thước 1536 chiều. Vector này được dùng để tìm kiếm K nearest neighbor trên collection kma_docs của Qdrant. Điểm then chốt là mọi truy vấn đều kèm bộ lọc agent_id bắt buộc: Qdrant chỉ tìm kiếm trong phân vùng tài liệu thuộc nghiệp vụ của agent đang xử lý.

Ví dụ, khi agent khao_thi nhận câu hỏi về quy chế thi, Qdrant chỉ tìm trong các văn bản quy chế và chuẩn đầu ra; tài liệu tuyển sinh hay bảng điểm của các agent khác bị loại khỏi không gian tìm kiếm. Thiết kế này giải quyết vấn đề nhiễu cross-domain: nếu không lọc, câu hỏi về điểm chuẩn có thể truy xuất nhầm bảng điểm học phần, cho câu trả lời sai dù điểm cosine cao.

Giá trị K không cố định. Với agent diem_thi và câu hỏi chứa MSSV, K tăng thêm 8 để đảm bảo đủ chunk từ nhiều file bảng điểm. Với các agent dùng cấu trúc Parent-Child, K được nhân với hệ số PARENT_SEARCH_POOL_MULTIPLIER để có đủ pool chunk con trước khi thu gọn về đoạn cha.

2.4.2.2. Tìm kiếm bổ sung bằng text-index

Embedding mã hóa ngữ nghĩa ngôn ngữ học của câu hỏi nhưng kém nhạy với chuỗi số, mã định danh và tên file cụ thể. Khi sinh viên hỏi “điểm chuẩn ngành CNTT 2024 là bao nhiêu?”, embedding của câu này có thể gần với các đoạn văn mô tả tổng quan hơn là bảng số liệu cụ thể như 26,20 hay 26,60. Vấn đề tương tự xảy ra với mã học phần, số tín chỉ, tên file biểu mẫu và MSSV.

Để bù đắp, sau bước vector search, hệ thống tổng hợp danh sách từ khóa đặc thù theo agent và nội dung câu hỏi, sau đó thực hiện tra cứu text-index qua Qdrant scroll API với bộ lọc MatchText. Qdrant text-index được tạo với tokenizer WORD, tối thiểu 2 ký tự, lowercase, cho phép khớp exact word trong nội dung chunk. Hệ thống lấy tối đa 4 chunk mỗi từ khóa và tối đa 8 từ khóa mỗi lượt truy xuất. Các chunk tìm được qua đường text-index nhận điểm cố định 0,88 và được hợp nhất vào pool kết quả ban đầu.

Agent	Từ khóa text-index khi có câu hỏi về...	Tài liệu đích
tuyen_sinh	“điểm chuẩn CNTT 2024” → 02_de_an_tuyen_sinh_2024, 26.20, 26.60	Đề án tuyển sinh 2024
bieu_mau	“nhập học” → Thu_tuc_nhap_hoc_2024, CCCD, học bạ	Hướng dẫn nhập học
khao_thi	“TOEIC” → TOEIC, 450, 350	Quy định chuẩn ngoại ngữ
ma_tran	“Tin học” → 13_ma_tran_de_thi_tin_hoc, 60 phút, 50 câu	Ma trận đề thi Tin học
diem_thi	MSSV “CT060310” → text-index exact match MSSV	Bảng điểm chứa MSSV đó

Bảng 2.28 Ví dụ từ khóa text-index theo từng agent

2.4.2.3. Xếp hạng lại theo đa tín hiệu

Sau khi gộp kết quả từ cả hai nguồn, toàn bộ chunk được tính lại điểm xếp hạng theo công thức:

$$rank_score(d) = score_{\cosine}(d) + \Delta_{\text{keyword}}(d, q, agent_id)$$

Hàm Δ_{keyword} tích lũy các điều chỉnh dương hoặc âm theo tín hiệu khớp đặc thù

nghiệp vụ.

Tín hiệu khớp	Điều kiện	Δ
MSSV khớp chính xác trong text (diem_thi)	Chuỗi “CT060310” có trong body chunk	+0,55
MSSV khớp trong danh sách thi (danh_sach_thi)	MSSV xuất hiện trong bảng dự thi	+0,50
File đúng đợt 2 (lich_thi)	Câu hỏi “đợt 2” và file có _dot2 trong tên	+0,55
File đúng đợt 1 (lich_thi)	Câu hỏi “đợt 1” và file có _dot1 trong tên	+0,22
File đúng đề án tuyển sinh (tuyen_sinh)	Hỏi điểm chuẩn 2024, file 02_de_an_...2024	+0,35
File nhập học (bieu_mau)	Hỏi nhập học, file thu_tuc_nhap_hoc	+0,35
File quy chế 120 tín chỉ (khao_thi)	Hỏi “120 tín chỉ”, file 25_quy_che_dao_tao	+0,30
File sai môn học (ma_tran)	Hỏi Tin học nhưng file là Toán	-0,22
File sai đợt (lich_thi)	Câu hỏi “đợt 1” nhưng file là dot2	-0,30 đến -0,35

Bảng 2.29 Các tín hiệu điều chỉnh điểm reranking

Sau reranking, hệ thống loại bỏ các chunk có $rank_score < best_score - 0,28$. Đây là ngưỡng lọc mềm: khi đã có chunk vượt trội rõ, ví dụ chunk chứa đúng MSSV, các chunk nhiễu bị loại nhanh; khi kết quả đồng đều nhau, ngưỡng này đủ rộng để giữ lại nhiều chunk đa dạng. Riêng với agent `lich_thi` và `danh_sach_thi`, hệ thống giữ thêm toàn bộ chunk cùng file với chunk tốt nhất vì bảng thi thường trải qua nhiều trang PDF; thiếu bất kỳ chunk nào có thể làm mất dữ liệu lịch thi hoặc tên sinh viên.

2.4.2.4. Thu gọn kết quả Parent-Child

Một số agent sử dụng cấu trúc chunk phân cấp hai tầng. Tài liệu được chia thành các đoạn cha (*parent*) đủ rộng để chứa đủ ngữ cảnh cho bước Generate; mỗi đoạn cha lại được chia thành các đoạn con (*child*) nhỏ hơn khi ingest vào Qdrant để tăng độ nhạy khi tìm kiếm. Mỗi chunk con lưu `parent_id` và `parent_text` trỏ về đoạn cha tương ứng.

Khi truy xuất, Qdrant tìm theo chunk con để tăng recall ở bước matching. Sau reranking, hàm `collapse_child_hits_to_parents` thay thế tất cả chunk con cùng parent bằng `parent_text` tương ứng. Kết quả cuối cùng là danh sách đoạn cha không trùng lặp, tối đa `PARENT_RETRIEVE_MAX` đoạn, mỗi đoạn đủ rộng để LLM có ngữ cảnh

đầy đủ mà không bị cắt đứt giữa câu hoặc giữa bảng số liệu.

2.4.3 Pipeline Native RAG ($Q_c < 0,50$)

Native RAG là pipeline tối giản nhất, xử lý câu hỏi tra cứu đơn theo luồng thẳng: truy xuất tài liệu, xây dựng context và gọi LLM một lần để sinh câu trả lời. Đây là pipeline phổ biến nhất trong giờ vận hành thực tế vì đa số câu hỏi học vụ của sinh viên là tra cứu một sự kiện cụ thể như điểm chuẩn, tên file biểu mẫu hoặc yêu cầu TOEIC.

2.4.3.1. Truy xuất có kiểm định chọn lọc

Đối với Native RAG, hệ thống áp dụng chiến lược kích hoạt Document Grader có chọn lọc thay vì kích hoạt toàn bộ hoặc tắt hoàn toàn. Điều kiện kích hoạt là $Q_c \in [0,35; 0,65)$. Câu hỏi có $Q_c < 0,35$ thường là tra cứu sự kiện đơn, xác suất cao retrieve đúng ngay lần đầu nên không cần Grader; câu hỏi có $Q_c \geq 0,65$ đã thuộc hybrid_rag trở lên và có cơ chế riêng. Vùng $[0,35; 0,65)$ là vùng bất định nhất, nơi Grader tạo ra hiệu quả thực sự.

Khi Grader được kích hoạt, quá trình `corrective_retrieve` diễn ra theo vòng lặp: truy xuất $\rightarrow$ LLM kiểm định YES/NO $\rightarrow$ nếu NO, sinh truy vấn mới và truy xuất bổ sung, tối đa `GRADER_MAX_REQUERY` lần. Mỗi lần thêm chunk mới, toàn bộ pool được deduplicate và rerank. Nếu sau tất cả các vòng Grader vẫn trả NO, pipeline leo thang sang Agentic RAG thay vì Generate với context không đủ.

Đối với agent `bieu_mau`, nếu Grader trả NO sau tất cả các vòng retry, hệ thống kích hoạt thêm catalog fallback: tra cứu danh mục biểu mẫu từ `catalog_service` và đưa kết quả vào `extra_context` trước Generate. Đây là cơ chế dự phòng đặc biệt vì tài liệu biểu mẫu vừa có tên file cụ thể, embedding kém, vừa đa dạng về loại đơn.

Trường hợp đặc biệt: với `ACCURACY_MODE=1`, Native RAG luôn chạy Corrective RAG bất kể Q_c , vì môi trường accuracy ưu tiên đúng hơn nhanh.

2.4.3.2. Chuẩn bị context và sinh câu trả lời

Context đưa vào LLM được xây dựng theo cấu trúc đánh số thứ tự chunk:

```
[1] (Nguồn: 02_de_an_tuyen_sinh_2024.pdf tr.12 | table)
Điểm chuẩn ngành CNTT 2024: 26,20...
[2] (Nguồn: 02_de_an_tuyen_sinh_2024.pdf tr.13 | prose)
Mô tả chi tiết ngưỡng xét tuyển...
```

Trước context, hệ thống chèn thêm `context_header`, một đoạn mô tả ngắn gọn tập hợp tài liệu như số lượng file và loại tài liệu, giúp LLM định hướng trước khi đọc nội dung. Nếu agent có chunk loại `table`, `persona_suffix_for_docs` bổ sung hướng dẫn đọc bảng vào system prompt.

Prompt RAG chuẩn như sau:

Dựa vào các đoạn tài liệu dưới đây của Học viện Kỹ thuật Mật mã (KMA), hãy trả lời câu hỏi một cách đầy đủ và chi tiết.

- Liệt kê rõ ràng các thông tin cụ thể có trong tài liệu (số liệu, tên, ngày tháng, điều kiện, v.v.).
- Nếu có nhiều ý, trình bày theo từng mục rõ ràng.
- Nếu không có thông tin trong tài liệu, trả lời:
"Không tìm thấy thông tin trong tài liệu KMA."
- Không bịa thêm thông tin ngoài tài liệu.

Tài liệu:

{context}

Câu hỏi: {question}

Trả lời:

Khi ACCURACY_MODE=1, khối quy tắc bổ sung được chèn ngay trước “Trả lời.”:

QUY TẮC ĐỘ CHÍNH XÁC TỐI ĐA (bắt buộc):

- Chỉ khẳng định số liệu, điều kiện, quy định khi có trong tài liệu; ghi kèm [số thứ tự đoạn] khi trích dẫn.
- Nếu câu hỏi yêu cầu danh sách: liệt kê ĐỦ TẤT CẢ mục có trong tài liệu, không gộp, không bỏ sót.
- Nếu câu hỏi so sánh: nêu rõ số liệu/fact của TỪNG bên, không chỉ mô tả chung.
- Nếu tài liệu mâu thuẫn hoặc không đủ: nêu rõ phần chắc chắn và phần không có trong tài liệu.
- Không suy diễn, không dùng kiến thức ngoài KMA.

Mỗi agent còn có system prompt nhân cách riêng định nghĩa phạm vi chuyên môn và quy tắc trả lời đặc thù. Ví dụ agent `lich_thi` có hướng dẫn giải mã mã khóa gộp như `A19C7D6 = AT19 + CT7 + DT6`; agent `danh_sach_thi` có quy tắc tra đúng hàng MSSV/SBD trong bảng và phân biệt với tra điểm; agent `tuyen_sinh` có quy tắc không trả “không tìm thấy” khi bảng điểm chuẩn có dòng ngành liên quan. Toàn bộ stack message gửi cho LLM theo thứ tự: system prompt (nhân cách + quy tắc) → lịch sử hội thoại → user prompt (context + câu hỏi).

2.4.4 Pipeline Hybrid RAG và cơ chế leo thang ($T_1 \leq Q_c < T_2$)

Hybrid RAG không phải một pipeline kiến trúc riêng biệt mà là chiến lược thực thi hai giai đoạn với cơ chế leo thang tự động. Điều này phản ánh triết lý thiết kế: không cần luôn gọi Agentic RAG tốn kém cho vùng Q_c trung bình, nhưng cũng không thể chấp nhận Native RAG đơn thuần cho các câu hỏi đã có tín hiệu phức tạp vừa đủ.

Pipeline khởi động bằng Native RAG với Grader kích hoạt đầy đủ. Sau đó hệ thống kiểm tra leo thang qua ba tầng.

Tầng kiểm tra	Điều kiện và hành động
Sau Grader, trước Generate	Nếu Document Grader trả NO sau tất cả vòng retry, pipeline leo sang Agentic RAG. Ngoại lệ: với lịch_thi và danh_sach_thi, nếu đã có chunk dạng bảng với <code>retrieval_confidence</code> $\geq 0,35$, pipeline vẫn tiếp tục Generate vì Grader thường báo NO với bảng dữ liệu phức tạp.
Sau Generate	Nếu câu trả lời chứa cụm “không tìm thấy” hoặc <code>retrieval_confidence</code> $< \text{MIN_RETRIEVAL_SCORE}$, pipeline leo sang Agentic RAG để thử lại.
Không có Grader khi FAST_MODE=1	Corrective RAG bị bỏ qua; pipeline chạy Native RAG thẳng và chỉ kiểm tra leo thang sau Generate dựa trên nội dung câu trả lời và confidence.

Bảng 2.30 Ba tầng kiểm tra leo thang trong Hybrid RAG

Kết quả cuối cùng của Hybrid RAG được gán nhãn là `hybrid_rag` nếu không leo thang, hoặc `agentic_rag` nếu đã leo thang. Nhãn này được ghi vào metadata phản hồi để phục vụ giám sát và phân tích tỉ lệ leo thang theo thời gian.

2.4.5 Pipeline Agentic RAG ($Q_c \geq 0,70$ hoặc leo thang)

Agentic RAG xử lý câu hỏi phức tạp theo nguyên lý Corrective RAG kết hợp Iterative Retrieval. Hệ thống không truy xuất một lần mà lặp lại chu trình tìm kiếm, đánh giá và tái truy vấn cho đến khi đủ bằng chứng hoặc đạt giới hạn vòng lặp.

2.4.5.1. Lập kế hoạch truy vấn

Thay vì dùng trực tiếp câu hỏi người dùng làm query, Agentic RAG gọi LLM để sinh một truy vấn tìm kiếm tập trung, dài khoảng 5–10 từ:

Bạn cần trả lời câu hỏi sau về Học viện Kỹ thuật Mật mã (KMA).
 Hãy tạo một truy vấn tìm kiếm ngắn (5-10 từ) để tìm thông tin liên quan.
 Chỉ trả lời bằng truy vấn tìm kiếm, không giải thích.
 Câu hỏi: {question}
 Truy vấn tìm kiếm:

Câu hỏi dài dòng của người dùng thường chứa nhiều từ đệm, đại từ, từ hỏi không có giá trị tìm kiếm. Bước này loại bỏ nhiều và hướng embedding về phía từ khóa chuyên ngành thực sự. Ví dụ, câu hỏi dài về sinh viên AT200201, học kỳ 2 năm 2024–2025, điểm rèn luyện và tiếng Anh đầu vào có thể được rút gọn thành truy vấn “AT200201 điều kiện tốt nghiệp điểm rèn luyện tiếng Anh”. Nếu LLM trả về chuỗi quá ngắn hoặc rỗng, câu hỏi gốc được dùng thay thế như fallback an toàn.

2.4.5.2. Vòng lặp truy xuất, kiểm định và tái truy vấn

Từ truy vấn đã lập kế hoạch, pipeline thực hiện vòng lặp tối đa MAX_ROUNDS lần với bộ tích lũy all_docs. Mỗi vòng gồm năm bước:

- **Retrieve:** gọi `QdrantRetriever.retrieve()` với query hiện tại, nối kết quả mới vào all_docs.
- **Deduplicate:** loại bỏ chunk trùng dựa trên 80 ký tự đầu của text. Cơ chế này đơn giản nhưng hiệu quả vì hai vòng retrieve thường trả về chunk chồng chéo khi query không khác biệt đủ lớn.
- **Rerank:** toàn bộ all_docs được rerank theo rank_score tổng hợp, giúp chunk từ các vòng khác nhau được so sánh trong cùng một thang điểm.
- **Grade:** LLM nhận danh sách chunk, tối đa GRADER_MAX_CHUNKS chunk, ưu tiên chunk dạng bảng cho các agent lịch_thi, danh_sach_thi, diem_thi, ma_tran, rồi đánh giá YES/NO.
- **Requery:** nếu Grade = NO và còn lượt thử, LLM sinh truy vấn tìm kiếm mới, cụ thể hơn truy vấn hiện tại.

Prompt Grader được dùng như sau:

```
Bạn là bộ kiểm định relevance cho RAG Học viện KMA.  
Câu hỏi sinh viên:  
{question}  
Các đoạn tài liệu retrieve:  
{chunks}  
Nhiệm vụ: Có ít nhất MỘT đoạn chứa thông tin TRỰC TIẾP để trả lời  
câu hỏi không?  
Trả lời ĐÚNG format (2 dòng):  
Dòng 1: YES hoặc NO  
Dòng 2: Giải thích ngắn tiếng Việt (tối đa 25 từ)
```

Quy tắc parse kết quả chỉ kiểm tra dòng đầu tiên. Chuỗi bắt đầu bằng “YES” hoặc “Y” được hiểu là `sufficient = True`; chuỗi bắt đầu bằng “NO” hoặc “N” được hiểu là `sufficient = False`; mọi trường hợp không xác định cũng trả về `False`, thiên về thận trọng. Lý do ở dòng thứ hai được lưu vào `GradingTrace` để phục vụ debug.

Khi cần requery, prompt sinh truy vấn mới là:

Câu hỏi: {question}

Các đoạn tài liệu hiện tại KHÔNG đủ để trả lời

(similarity cao nhưng sai/ngắt đoạn).

{chunks}

Tạo MỘT truy vấn tìm kiếm mới (8-14 từ tiếng Việt, từ khóa cụ thể từ quy chế KMA).

Chỉ trả truy vấn, không giải thích:

Nếu truy vấn mới giống truy vấn cũ, vòng lặp thoát sớm để tránh lãng phí các lần retrieve không làm thay đổi pool. Cơ chế này quan trọng với câu hỏi về thông tin chưa có trong corpus, vì LLM có xu hướng sinh các truy vấn tương tự nhau.

Điều kiện	Hành động tiếp theo
Grade = YES ở bất kỳ vòng nào	Thoát ngay, tiến sang Generate
Đạt MAX_ROUNDS lần lặp	Thoát, dùng pool hiện có cho Generate
Truy vấn mới = truy vấn cũ	Thoát sớm, dùng pool hiện có
Pool rỗng sau retrieve	Thoát, Generate với context rỗng để trả không tìm thấy

Bảng 2.31 Điều kiện thoát vòng lặp Agentic RAG

2.4.5.3. Tổng hợp context và sinh câu trả lời

Sau vòng lặp, all_docs được sắp xếp theo rank_score giảm dần và giữ top-8 chunk chất lượng nhất. Con số này cân bằng giữa độ đầy đủ context và giới hạn context window của LLM. Top-8 chunk được rerank thêm một lần cuối trước khi xây dựng context.

Khác với Native RAG, trong ACCURACY_MODE=1, Agentic RAG đưa toàn bộ nội dung mỗi chunk vào context thay vì chỉ 400 ký tự đầu. Với câu hỏi phức tạp, thông tin quyết định thường nằm ở giữa hoặc cuối đoạn; cắt ngắn chunk ở đây có thể làm mất đáp án. Phần Generate dùng cùng cấu trúc prompt và stack message như Native RAG, với sự khác biệt là context phong phú hơn nhờ nhiều vòng tích lũy.

2.4.6 Pipeline Grade Lookup cho tra cứu điểm

Grade Lookup là pipeline chuyên biệt cho agent diem_thi, được kích hoạt ưu tiên bất kể Q_c khi câu hỏi chứa MSSV dạng ATxxxxxx/CTxxxxxx và kèm một trong các trigger: điểm, bảng điểm, kết quả, học kỳ, đạt/không đạt, tiếng Anh đầu vào, hoặc tên sinh viên được nhận diện bởi regex. Pipeline này không dùng embedding làm phương

pháp tìm kiếm chính vì embedding mã hóa ngữ nghĩa của câu hỏi, không phải chuỗi MSSV ngẫu nhiên. Hai câu hỏi về hai sinh viên khác nhau có vector embedding gần như giống hệt nhau, không phân biệt được ai cần bảng điểm nào.

2.4.6.1. Tìm kiếm MSSV bằng text-index và scroll

Qdrant được truy vấn theo hai cơ chế lần lượt. Cơ chế thứ nhất là text-index MatchText, tìm tất cả chunk có chứa chuỗi MSSV cụ thể qua chỉ mục text đã tạo. Cơ chế thứ hai là scroll fallback, duyệt toàn bộ collection với filter agent_id = diem_thi và kiểm tra text thủ công. Text-index nhanh và chính xác khi chỉ mục sẵn sàng; scroll fallback chậm hơn nhưng đảm bảo không bỏ sót khi text-index chưa build xong. Chunk tìm được qua MSSV nhận điểm cố định 0,95, cao hơn đa số kết quả embedding, đảm bảo chúng luôn được ưu tiên tuyệt đối trong rerank.

Nếu câu hỏi có cả MSSV lẫn tên sinh viên, ví dụ “điểm của sinh viên Nguyễn Văn An có MSSV CT060310”, hệ thống tích thêm điều kiện lọc theo tên: chỉ giữ chunk mà text chứa từng phần tên sinh viên, không phân biệt hoa thường và đã chuẩn hóa Unicode. Nếu lọc tên quá chặt làm rộng kết quả, hệ thống log cảnh báo và trả về toàn bộ chunk MSSV, không để sinh viên mất kết quả chỉ vì tên gõ khác dấu.

2.4.6.2. Lọc theo học kỳ, đợt và năm học

Sau khi có tập chunk chứa MSSV, hệ thống phân tích câu hỏi để trích xuất bộ ba: học kỳ (hk1/hk2), đợt (dot1/dot2) và năm học, ví dụ 20242025. Hệ thống chỉ giữ chunk từ file có tên khớp cả ba thông tin này. Nếu không có file nào khớp, tức tài liệu học kỳ đó chưa được nạp, hệ thống log cảnh báo và trả về toàn bộ chunk MSSV với thông báo rõ ràng.

Tiếp theo, các chunk được nhóm theo file nguồn. Mỗi file được tính điểm tổng hợp:

$$score_file = \Delta_{keyword}^{\max} + 0,05 \times N_{chunk}$$

trong đó $\Delta_{keyword}^{\max}$ là điểm keyword adjustment cao nhất trong các chunk của file đó, và N_{chunk} là số chunk có MSSV trong file. File có điểm cao nhất được ưu tiên. Nếu chỉ có một file khớp đúng kỳ/đợt, toàn bộ chunk của file đó được giữ nguyên theo thứ tự trang, không rerank thêm để tránh mất dòng bảng điểm do sắp xếp lại.

2.4.6.3. Chuẩn bị context chuyên biệt và sinh câu trả lời

Trước khi xây dựng prompt, Grade Lookup thực hiện hai bước tiền xử lý context đặc thù. Bước thứ nhất là *Subject-Grade Index*: hệ thống quét tất cả chunk tìm dòng “Môn thi: Tên học phần”, xây dựng ánh xạ (file, trang) → tên môn. Sau đó, với mỗi chunk chứa MSSV, hệ thống trích các dòng bảng điểm của sinh viên đó và gắn tên môn tương ứng.

Kết quả là một chỉ mục môn-điểm có cấu trúc được chèn vào đầu context:

CHỈ MỤC MÔN THI + DÒNG ĐIỂM:

- Cơ sở an toàn và bảo mật thông tin (nguồn: hk2_...pdf tr.5)
 - CT060310 | ... | 8.5 | A
- Lập trình hướng đối tượng (nguồn: hk2_...pdf tr.8)
 - CT060310 | ... | 7.0 | B+

Bước thứ hai là *MSSV Row Hints*: tất cả dòng trong các chunk chứa MSSV được trích xuất kèm tên file, số trang và tên môn, tạo thành một danh sách gợi ý hàng bảng trực tiếp để LLM không bỏ sót môn nào khi liệt kê.

Prompt Generate của Grade Lookup có cấu trúc riêng với 8 quy tắc bắt buộc: đọc đúng hàng-cột, liệt kê tất cả môn học với tên đầy đủ từ dòng “Môn thi:”, chỉ lấy đúng học kỳ/đợt được hỏi và không bịa điểm. Đặc biệt, system prompt của Grade Lookup không dùng PERSONA_ACCURACY_SUFFIX chung; kinh nghiệm thực tế cho thấy quy tắc accuracy chung đôi khi khiến LLM cẩn thận thái quá và trả “không tìm thấy” dù MSSV đã có trong context.

Cơ chế retry tự động được kích hoạt nếu LLM trả lời “không tìm thấy” nhưng hệ thống xác nhận MSSV có trong ít nhất một chunk. Khi đó pipeline gọi LLM lần hai với prompt bổ sung cảnh báo bắt buộc: “Tài liệu trên CÓ chứa MSSV {mssv}. Bạn **PHẢI** liệt kê từng môn và điểm”. Cơ chế này xử lý trường hợp LLM nhầm lẫn do layout bảng phức tạp mà vẫn đảm bảo không trả kết quả trống khi dữ liệu thực sự tồn tại.

2.4.7 Đường nhanh cho lịch thi và danh sách thi

Ngoài bốn pipeline chính, hệ thống có hai đường nhanh đặc biệt cho agent *lich_thi* và *danh_sach_thi*. Cả hai loại tài liệu này có cấu trúc bảng rất rõ ràng, nơi thông tin cần thiết là số liệu cụ thể trong từng ô bảng, và Document Grader thường xuyên báo NO dù chunk đã đủ. Gọi LLM Grader và Agentic RAG lặp lại cho loại câu hỏi này gây lãng phí 3–8 giây latency không cần thiết.

Schedule Extract được kích hoạt khi câu hỏi chứa marker lịch thi như “KTHP”, “lịch thi”, “học kỳ”, “đợt thi”, “các môn”. Hệ thống truy xuất chunk bảng lịch KTHP, parse trực tiếp từ cấu trúc bảng markdown để liệt kê môn thi, hình thức thi, khóa đào tạo, thời gian, rồi trả về câu trả lời được định dạng sẵn mà không cần gọi LLM. Toàn bộ quá trình chỉ tốn một lần truy vấn Qdrant, thường dưới 100 ms.

Exam List Extract được kích hoạt cho agent *danh_sach_thi* khi câu hỏi chứa MSSV hoặc SBD kèm ngày thi. Hệ thống extract trực tiếp hàng sinh viên từ bảng danh sách dự thi và trả về thông tin ca thi, phòng thi, môn thi, ghi chú nếu có mà không qua LLM.

Hai đường nhanh này được kiểm tra trước tất cả các bước còn lại: trước Q_c , trước Router và trước pipeline routing. Nếu điều kiện thỏa mãn và trích xuất thành công, kết quả được trả về ngay; nếu không trích được do tài liệu chưa nạp hoặc câu hỏi quá mơ hồ, hệ thống không fallback sang LLM mà trả thông báo hướng dẫn rõ ràng, tránh trường hợp LLM sinh ảo lịch thi hoặc danh sách sinh viên khi không có dữ liệu thực.

2.5 Phân rã nghiệp vụ thành các tác tử chuyên biệt

2.5.1 Nguyên lý phân chia nghiệp vụ

Hệ thống triển khai bảy tác tử chuyên biệt, mỗi tác tử phụ trách một miền thông tin học vụ riêng biệt. Nguyên lý cốt lõi là cô lập corpus tài liệu theo nghiệp vụ: toàn bộ tài liệu được lưu trong một collection Qdrant duy nhất kma_docs, nhưng mỗi chunk mang trường metadata agent_id xác định nghiệp vụ sở hữu. Khi một Specialist Runner truy xuất tài liệu, bộ lọc agent_id bắt buộc giới hạn không gian tìm kiếm chỉ trong phân vùng của tác tử đó, loại bỏ hoàn toàn nhiễu cross-domain.

Bảng 2.32 Tổng quan bảy tác tử chuyên biệt

Agent ID	Tên	Nghiệp vụ	Chiến lược chunking	Pipeline đặc biệt
tuyen_sinh	Tuyển sinh & CTĐT	Điểm chuẩn, đề án tuyển sinh, ngành học, chương trình đào tạo	Parent-Child	–
khao_thi	Khảo thí & Quy chế	Quy chế đào tạo, chuẩn ngoại ngữ, quy định thi, thi tốt nghiệp	Parent-Child	–
ma_tran	Ma trận đề thi	Cấu trúc đề thi, số câu, thời gian, tỉ lệ nhận thức từng môn	Parent-Child + Table Metadata	–
diem_thi	Kết quả học tập	Bảng điểm theo MSSV, kết quả học kỳ, thi Anh văn đầu vào	Table Metadata	Grade Lookup

Bảng 2.32 Tổng quan bảy tác tử chuyên biệt (tiếp)

Agent ID	Tên	Nghiệp vụ	Chiến lược chunking	Pipeline đặc biệt
bieu_mau	Biểu mẫu & Thủ tục	Đơn từ, biểu mẫu hành chính, thủ tục nhập học, phí	Flat	Catalog Service
danh_sach_thi	Danh sách thi	Danh sách dự thi theo ngày/ca/phòng, tra MSSV trong bảng thi	Table Metadata	Exam List Extract
lich_thi	Lịch thi	Lịch KTHP theo học kỳ/đợt/năm, lịch thi lại	Table Metadata	Schedule Extract

2.5.2 Tổ chức và phân đoạn tài liệu

Tài liệu học vụ của Học viện KMA được phân loại thành bảy nhóm nghiệp vụ tương ứng với bảy tác tử. Khi nạp vào Qdrant, mỗi chunk được gán agent_id và một tập metadata phong phú: tên file nguồn, số trang, loại tài liệu (prose/table/matrix), tiêu đề phần và các trường đặc thù tùy loại. Việc lựa chọn chiến lược phân đoạn tài liệu (chunking) phụ thuộc vào đặc điểm cấu trúc của từng nhóm tài liệu.

2.5.2.1. Flat Chunking

Áp dụng cho nhóm tài liệu có cấu trúc đơn giản như bảng điểm và biểu mẫu hành chính. Tài liệu được chia theo cửa sổ từ, 500 từ mỗi chunk với overlap 50 từ, không phân tầng phức tạp. Chiến lược này phù hợp vì bảng điểm có cấu trúc lặp lại đều đặn theo hàng, mỗi hàng là một sinh viên, không cần ngữ cảnh cha; biểu mẫu hành chính thường ngắn, đủ nằm trọn trong một đến hai chunk.

2.5.2.2. Parent-Child Chunking

Áp dụng cho tài liệu quy chế, văn bản pháp lý và đề án tuyển sinh, tức loại tài liệu có cấu trúc phân cấp phức tạp với nhiều điều, khoản, mục lồng nhau. Mỗi tài liệu được phân đoạn theo hai tầng:

- **Đoạn cha:** khoảng 1500 ký tự, ưu tiên ngắt tại ranh giới cấu trúc văn bản pháp lý tiếng Việt như Điều [số], Mục [số], Chương [số], hoặc đoạn trống đôi. Đây là đơn vị đưa vào bước Generate, đủ rộng để LLM nhận đủ ngữ cảnh liền mạch.
- **Đoạn con:** khoảng 200 ký tự, overlap 40 ký tự, được tạo ra từ nội dung đoạn cha, gán parent_id và parent_text dưới dạng metadata. Đây là đơn vị được embedding và lưu vào Qdrant, kích thước nhỏ giúp vector embedding nhạy hơn với từ khóa cụ thể trong từng điều khoản.

Khi truy xuất, Qdrant tìm kiếm qua các đoạn con, chính xác và tập trung; nhưng sau reranking, toàn bộ đoạn con cùng parent được thay thế bằng parent_text tương ứng, tối đa sáu đoạn cha cho một lần Generate. Cơ chế này kết hợp ưu điểm của cả hai: độ chính xác khi matching và độ đầy đủ ngữ cảnh khi sinh câu trả lời.

2.5.2.3. Table Metadata Ingest

Áp dụng bổ sung cho các tác tử xử lý tài liệu dạng bảng cấu trúc: ma trận đề thi, bảng điểm, danh sách thi và lịch thi. Khi nạp, mỗi chunk bảng được gán document_type = “table” kèm metadata: table_headers (tiêu đề cột), row_count (số hàng), table_index (thứ tự bảng trong file), section (tiêu đề phần chứa bảng).

Metadata này phục vụ hai mục đích: Document Grader ưu tiên đọc chunk bảng trước khi đánh giá YES/NO; hệ thống nhận diện cần bổ sung hướng dẫn đọc bảng vào system prompt khi chunk đầu ra chứa dữ liệu dạng bảng.

2.5.2.4. Tránh trùng lặp khi cập nhật tài liệu

Mỗi chunk được gán ID xác định theo UUID5:

$$point_id = \text{UUID5}(\text{NAMESPACE_URL}, \text{"source::page\{p\}::chunk\{i\}"})$$

Tính xác định của UUID5 đảm bảo khi cùng một tài liệu được nạp lại sau cập nhật, điểm mới ghi đè điểm cũ thay vì tạo bản trùng. Trước mỗi lần nạp, toàn bộ điểm có source khớp được xóa trước để không còn chunk lỗi thời trong collection.

2.5.3 Đặc điểm kỹ thuật của từng tác tử

2.5.3.1. Tác tử Tuyển sinh & Chương trình đào tạo (tuyen_sinh)

Tác tử này phụ trách toàn bộ thông tin tuyển sinh: điểm chuẩn, chỉ tiêu, ngưỡng trúng tuyển, phương thức xét tuyển và chương trình đào tạo các ngành CNTT, ATTT, DTVT.

Câu hỏi đặc trưng của mảng này thường mang tính so sánh theo năm, ví dụ “điểm chuẩn CNTT 2023 và 2024 khác nhau thế nào?”, hoặc giữa ngành, ví dụ “ATTT và CNTT ngành nào cao điểm hơn?”. Để xử lý chính xác, text-index bổ sung các con số điểm chuẩn cụ thể và tên file đề án tuyển sinh vào pool truy xuất, giải quyết điểm yếu của embedding khi gặp số liệu. Chiến lược Parent-Child đặc biệt hiệu quả ở đây vì đề án tuyển sinh là văn bản dài nhiều chương: đoạn con nhảy với từ khóa từng ngành, đoạn cha đủ rộng để LLM thấy toàn bộ bảng điểm chuẩn nhiều ngành cùng lúc.

Quy tắc trả lời đặc thù: không trả “không tìm thấy” khi bảng điểm chuẩn đã có dòng ngành được hỏi, tránh LLM cẩn thận thái quá và bỏ sót dòng bảng do đọc không

đủ.

2.5.3.2. Tác tử Khảo thí & Quy chế (khao_thi)

Tác tử này phụ trách các văn bản quy chế đào tạo như số tín chỉ tối thiểu, điều kiện tốt nghiệp, quy trình thi và chuẩn đầu ra ngoại ngữ như yêu cầu TOEIC/VSTEP theo từng đối tượng sinh viên.

Corpus của tác tử này là văn bản pháp lý có mật độ thông tin cao và cấu trúc phân cấp nhiều tầng; đặc điểm này trực tiếp dẫn đến việc chọn Parent-Child thay vì Flat. Câu hỏi thường gặp nhất là về chuẩn TOEIC. Text-index bổ sung các giá trị điểm cụ thể như 450, 350, 300 và từ khóa chứng chỉ VSTEP để đảm bảo điều khoản liên quan luôn xuất hiện trong context.

Quy tắc trả lời đặc thù: liệt kê đầy đủ các mức yêu cầu theo từng loại TA1/TA2/TA3; chú ý phân biệt yêu cầu 120 tín chỉ chuẩn cử nhân với 150 tín chỉ của một số chương trình đặc biệt để tránh nhầm lẫn khi sinh viên hỏi điều kiện tốt nghiệp.

2.5.3.3. Tác tử Ma trận đề thi (ma_tran)

Tác tử này phụ trách cấu trúc đề thi các môn học: số câu hỏi, thời gian làm bài, phân bổ các mức độ nhận thức như nhận biết, thông hiểu và vận dụng. Corpus chứa các tài liệu ma trận đề thi dạng bảng, được ingest với cả hai chiến lược Parent-Child và Table Metadata để xử lý được cả câu hỏi mô tả cấu trúc lẫn câu hỏi tra số liệu cụ thể.

Thách thức chính là phân biệt giữa các file ma trận cùng thư mục nhưng khác môn học. Hệ thống giải quyết bằng keyword adjustment có hướng: khi câu hỏi đề cập tên môn cụ thể như Tin học, CSDL, Toán cao cấp, file ma trận đúng môn được tăng điểm đáng kể còn file môn khác bị giảm điểm, đảm bảo LLM nhận đúng bảng cấu trúc cần tra.

Quy tắc trả lời đặc thù: khi tài liệu có dòng “Tổng số câu hỏi” và “Thời gian làm bài”, phải trích đúng con số, không ước tính hay làm tròn.

2.5.3.4. Tác tử Kết quả học tập (diem_thi)

Tác tử này là kỹ thuật phức tạp nhất trong bảy tác tử do đặc thù dữ liệu: thông tin điểm trải qua nhiều tài liệu PDF phân chia theo học kỳ và đợt thi, mỗi tài liệu chứa bảng hàng trăm đến hàng nghìn dòng sinh viên với nhiều cột điểm thành phần.

Khi câu hỏi chứa MSSV, pipeline chuyển sang Grade Lookup chuyên biệt đã trình bày tại Mục 2.4.6. Khi không có MSSV, ví dụ “danh sách sinh viên đạt TA đầu vào học kỳ 2”, hệ thống sử dụng RAG thông thường với text-index tìm từ khóa liên quan.

Giới hạn tối đa 100 chunk và 14000 ký tự context phản ánh thực tế sinh viên học nhiều môn qua nhiều học kỳ; đây là giới hạn cao nhất trong bảy tác tử, thể hiện mức độ ưu tiên chính xác tuyệt đối cho mảng tra điểm.

Quy tắc trả lời đặc thù: khi tài liệu chứa MSSV được hỏi kèm điểm, phải liệt kê đủ từng môn theo đúng thứ tự bảng, không gộp, không bỏ sót; chỉ trả “không tìm thấy” khi đã đọc hết tài liệu và xác nhận không có MSSV.

2.5.3.5. Tác tử Biểu mẫu & Thủ tục (biểu_mau)

Tác tử này phụ trách tra cứu đơn từ và thủ tục hành chính: đơn xin nghỉ học, đơn phúc khảo, biểu mẫu BHYT, hướng dẫn nhập học và danh mục lệ phí.

Corpus của tác tử này có đặc điểm khác hẳn các tác tử còn lại: sinh viên thường cần biết tên file cụ thể hoặc danh sách giấy tờ, đây là loại thông tin mà embedding kém nhạy. Để giải quyết, hệ thống triển khai Catalog Service bổ sung: một danh mục có cấu trúc của tất cả biểu mẫu hiện có với tên file, mô tả và đường dẫn tải về. Catalog được tích hợp vào hai điểm: context mặc định của mỗi lượt trả lời luôn chứa kết quả tìm kiếm catalog; khi Document Grader trả NO, catalog đóng vai trò nguồn dự phòng.

Tác tử này còn hỗ trợ chức năng điền biểu mẫu tự động: khi sinh viên chọn được biểu mẫu và yêu cầu hỗ trợ điền, tác tử hỏi từng trường thông tin và tạo file Word đã điền sẵn dựa trên template gốc.

Quy tắc trả lời đặc thù: khi hỏi về biểu mẫu cụ thể, ghi rõ tên file gốc đầy đủ; khi hỏi về nhập học, liệt kê đủ danh mục giấy tờ và lệ phí theo hướng dẫn chính thức.

2.5.3.6. Tác tử Danh sách thi (danh_sach_thi)

Tác tử này phụ trách tra cứu danh sách dự thi: ca thi, phòng thi, số báo danh và ghi chú đặc biệt như cấm thi, nợ học phần. Tài liệu là các bảng danh sách dự thi được nạp với Table Metadata đầy đủ; header bảng như Môn thi, Hình thức thi và từng hàng dữ liệu như STT, SBD, MSSV, Ngày thi, Ca thi, Phòng, Ghi chú được lưu kèm.

Tác tử có hai đường xử lý ưu tiên. Exam List Extract được kích hoạt khi câu hỏi chứa MSSV hoặc số báo danh kèm ngày thi; hệ thống parse trực tiếp hàng dữ liệu từ bảng Qdrant và trả kết quả mà không cần gọi LLM, tiết kiệm đáng kể thời gian phản hồi. Khi không thể trích xuất được, tác tử trả thông báo hướng dẫn cụ thể thay vì để LLM sinh câu trả lời sai từ thông tin không đủ.

Quy tắc trả lời đặc thù: phân biệt rõ ràng giữa tra danh sách thi như ca/phòng/SBD và tra điểm thi như điểm số; sinh viên có ghi chú “Cấm thi/Nợ HP” thường không có SBD, ca và phòng thi.

2.5.3.7. Tác tử Lịch thi (lich_thi)

Tác tử này phụ trách lịch thi kết thúc học phần (KTHP): học kỳ, đợt thi, tên môn, hình thức thi, ngày giờ, địa điểm và khóa đào tạo áp dụng. Tài liệu lịch thi được phân biệt theo quy ước đặt tên phản ánh học kỳ, năm học và đợt thi; tài liệu lịch thi lại được

phân biệt riêng biệt với lịch thi chính thức.

Keyword adjustment của tác tử này là tinh tế nhất trong bảy tác tử do dữ liệu có nhiều chiều: học kỳ $\times$ đợt $\times$ năm $\times$ môn $\times$ khóa học. Hệ thống áp dụng boost/penalty theo từng chiều độc lập: tài liệu đúng đợt thi được tăng điểm mạnh, tài liệu sai đợt bị giảm điểm, đảm bảo câu hỏi về “đợt 2 học kỳ 1” không trả lịch của “đợt 1” hay “học kỳ 2”.

Schedule Extract được kích hoạt khi câu hỏi yêu cầu liệt kê môn thi; hệ thống parse trực tiếp từ bảng lịch KTHP và trả danh sách môn định dạng sẵn mà không gọi LLM.

Quy tắc trả lời đặc thù: bảng lịch thi KMA sử dụng mã khóa gộp, ví dụ A19C7D6 = AT19 + CT7 + DT6, trong đó A tương ứng với AT, C tương ứng với CT, D tương ứng với DT và số phía sau là khóa tương ứng; ô khóa gộp nhiều dòng, dòng sau để trống vẫn thuộc khóa ở dòng trên.

2.5.4 *Luồng xử lý của Specialist Runner*

Mỗi Specialist Runner khi nhận câu hỏi thực hiện tuần tự các bước:

1. **Kiểm tra đường nhanh:** ưu tiên Exam List Extract với agent danh_sach_thi và Schedule Extract với agent lich_thi trước mọi bước còn lại. Nếu thỏa mãn và trích xuất thành công, trả về ngay mà không cần tính Q_c hay chọn pipeline.
2. **Kiểm tra Grade Lookup:** nếu là agent diem_thi và câu hỏi chứa MSSV kèm trigger điểm, chuyển sang pipeline tra điểm chuyên biệt.
3. **Tính Q_c và chọn pipeline:** tính chỉ số Q_c , kết hợp Supervisor intent và Accuracy Policy để chọn pipeline phù hợp.
4. **Thực thi pipeline:** chạy pipeline đã chọn, với khả năng leo thang lên Agentic RAG nếu kết quả không đủ chất lượng.
5. **Đóng gói kết quả:** trả về câu trả lời, danh sách tài liệu nguồn, thông số thời gian và nhãn pipeline thực tế đã chạy.

Kết quả được thu thập tại lớp điều phối: nếu chỉ một tác tử trả lời thì truyền thẳng cho người dùng; nếu nhiều tác tử cùng trả lời thì chuyển sang Aggregator để tổng hợp thành một câu trả lời mạch lạc.

2.6 Quản lý ngữ cảnh hội thoại đa lượt

2.6.1 *Bài toán bộ nhớ trong hội thoại đa lượt*

Một hệ thống chatbot phục vụ học vụ cần duy trì ngữ cảnh liên tục qua nhiều lượt hội thoại: sinh viên có thể hỏi “MSSV của tôi là CT060310, điểm học kỳ 2 như thế nào?” rồi tiếp tục “còn học kỳ 1 thì sao?” mà không cần lặp lại MSSV. Đây là thách thức đặc trưng của hệ thống đa lượt: các tác tử cần biết ngữ cảnh từ lượt trước để xử lý câu hỏi

hiện tại chính xác.

Đồng thời, hệ thống phục vụ nhiều sinh viên đồng thời, mỗi sinh viên có phiên độc lập và lịch sử hội thoại riêng. Yêu cầu đặt ra là tốc độ truy cập nhanh cho phiên đang hoạt động, lưu trữ bền vững cho lịch sử dài hạn, và kiểm soát chi phí bộ nhớ khi số lượng phiên tăng lên.

2.6.2 Kiến trúc lưu trữ nóng-lạnh

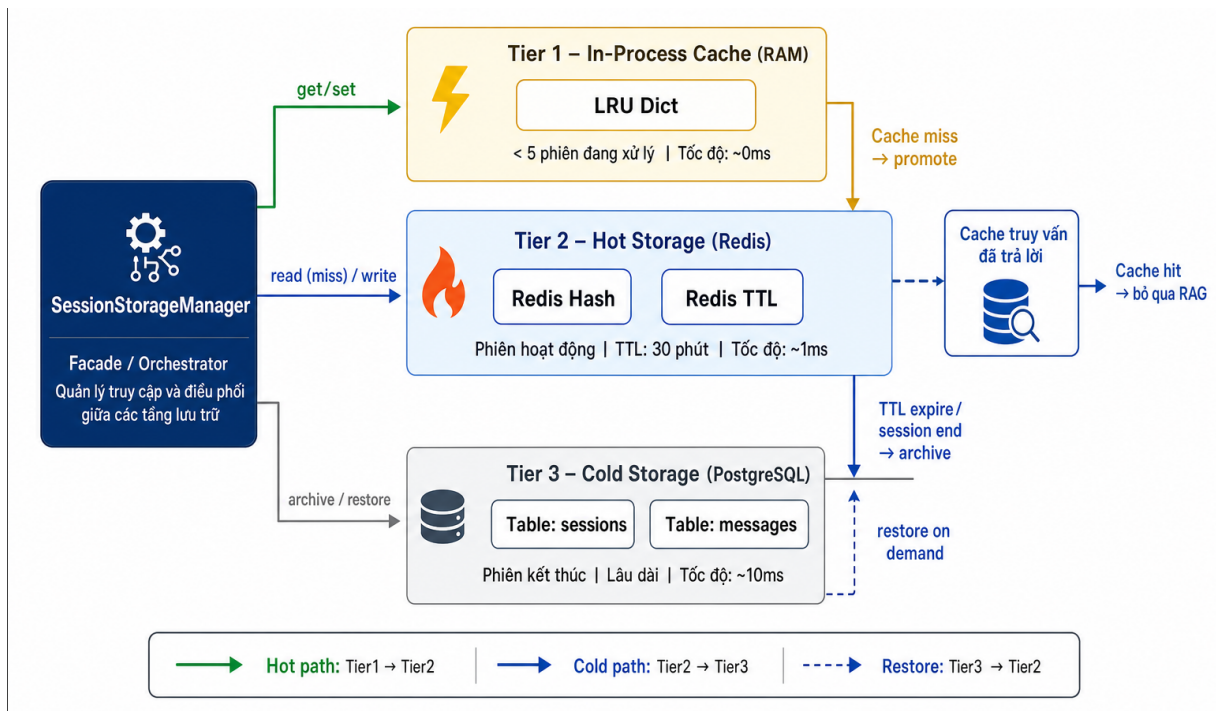
Hệ thống áp dụng kiến trúc Hot-Cold Storage với hai tầng lưu trữ phân biệt rõ ràng theo tần suất truy cập.

Tầng nóng (Hot Storage) – Redis: phiên đang hoạt động được lưu trong Redis với TTL là 2 giờ. Redis cung cấp truy cập trong bộ nhớ với độ trễ dưới 1 ms, phù hợp cho mỗi lượt hội thoại cần đọc/ghi trạng thái phiên. Dữ liệu được serialize thành JSON và lưu theo khóa `kma:session:{session_id}`. Chỉ 6 lượt hội thoại gần nhất (`hot_messages`) được lưu vào Redis; giới hạn này cân bằng giữa ngữ cảnh đủ cho các tác tử và dung lượng Redis.

Tầng lạnh (Cold Storage) – PostgreSQL: khi một phiên không hoạt động quá một giờ hoặc RAM đạt giới hạn 800 phiên đồng thời, trạng thái phiên được chuyển xuống PostgreSQL để lưu trữ lâu dài. Dữ liệu lịch sử đầy đủ, bao gồm tóm tắt phiên, thông tin hồ sơ sinh viên và số lượt hội thoại, được bảo toàn ở đây.

Tầng dự phòng – RAM: khi cả Redis và PostgreSQL không khả dụng, hệ thống fallback về lưu trữ RAM nội bộ với cùng interface. Cơ chế này đảm bảo hệ thống vận hành liên tục ngay cả khi các thành phần lưu trữ ngoài gặp sự cố.

Luồng truy cập phiên theo thứ tự ưu tiên là RAM in-process → Redis → PostgreSQL. Khi phiên được tìm thấy ở tầng dưới, nó được nạp lên tầng trên để phục vụ các lượt tiếp theo nhanh hơn.



Hình 2.5 Kiến trúc quản lý phiên Hot–Cold

2.6.3 Trạng thái phiên hội thoại

Mỗi phiên hội thoại được biểu diễn bởi đối tượng SessionState với các trường chính như Bảng 2.33.

Trường	Kiểu	Mô tả
session_id	str	Định danh phiên duy nhất, sinh theo UUID4.
summary	str	Tóm tắt phiên do LLM sinh, tối đa 1200 ký tự.
turn_count	int	Số lượt hội thoại đã thực hiện trong phiên.
hot_messages	list[dict]	Tối đa 6 cặp hỏi-đáp gần nhất theo dạng OpenAI message.
last_agents	list[str]	Danh sách tác tử đã xử lý ở lượt trước.
student_profile	StudentProfile	Hồ sơ sinh viên tích lũy qua phiên, ví dụ MSSV, ngành và các thông tin đã khai báo.
query_cache	dict	Cache kết quả câu hỏi đã xử lý, TTL 600 giây.
last_activity	float	Timestamp Unix của lượt hoạt động gần nhất.
last_form_filename	str	Tên biểu mẫu gần nhất, phục vụ chức năng điền biểu mẫu.

Bảng 2.33 Cấu trúc đối tượng SessionState

2.6.4 Tóm tắt phiên tự động

Hội thoại dài làm tăng kích thước context đưa vào các tác tử, ảnh hưởng trực tiếp đến chi phí và độ trễ. Thay vì truyền toàn bộ lịch sử, hệ thống áp dụng cơ chế tóm tắt phiên tự động: sau mỗi SESSION_SUMMARY_EVERY = 3 lượt hội thoại, LLM sinh một đoạn tóm tắt mới tích hợp trạng thái cũ và nội dung mới nhất.

```
Tóm tắt ngắn phiên KMA (tối đa {SESSION_SUMMARY_MAX_CHARS} ký tự).
Tóm tắt cũ:
{summary cũ}
Hội thoại:
{lịch sử gần nhất}
Câu hỏi: {câu hỏi vừa trả lời}
Trả lời: {câu trả lời vừa sinh}
Tóm tắt mới (một đoạn):
```

Tóm tắt này được giới hạn tối đa 1200 ký tự và cập nhật trực tiếp vào SessionState.summary. Từ đó về sau, thay vì đưa toàn bộ lịch sử vào prompt, các tác tử nhận

tóm tắt phiên như một đoạn ngắn gọn phía trên hội thoại gần nhất, đủ để hiểu ngữ cảnh mà không tốn token không cần thiết.

Cơ chế này đặc biệt quan trọng với Guardrail, Query Rewriter và Supervisor: cả ba đều nhận session_summary để hiểu ngữ cảnh phiên mà không cần đọc lại toàn bộ lịch sử hội thoại.

2.6.5 Phân phối lịch sử theo vai trò xử lý

Không phải tất cả các tác tử đều cần cùng một lượng lịch sử. Cung cấp quá nhiều lịch sử cho tác tử không cần thiết làm tốn context window; quá ít làm mất ngữ cảnh. Hệ thống phân phối lịch sử theo vai trò như Bảng 2.34.

Tác tử	Số lượt lịch sử	Lý do
Query Rewriter	8 lượt gần nhất	Cần đủ ngữ cảnh để giải quyết tham chiếu đại từ và câu hỏi tiếp nối.
Supervisor	6 lượt gần nhất	Đủ để nhận diện câu follow-up và giữ đúng nghiệp vụ đang hỏi.
Generate (RAG)	10 lượt gần nhất	Cần lịch sử dài nhất vì LLM sinh câu trả lời cần hiểu mạch trò chuyện.
Grade Lookup	10 lượt, lọc stub	Giống Generate nhưng lọc bỏ các lượt trả lời “không tìm thấy” ngăn để không bias LLM.

Bảng 2.34 Phân phối lịch sử hội thoại theo vai trò xử lý

Cơ chế lọc lịch sử cho Grade Lookup đáng chú ý: các lượt trả lời của trợ lý chỉ chứa chuỗi “Không tìm thấy thông tin trong tài liệu KMA” và dưới 200 ký tự bị loại khỏi context trước khi đưa vào LLM. Lý do là nếu lượt trước trả lời “không tìm thấy” do retrieve thất bại nhưng lượt này đã có dữ liệu, câu trả lời cũ sẽ làm LLM thiên về kết luận không có thông tin dù context đã đủ.

2.6.6 Cache truy vấn trong phiên

Trong một phiên hội thoại, sinh viên thỉnh thoảng hỏi lại câu hỏi tương tự, đặc biệt sau khi nhận được thông tin một phần và muốn xác nhận lại. Để tránh xử lý lại toàn bộ pipeline cho câu hỏi đã có kết quả, SessionState tích hợp bộ nhớ đệm trong phiên (in-session query cache).

Mỗi câu hỏi được chuẩn hóa bằng cách chuyển thường và loại khoảng trắng thừa, sau đó tính hash MD5 làm khóa. Kết quả được lưu kèm timestamp và có hiệu lực trong 600 giây. Khi một câu hỏi mới đến, hệ thống kiểm tra cache trước khi bắt đầu bất kỳ

bước xử lý nào; nếu tìm thấy kết quả còn hiệu lực, hệ thống trả về ngay mà không gọi bất kỳ tác tử nào.

Cache này hoạt động ở cấp phiên, không chia sẻ giữa các sinh viên, đảm bảo tính riêng tư: kết quả tra điểm của sinh viên A không ảnh hưởng đến kết quả tra cứu của sinh viên B dù câu hỏi giống nhau. Câu hỏi tra điểm theo MSSV, vốn có khả năng trùng lặp cao nhất trong một phiên, được hưởng lợi nhiều nhất từ cơ chế này.

2.6.7 Tích hợp ngữ cảnh vào luồng xử lý

Ngữ cảnh phiên, gồm `session_summary`, `hot_messages` và `last_agents`, được tích hợp xuyên suốt luồng xử lý của mỗi lượt hội thoại:

- Guardrail nhận `session_summary` để nhận biết câu hỏi follow-up nằm trong phạm vi KMA, tránh phân loại nhầm câu hỏi ngăn tiếp nối là off-topic.
- Query Rewriter nhận lịch sử 8 lượt để viết lại câu hỏi mơ hồ thành câu đứng độc lập có đủ nghĩa, quan trọng khi sinh viên dùng đại từ như “đó”, “môn đó”, “kỳ đó”.
- Supervisor nhận lịch sử 6 lượt và `session_summary` để nhận diện câu follow-up và định tuyến đúng tác tử, tránh tình huống câu hỏi “còn học kỳ 1?” được định tuyến sang tác tử khác thay vì tác tử đang phục vụ.
- Generate nhận lịch sử 10 lượt để LLM trả lời mạch lạc, liên kết câu trả lời hiện tại với các câu trả lời trước trong phiên.

Sau mỗi lượt xử lý hoàn chỉnh, hàm cập nhật sau lượt hội thoại ghi lại lịch sử gần nhất, số lượt đã xử lý, danh sách tác tử vừa tham gia và kích hoạt tóm tắt phiên nếu đến chu kỳ. Toàn bộ trạng thái được đồng bộ về Redis ngay trong lượt đó để đảm bảo lượt tiếp theo luôn thấy trạng thái nhất quán.

2.7 Tổng kết chương

Chương 2 đã trình bày chi tiết phương pháp xây dựng hệ thống trợ lý ảo đa tác tử cho Học viện KMA. Nội dung chương bao gồm kiến trúc tổng quan với sáu cơ chế điều phối chính là Guardrail, Query Rewriter, Planner, Supervisor, Router và Aggregator; phương pháp đánh giá độ phức tạp truy vấn Q_c dựa trên bốn thành phần đặc trưng kết hợp Shannon Entropy; ba pipeline truy xuất và sinh câu trả lời được lựa chọn động theo Q_c ; bảy tác tử chuyên biệt phụ trách từng nghiệp vụ học vụ; và cơ chế quản lý phiên hội thoại hai tầng nóng–lạnh.

Các phương pháp được trình bày trong chương này tạo nền tảng cho Chương 3, nơi hệ thống được phân tích, thiết kế và hiện thực hóa đầy đủ.

CHƯƠNG 3: PHÂN TÍCH THIẾT KẾ HỆ THỐNG

3.1 Yêu cầu hệ thống

3.1.1 Yêu cầu về hình thức sản phẩm

Hệ thống trợ lý ảo AI đa tác tử hỗ trợ sinh viên Học viện Kỹ thuật Mật mã được đóng gói và triển khai dưới dạng ứng dụng web, bao gồm các thành phần sau:

- **Giao diện người dùng:** Giao diện web được xây dựng bằng HTML, CSS và JavaScript thuần, phục vụ trực tiếp thông qua máy chủ backend mà không yêu cầu môi trường build hay framework frontend riêng biệt. Giao diện bao gồm trang chat chính cho sinh viên, trang tra cứu bảng điểm, trang lịch học và trang chủ hiển thị tin tức thông báo.
- **Backend API:** Hệ thống backend được xây dựng bằng FastAPI – một framework Python hiệu năng cao hỗ trợ lập trình bất đồng bộ. Backend đóng vai trò trung gian điều phối toàn bộ luồng xử lý, từ tiếp nhận yêu cầu của người dùng, gọi hệ thống đa tác tử AI, đến trả về phản hồi theo định dạng phù hợp.
- **Hỗ trợ phản hồi thời gian thực:** Hệ thống hỗ trợ cơ chế Server-Sent Events (SSE), cho phép phát trực tuyến (streaming) từng token phản hồi từ mô hình ngôn ngữ lớn đến giao diện người dùng theo thời gian thực, tăng trải nghiệm tương tác so với phương thức chờ phản hồi hoàn chỉnh truyền thống.
- **Trang quản trị:** Hệ thống có trang quản trị riêng dành cho quản trị viên, bao gồm giao diện đăng nhập, bảng điều khiển (dashboard) và các chức năng quản lý tài liệu nghiệp vụ, tin tức thông báo. Trang quản trị được bảo vệ bằng cơ chế xác thực hai bước qua OTP email.
- **Môi trường triển khai:** Hệ thống được thiết kế để triển khai trên máy chủ Linux hoặc Windows có cài đặt Python 3.10 trở lên. Các thành phần lưu trữ bao gồm Qdrant (cơ sở dữ liệu vector), PostgreSQL (cơ sở dữ liệu quan hệ) và Redis (lưu trữ phiên tốc độ cao), có thể được cung cấp qua dịch vụ cloud hoặc triển khai cục bộ.

3.1.2 Yêu cầu chức năng

3.1.2.1. Yêu cầu chức năng đối với sinh viên

Các chức năng dành cho sinh viên trong vai trò người dùng cuối của hệ thống bao gồm:

- **Chức năng hỏi đáp thông tin tuyển sinh và chương trình đào tạo:** Sinh viên có thể đặt câu hỏi về đề án tuyển sinh, điểm chuẩn, ngành học, chương trình đào tạo,

thông tin nhập học đại học và thạc sĩ của Học viện Kỹ thuật Mật mã. Hệ thống tra cứu và tổng hợp thông tin từ tài liệu tuyển sinh chính thức đã được nạp vào cơ sở tri thức.

- **Chức năng hỏi đáp quy chế đào tạo và quy định thi:** Sinh viên có thể tra cứu các quy định liên quan đến quy chế đào tạo, quy trình thi học phần, chuẩn đầu ra ngoại ngữ, thi tốt nghiệp và sát hạch. Hệ thống cung cấp câu trả lời dựa trên các văn bản quy chế chính thức của Học viện.
- **Chức năng tra cứu ma trận đề thi:** Sinh viên có thể hỏi về cấu trúc đề thi và ma trận đề thi của từng môn học, bao gồm tỷ lệ điểm, số lượng câu hỏi theo từng cấp độ nhận thức và nội dung trọng tâm ôn tập.
- **Chức năng tra cứu kết quả học tập theo mã số sinh viên:** Sinh viên có thể truy vấn điểm thi học phần theo mã số sinh viên (MSSV). Hệ thống tra cứu và tổng hợp kết quả từ các file bảng điểm công bố chính thức, hỗ trợ tìm kiếm theo học kỳ hoặc toàn bộ quá trình học tập.
- **Chức năng tra cứu lịch thi học phần:** Sinh viên có thể hỏi về lịch thi học phần (KTHP) theo học kỳ, năm học và đợt thi, bao gồm cả lịch thi lần 2 (thi lại). Hệ thống trả về thông tin ngày giờ, môn thi theo yêu cầu của sinh viên.
- **Chức năng tra cứu danh sách sinh viên dự thi:** Sinh viên có thể tra cứu thông tin về danh sách dự thi theo ngày thi, ca thi (sáng/chiều), phòng thi và môn thi, phục vụ nhu cầu xác nhận thông tin trước kỳ thi.
- **Chức năng hỏi đáp biểu mẫu và thủ tục hành chính:** Sinh viên có thể hỏi về các loại biểu mẫu như đơn xin nghỉ học, đơn phúc khảo, thủ tục cấp thẻ sinh viên, bảo hiểm y tế, thực tập, ra trường và các thủ tục hành chính khác. Hệ thống có khả năng cung cấp đường dẫn tải biểu mẫu sau khi điền sẵn thông tin.
- **Chức năng xử lý câu hỏi đa ý định:** Khi sinh viên đặt câu hỏi chứa nhiều ý định hoặc liên quan đến nhiều nghiệp vụ khác nhau trong cùng một lượt, hệ thống tự động phân tách và xử lý song song, sau đó tổng hợp thành một câu trả lời thống nhất.
- **Chức năng duy trì ngữ cảnh hội thoại đa lượt:** Hệ thống ghi nhận và duy trì ngữ cảnh trong suốt phiên hội thoại, cho phép sinh viên đặt các câu hỏi theo dõi (follow-up) mà không cần lặp lại thông tin đã đề cập trước đó. Lịch sử hội thoại được lưu trữ theo phiên (session) để phục vụ xử lý ngữ cảnh.

3.1.2.2. Yêu cầu chức năng đối với quản trị viên

Các chức năng dành cho quản trị viên của hệ thống bao gồm:

- **Chức năng đăng nhập hệ thống:** Quản trị viên đăng nhập qua cơ chế xác thực hai bước: nhập tên đăng nhập và mật khẩu ở bước đầu, sau đó xác nhận mã OTP được gửi đến địa chỉ email đã đăng ký. Sau khi xác thực thành công, hệ thống cấp

phát cặp JWT (access token và refresh token) để duy trì phiên làm việc.

- **Chức năng quản lý tài liệu nghiệp vụ:** Quản trị viên có thể tải lên, xóa và tải nạp (re-index) tài liệu PDF cho từng tác tử chuyên biệt trong hệ thống. Khi tải lên, hệ thống tự động thực hiện quét virus, trích xuất văn bản, phân đoạn (chunking), tạo vector embedding và lưu vào cơ sở dữ liệu vector Qdrant. Thông tin tài liệu (tên file, số chunk, trạng thái nạp, người upload) được ghi nhận vào cơ sở dữ liệu quan hệ PostgreSQL.
- **Chức năng quản lý danh mục biểu mẫu:** Quản trị viên có thể cập nhật tên hiển thị và thể loại cho từng file biểu mẫu trong danh mục, phục vụ việc hệ thống trả về thông tin chính xác khi sinh viên yêu cầu biểu mẫu.
- **Chức năng quản lý tin tức thông báo:** Quản trị viên có thể đăng tải hoặc xóa các file tin tức thông báo dưới dạng PDF. Nội dung tin tức được hiển thị công khai trên trang chủ hệ thống dành cho sinh viên.

3.1.3 Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, hệ thống cần đáp ứng một số yêu cầu về chất lượng vận hành như sau:

Hiệu năng và khả năng đáp ứng

Hệ thống cần đảm bảo tốc độ phản hồi chấp nhận được trong điều kiện vận hành thực tế. Cơ chế phản hồi streaming qua SSE cho phép người dùng nhận được token đầu tiên trong thời gian ngắn thay vì phải chờ toàn bộ câu trả lời được tạo ra. Để giảm độ trễ tổng thể, hệ thống áp dụng cơ chế cache truy vấn – các câu hỏi có nội dung tương đồng đã từng được xử lý trong phiên làm việc sẽ được phản hồi trực tiếp từ bộ nhớ đệm mà không cần gọi lại toàn bộ pipeline. Ngoài ra, các tác tử chuyên biệt xử lý câu hỏi đa ý định được thực thi song song (bất đồng bộ), rút ngắn thời gian xử lý tổng thể so với xử lý tuần tự.

Độ chính xác và tin cậy của câu trả lời

Hệ thống cần đảm bảo câu trả lời được cung cấp có căn cứ từ tài liệu nghiệp vụ chính thức, không bịa đặt thông tin. Để đạt yêu cầu này, hệ thống tích hợp cơ chế lọc phạm vi (Guardrail) nhằm từ chối các câu hỏi nằm ngoài phạm vi nghiệp vụ của Học viện. Bộ đánh giá độ liên quan tài liệu (Document Relevance Grader) được áp dụng để kiểm tra chất lượng tài liệu truy xuất trước khi đưa vào sinh câu trả lời. Đối với các câu hỏi phức tạp, hệ thống tự động leo thang lên pipeline Agentic RAG với cơ chế truy vấn lặp và viết lại truy vấn nhằm nâng cao chất lượng thông tin tìm kiếm được.

Khả năng mở rộng

Hệ thống được thiết kế theo kiến trúc module hóa, cho phép bổ sung tác tử chuyên biệt mới hoặc mở rộng sang nghiệp vụ mới mà không cần thay đổi logic điều phối cốt

lỗi. Việc thêm một nghiệp vụ mới chỉ yêu cầu khai báo cấu hình trong tệp cấu hình tập trung và nạp tài liệu tương ứng vào cơ sở dữ liệu vector. Kiến trúc Supervisor–Specialist đảm bảo các tác tử mới được tích hợp liền mạch vào luồng định tuyến hiện có.

Bảo mật

Hệ thống áp dụng các biện pháp bảo mật đa lớp, đặc biệt đối với khu vực quản trị:

- Xác thực hai bước (OTP qua email) cho tài khoản quản trị viên, ngăn chặn truy cập trái phép ngay cả khi mật khẩu bị lộ.
- Cơ chế giới hạn tốc độ (rate limiting) áp dụng cho các endpoint đăng nhập, xác thực OTP và upload tài liệu nhằm chống tấn công brute-force.
- Quét virus tự động đối với toàn bộ file tải lên trước khi xử lý và lưu trữ.
- Ghi nhật ký kiểm toán (audit log) đầy đủ cho mọi thao tác của quản trị viên bao gồm đăng nhập, upload, xóa tài liệu và cập nhật cấu hình.
- Phân quyền truy cập rõ ràng: sinh viên chỉ truy cập các endpoint công khai; toàn bộ tính năng quản trị yêu cầu JWT hợp lệ với vai trò quản trị viên.

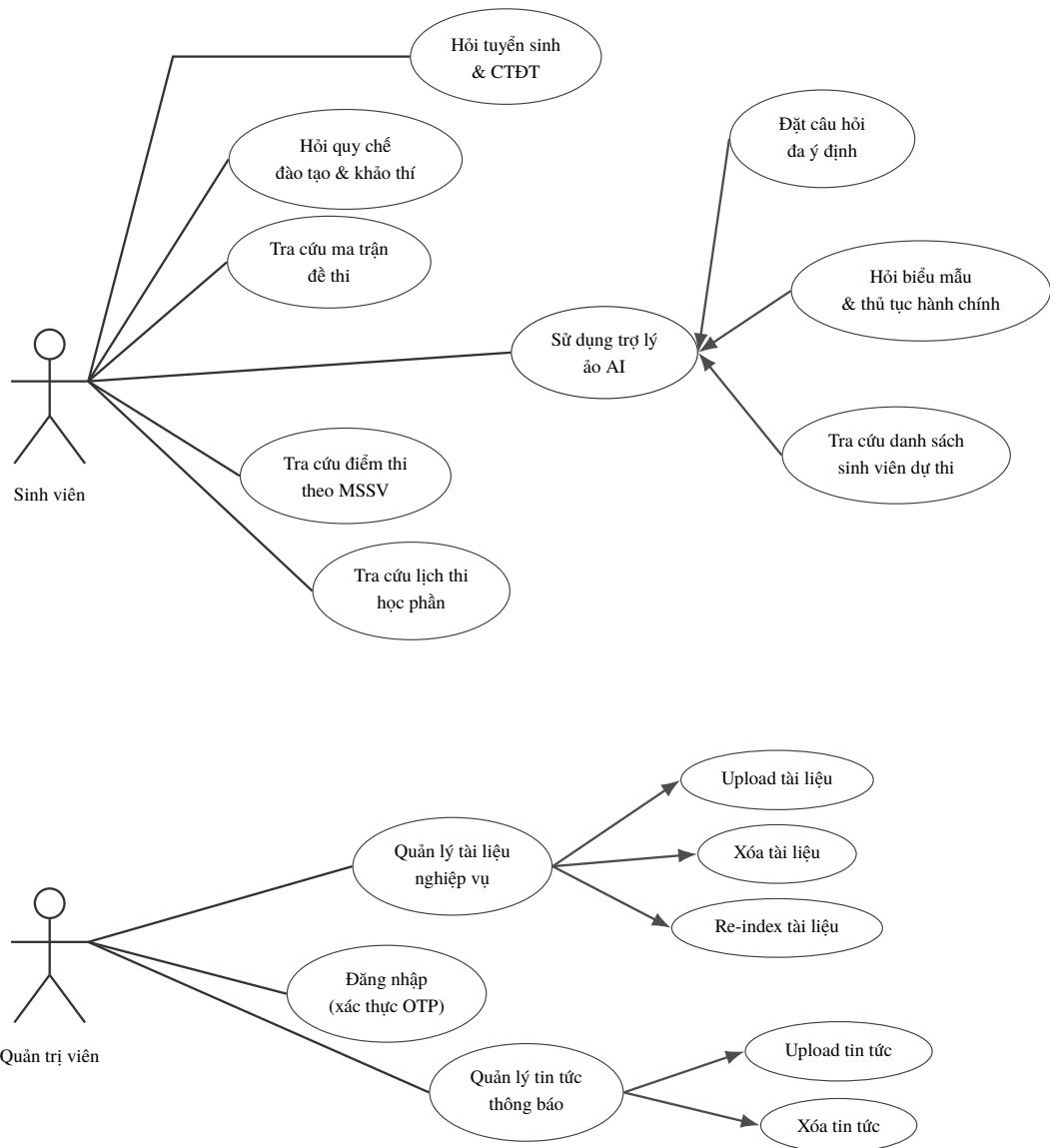
Tính ổn định và khả năng phục hồi

Hệ thống cần vận hành ổn định trong điều kiện thực tế với nhiều phiên hội thoại đồng thời. Các lỗi phát sinh từ mô hình ngôn ngữ lớn hoặc cơ sở dữ liệu được xử lý có kiểm soát, tránh để lỗi lan rộng ảnh hưởng đến phiên khác. Hệ thống cung cấp endpoint /health để giám sát trạng thái hoạt động. Phiên hội thoại được lưu trữ theo kiến trúc hot-cold: dữ liệu phiên đang hoạt động được giữ trong Redis với TTL hai giờ, và được chuyển xuống PostgreSQL khi kết thúc, đảm bảo tính bền vững của dữ liệu ngay cả khi hệ thống khởi động lại.

3.2 Phân tích hệ thống

3.2.1 Biểu đồ usecase tổng quát

Hệ thống có hai tác nhân chính là Sinh viên (người dùng cuối) và Quản trị viên. Biểu đồ usecase tổng quát được thể hiện như Hình 3.1.



Hình 3.1 Biểu đồ usecase tổng quát của hệ thống

Mã PlantUML dùng để sinh biểu đồ được lưu kèm trong thư mục PlantUML của dự án.

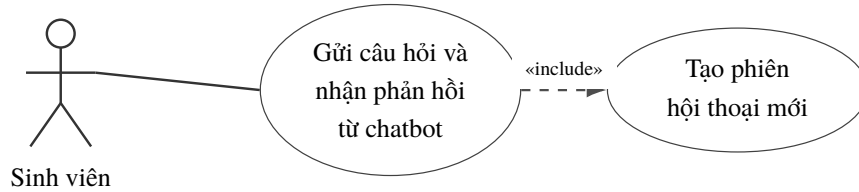
Hệ thống có các usecase chính là:

- Hỏi đáp thông tin tuyển sinh và chương trình đào tạo.
- Hỏi đáp quy chế đào tạo và khảo thí.
- Tra cứu ma trận đề thi theo môn học.
- Tra cứu điểm thi theo mã số sinh viên (MSSV).
- Tra cứu lịch thi học phần.
- Tra cứu danh sách sinh viên dự thi.
- Hỏi đáp biểu mẫu và thủ tục hành chính.
- Đặt câu hỏi đa ý định (nhiều nghiệp vụ trong một câu).
- Đăng nhập hệ thống quản trị (xác thực OTP email).

- Quản lý tài liệu nghiệp vụ (upload, xóa, re-index).
- Quản lý tin tức thông báo (upload, xóa).

3.2.2 Đặc tả usecase

3.2.2.1. Usecase gửi câu hỏi và nhận phản hồi từ chatbot

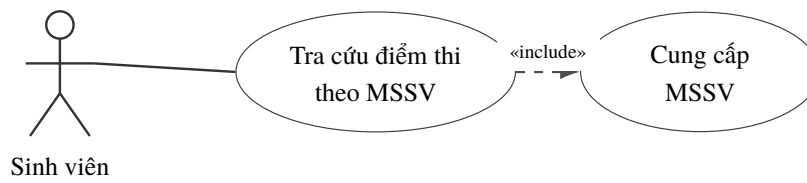


Hình 3.2 Usecase gửi câu hỏi và nhận phản hồi từ chatbot

Bảng 3.1 Đặc tả Usecase gửi câu hỏi và nhận phản hồi từ chatbot

Tên Usecase	UC01 – Gửi câu hỏi và nhận phản hồi từ chatbot
Tác nhân	Sinh viên
Mô tả	Usecase mô tả cách sinh viên gửi câu hỏi liên quan đến thông tin nghiệp vụ của Học viện và nhận câu trả lời từ hệ thống chatbot AI đa tác tử.
Luồng chính	1. Sinh viên truy cập giao diện chat trên website. 2. Hệ thống tự động tạo phiên hội thoại mới (session). 3. Sinh viên nhập câu hỏi và gửi. 4. Hệ thống lần lượt thực hiện: lọc phạm vi (Guardrail), viết lại truy vấn nếu cần (Query Rewriter), định tuyến đến tác tử chuyên biệt phù hợp (Supervisor). 5. Tác tử chuyên biệt truy xuất tài liệu từ Qdrant và sinh câu trả lời qua mô hình ngôn ngữ lớn. 6. Hệ thống phát trực tuyến (streaming) câu trả lời theo từng token đến giao diện người dùng.
Luồng thay thế	Nếu câu hỏi nằm ngoài phạm vi nghiệp vụ của Học viện, hệ thống trả về thông báo từ chối và đề nghị sinh viên đặt lại câu hỏi phù hợp.
Kết quả	Sinh viên nhận được câu trả lời chính xác từ hệ thống chatbot.
Yêu cầu đặc biệt	Không có.

3.2.2.2. Usecase tra cứu điểm thi theo mã số sinh viên

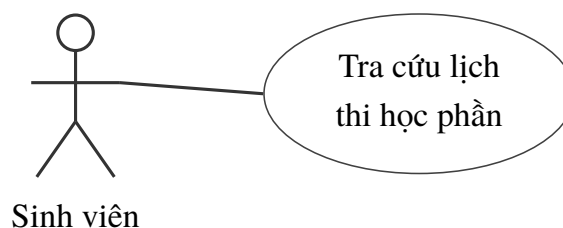


Hình 3.3 Usecase tra cứu điểm thi theo mã số sinh viên

Bảng 3.2 Đặc tả Usecase tra cứu điểm thi theo MSSV

Tên Usecase	UC02 – Tra cứu điểm thi theo mã số sinh viên
Tác nhân	Sinh viên
Mô tả	Usecase mô tả cách sinh viên tra cứu kết quả học tập theo mã số sinh viên thông qua hệ thống chatbot.
Luồng chính	1. Sinh viên đặt câu hỏi về điểm thi có kèm mã số sinh viên (MSSV). 2. Hệ thống nhận diện ý định tra cứu điểm thi và định tuyến đến tác tử diem_thi. 3. Hệ thống trích xuất MSSV từ câu hỏi và tìm kiếm trong tập tài liệu bảng điểm công bố. 4. Hệ thống tổng hợp kết quả bao gồm tên môn học, điểm thành phần, điểm thi và điểm tổng kết. 5. Câu trả lời được trả về cho sinh viên.
Luồng thay thế	Nếu MSSV không tìm thấy trong cơ sở dữ liệu, hệ thống thông báo không có kết quả và đề nghị kiểm tra lại mã số.
Kết quả	Sinh viên nhận được thông tin điểm thi theo yêu cầu.
Yêu cầu đặc biệt	MSSV phải được cung cấp rõ ràng trong nội dung câu hỏi.

3.2.2.3. Usecase tra cứu lịch thi học phần

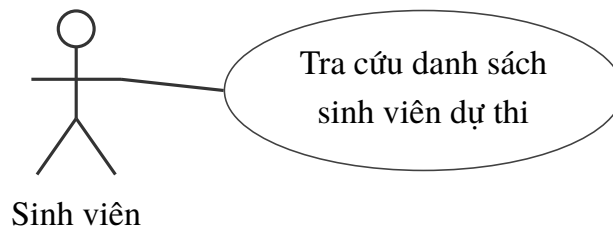


Hình 3.4 Usecase tra cứu lịch thi học phần

Bảng 3.3 Đặc tả Usecase tra cứu lịch thi học phần

Tên Usecase	UC03 – Tra cứu lịch thi học phần
Tác nhân	Sinh viên
Mô tả	Usecase mô tả cách sinh viên tra cứu lịch thi học phần (KTHP) theo học kỳ, năm học hoặc đợt thi thông qua hệ thống chatbot.
Luồng chính	1. Sinh viên đặt câu hỏi về lịch thi học phần, có thể kèm theo học kỳ, năm học hoặc tên môn. 2. Hệ thống định tuyến đến tác tử <code>lich_thi</code>. 3. Tác tử tra cứu tài liệu lịch thi tương ứng trong cơ sở dữ liệu vector. 4. Hệ thống trả về thông tin ngày thi, ca thi, môn thi theo yêu cầu.
Luồng thay thế	Nếu không tìm thấy lịch thi phù hợp, hệ thống thông báo chưa có dữ liệu và đề nghị sinh viên kiểm tra lại thông tin học kỳ.
Kết quả	Sinh viên nhận được thông tin lịch thi theo yêu cầu.
Yêu cầu đặc biệt	Không có.

3.2.2.4. Usecase tra cứu danh sách sinh viên dự thi

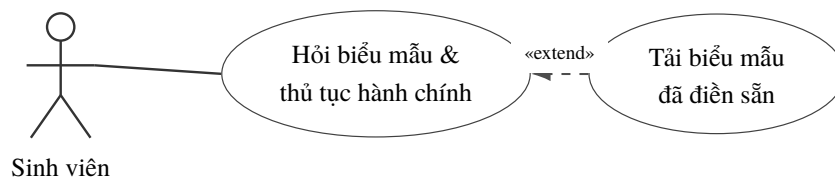


Hình 3.5 Usecase tra cứu danh sách sinh viên dự thi

Bảng 3.4 Đặc tả Usecase tra cứu danh sách sinh viên dự thi

Tên Usecase	UC04 – Tra cứu danh sách sinh viên dự thi
Tác nhân	Sinh viên
Mô tả	Usecase mô tả cách sinh viên tra cứu danh sách dự thi theo ngày thi, ca thi, phòng thi hoặc môn thi.
Luồng chính	1. Sinh viên đặt câu hỏi về danh sách thi, có thể kèm theo ngày thi, ca thi, tên môn hoặc MSSV. 2. Hệ thống định tuyến đến tác tử danh_sach_thi. 3. Tác tử tra cứu tài liệu danh sách thi công bố, tìm kiếm theo các tiêu chí được cung cấp. 4. Hệ thống trả về danh sách sinh viên dự thi hoặc thông tin phòng thi, ca thi cụ thể.
Luồng thay thế	Nếu không tìm thấy thông tin phù hợp, hệ thống thông báo không có dữ liệu cho tiêu chí đã cung cấp.
Kết quả	Sinh viên nhận được thông tin danh sách dự thi theo yêu cầu.
Yêu cầu đặc biệt	Không có.

3.2.2.5. Usecase hỏi biểu mẫu và thủ tục hành chính

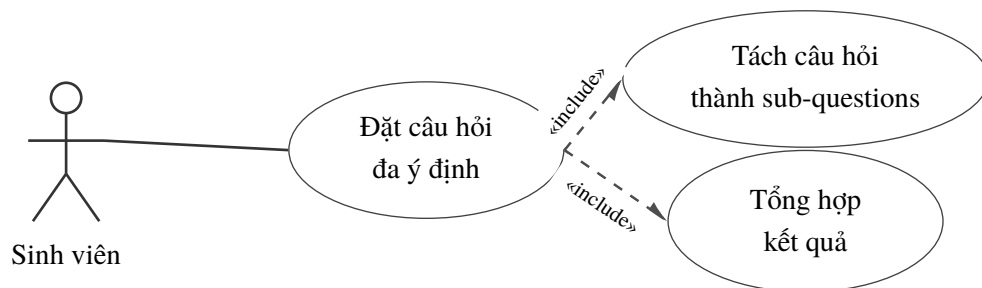


Hình 3.6 Usecase hỏi biểu mẫu và thủ tục hành chính

Bảng 3.5 Đặc tả Usecase hỏi biểu mẫu và thủ tục hành chính

Tên Usecase	UC05 – Hỏi biểu mẫu và thủ tục hành chính
Tác nhân	Sinh viên
Mô tả	Usecase mô tả cách sinh viên tra cứu thông tin về biểu mẫu, thủ tục hành chính và nhận đường dẫn tải biểu mẫu khi cần.
Luồng chính	1. Sinh viên đặt câu hỏi về biểu mẫu hoặc thủ tục hành chính như phúc khảo, nghỉ học, thực tập. 2. Hệ thống định tuyến đến tác tử bieu_mau. 3. Tác tử tìm kiếm thông tin biểu mẫu phù hợp trong cơ sở tri thức. 4. Hệ thống trả về hướng dẫn thủ tục và danh sách biểu mẫu liên quan kèm đường dẫn tải về.
Luồng thay thế	Nếu không tìm thấy biểu mẫu phù hợp, hệ thống cung cấp hướng dẫn liên hệ phòng ban có thẩm quyền.
Kết quả	Sinh viên nhận được thông tin thủ tục và có thể tải biểu mẫu cần thiết.
Yêu cầu đặc biệt	Không có.

3.2.2.6. Usecase đặt câu hỏi đa ý định

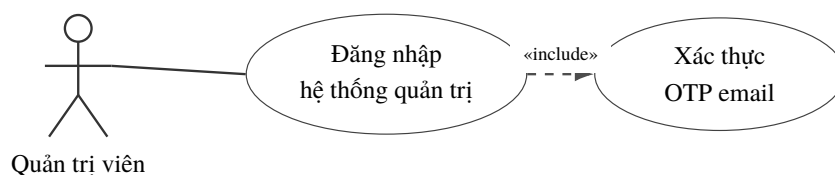


Hình 3.7 Usecase đặt câu hỏi đa ý định

Bảng 3.6 Đặc tả Usecase đặt câu hỏi đa ý định

Tên Usecase	UC06 – Đặt câu hỏi đa ý định
Tác nhân	Sinh viên
Mô tả	Usecase mô tả khả năng xử lý câu hỏi chứa nhiều ý định hoặc liên quan đến nhiều nghiệp vụ khác nhau trong cùng một lượt hỏi.
Luồng chính	1. Sinh viên đặt câu hỏi chứa nhiều nội dung liên quan đến các nghiệp vụ khác nhau. 2. Thành phần Planner phân tích và tách câu hỏi thành các sub-questions độc lập. 3. Hệ thống định tuyến từng sub-question đến tác tử chuyên biệt phù hợp, xử lý song song. 4. Thành phần Aggregator tổng hợp kết quả từ các tác tử thành một câu trả lời thống nhất. 5. Hệ thống trả về câu trả lời đầy đủ cho sinh viên.
Luồng thay thế	Nếu một trong các sub-questions không tìm được kết quả, hệ thống vẫn trả về câu trả lời cho các phần còn lại và thông báo phần không có dữ liệu.
Kết quả	Sinh viên nhận được câu trả lời đầy đủ cho toàn bộ nội dung đã hỏi.
Yêu cầu đặc biệt	Không có.

3.2.2.7. Usecase đăng nhập hệ thống quản trị

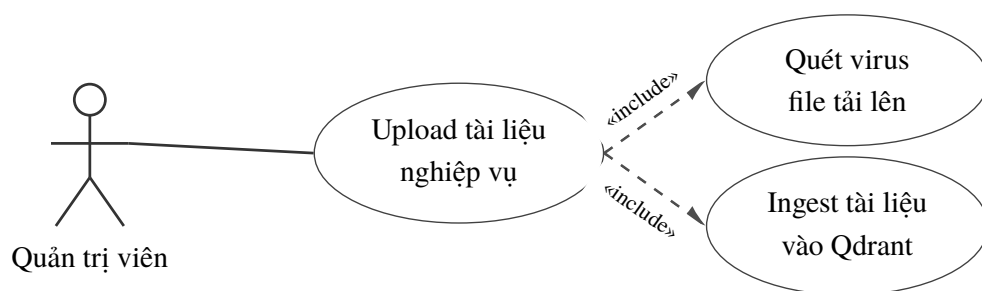


Hình 3.8 Usecase đăng nhập hệ thống quản trị

Bảng 3.7 Đặc tả Usecase đăng nhập hệ thống quản trị

Tên Usecase	UC07 – Đăng nhập hệ thống quản trị
Tác nhân	Quản trị viên
Mô tả	Usecase mô tả quá trình quản trị viên xác thực danh tính để truy cập trang quản trị hệ thống thông qua cơ chế xác thực hai bước.
Luồng chính	1. Quản trị viên truy cập trang đăng nhập /admin/login. 2. Quản trị viên nhập tên đăng nhập và mật khẩu. 3. Hệ thống xác thực thông tin và gửi mã OTP đến địa chỉ email đã đăng ký. 4. Quản trị viên nhập mã OTP trong thời hạn hiệu lực. 5. Hệ thống xác thực OTP thành công và cấp phát cặp JWT (access token, refresh token). 6. Quản trị viên được điều hướng vào trang dashboard quản trị.
Luồng thay thế	Nếu mật khẩu sai hoặc OTP không đúng, hệ thống thông báo lỗi và giới hạn số lần thử. Nếu vượt quá số lần cho phép, phiên xác thực bị khóa.
Kết quả	Quản trị viên đăng nhập thành công và có thể sử dụng các chức năng quản trị.
Yêu cầu đặc biệt	Tài khoản quản trị viên phải có vai trò DEAN; hệ thống SMTP phải được cấu hình để gửi OTP.

3.2.2.8. Usecase upload tài liệu nghiệp vụ



Hình 3.9 Usecase upload tài liệu nghiệp vụ

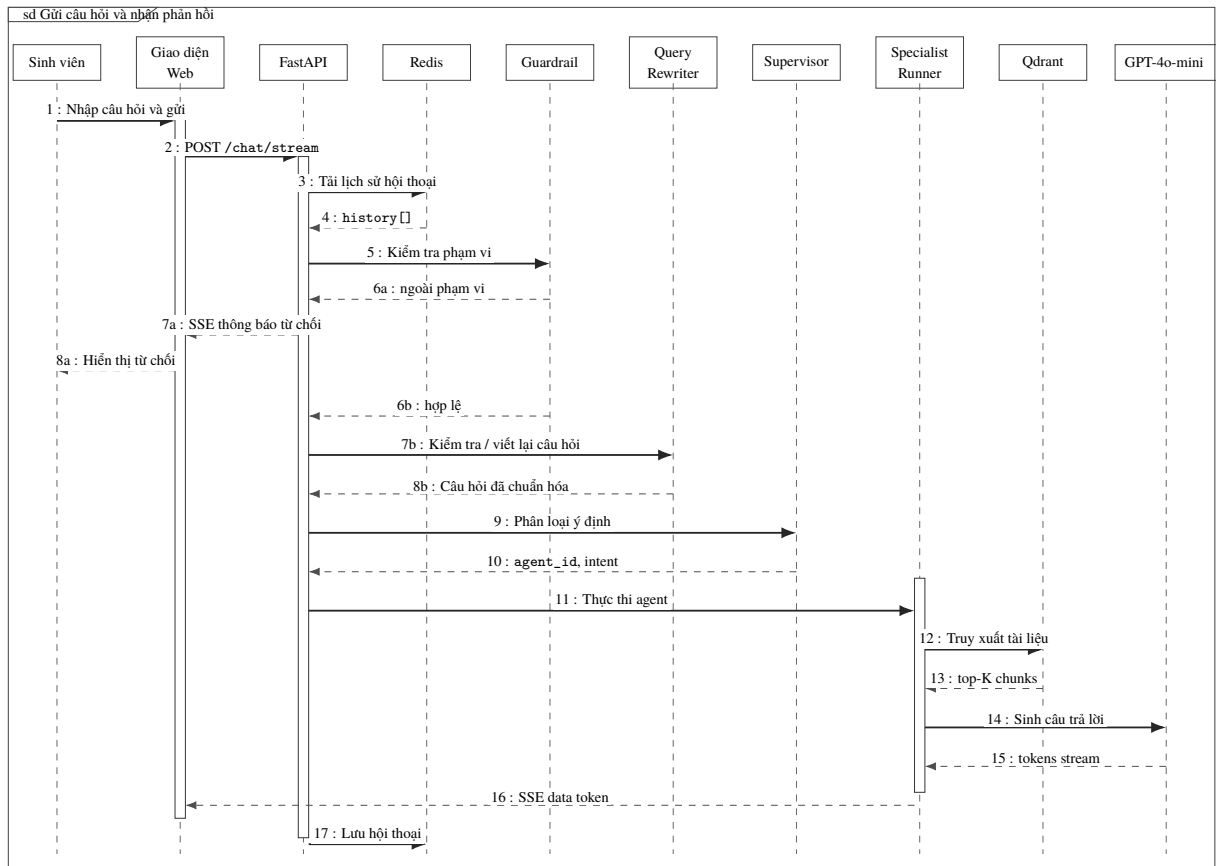
Bảng 3.8 Đặc tả Usecase upload tài liệu nghiệp vụ

Tên Usecase	UC08 – Upload tài liệu nghiệp vụ vào hệ thống
Tác nhân	Quản trị viên
Mô tả	Usecase mô tả quá trình quản trị viên tải lên tài liệu PDF và nạp vào cơ sở dữ liệu vector để hệ thống có thể tra cứu.
Luồng chính	1. Quản trị viên truy cập trang quản lý tài liệu trên dashboard. 2. Quản trị viên chọn tác tử chuyên biệt đích, ví dụ tuyen_sinh, lich_thi, và chọn file PDF cần tải lên. 3. Hệ thống thực hiện quét virus đối với file trước khi xử lý. 4. Hệ thống lưu file, trích xuất văn bản, thực hiện phân đoạn (chunking), tạo vector embedding và lưu vào Qdrant collection kma_docs với metadata agent_id tương ứng. 5. Hệ thống ghi nhận thông tin tài liệu (tên file, số chunk, trạng thái) vào PostgreSQL và tạo bản ghi kiểm toán.
Luồng thay thế	Nếu file đã tồn tại, hệ thống yêu cầu quản trị viên xác nhận ghi đè. Nếu quét virus phát hiện mối đe dọa, file bị từ chối.
Kết quả	Tài liệu được nạp thành công, sẵn sàng phục vụ truy vấn của sinh viên.
Yêu cầu đặc biệt	File phải đúng định dạng PDF; kích thước không vượt quá giới hạn cấu hình.

3.2.3 Biểu đồ tuần tự

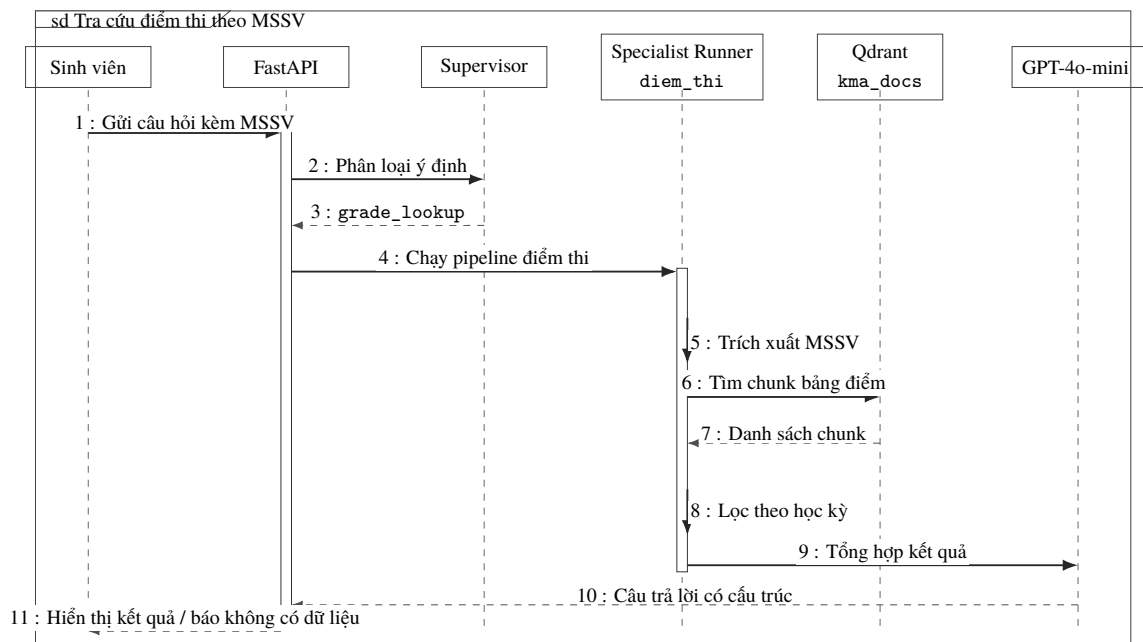
Biểu đồ tuần tự mô tả sự tương tác theo thời gian giữa người dùng, giao diện, back-end và các thành phần xử lý chính của hệ thống.

3.2.3.1. Biểu đồ tuần tự luồng gửi câu hỏi và nhận phản hồi



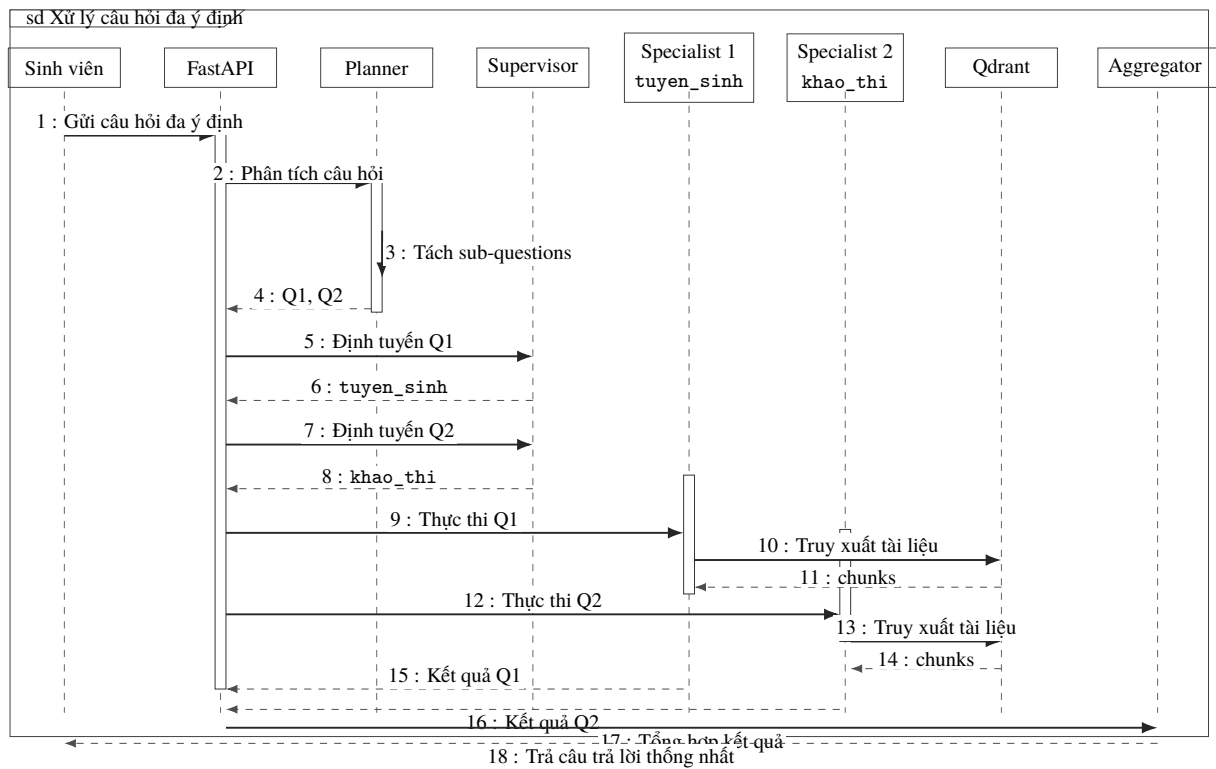
Hình 3.10 Biểu đồ tuần tự luồng gửi câu hỏi và nhận phản hồi

3.2.3.2. Biểu đồ tuần tự luồng tra cứu điểm thi theo MSSV



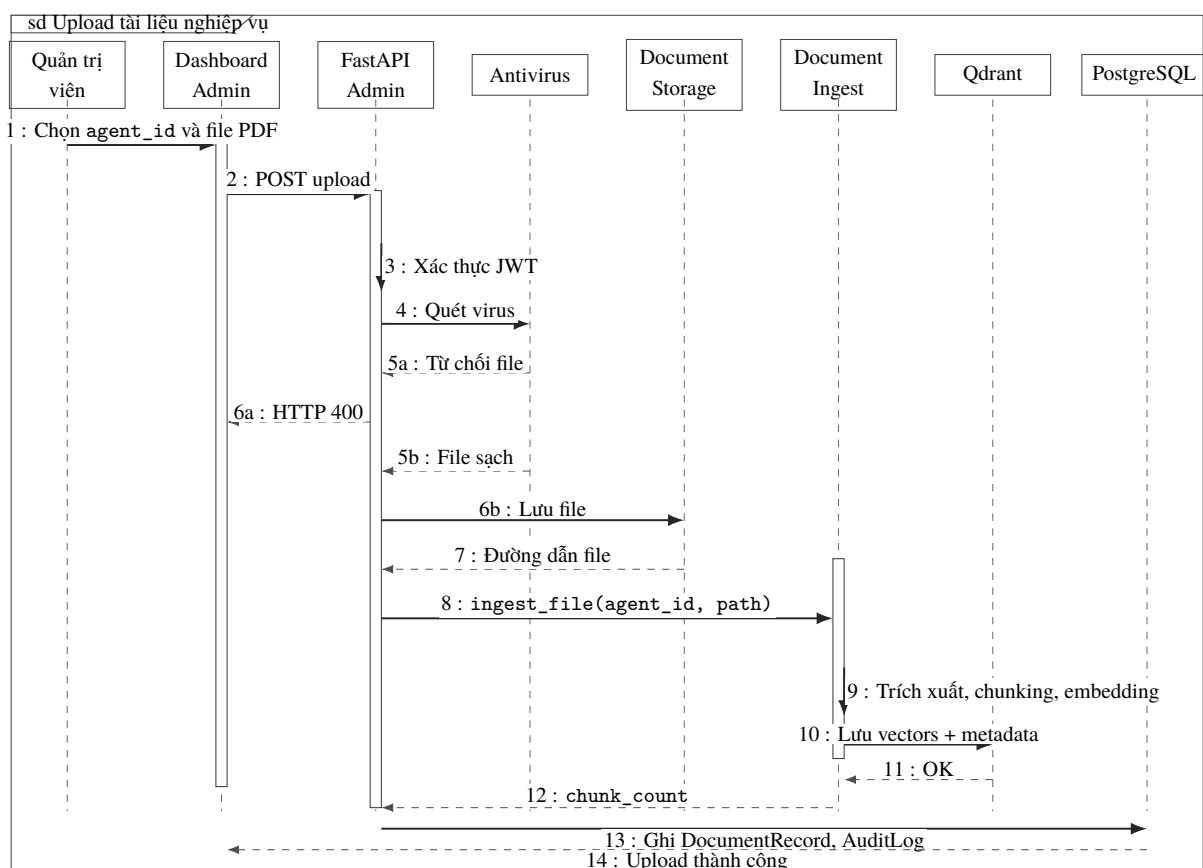
Hình 3.11 Biểu đồ tuần tự luồng tra cứu điểm thi theo MSSV

3.2.3.3. Biểu đồ tuần tự luồng xử lý câu hỏi đa ý định



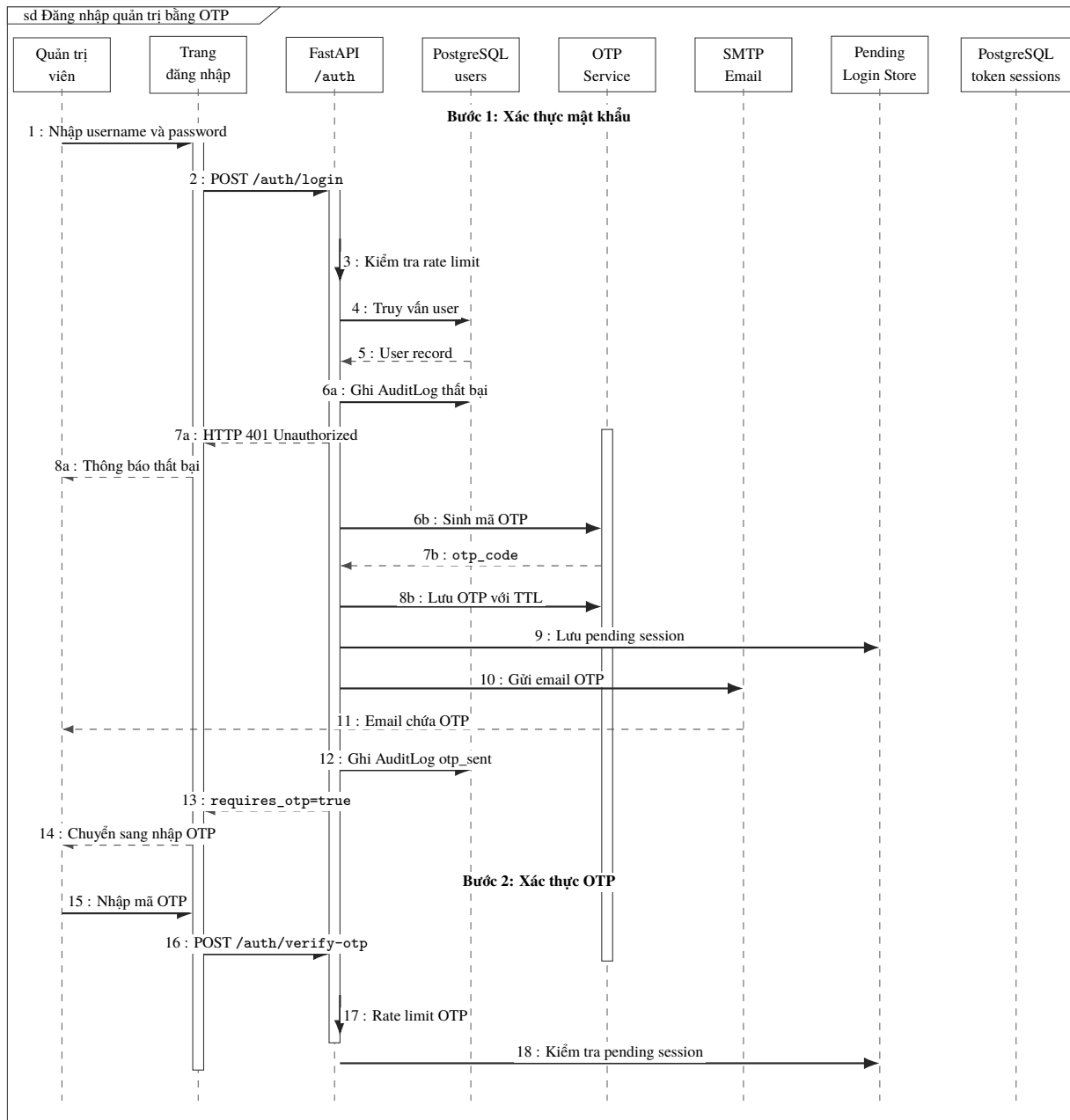
Hình 3.12 Biểu đồ tuần tự luồng xử lý câu hỏi đa ý định

3.2.3.4. Biểu đồ tuần tự luồng upload tài liệu nghiệp vụ

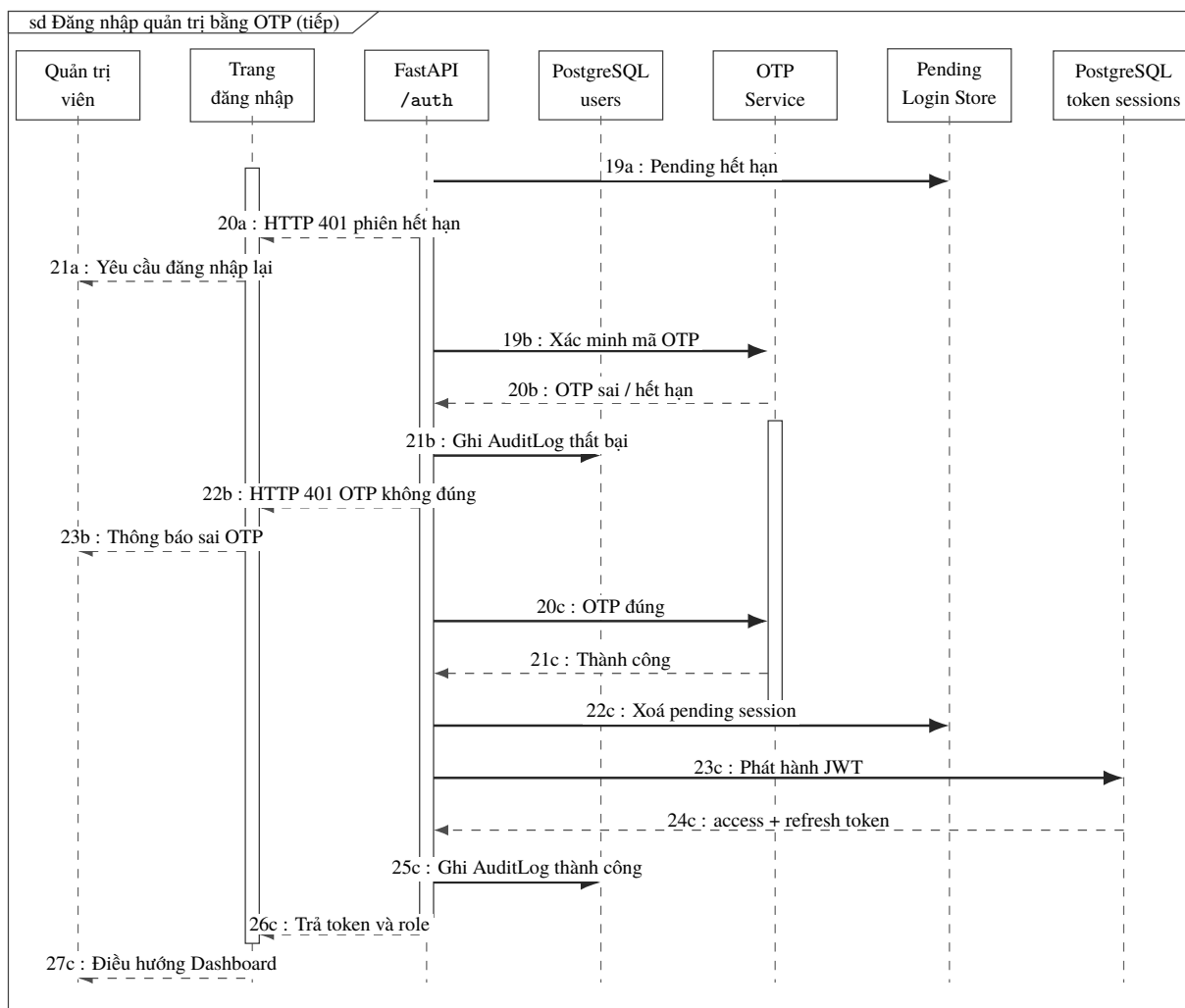


Hình 3.13 Biểu đồ tuần tự luồng upload tài liệu nghiệp vụ

3.2.3.5. Biểu đồ tuần tự luồng đăng nhập hệ thống quản trị



Hình 3.14 Biểu đồ tuần tự luồng đăng nhập hệ thống quản trị bằng OTP



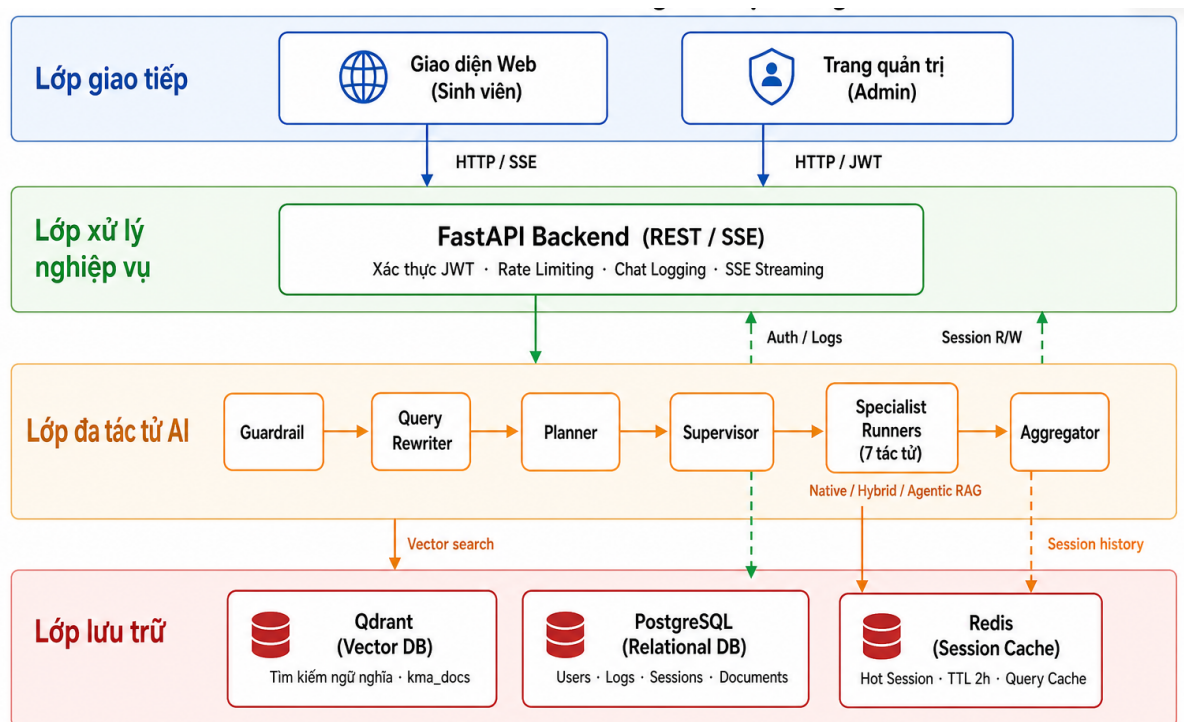
Hình 3.15 Biểu đồ tuần tự luồng xác thực OTP và phát hành JWT

Luồng đăng nhập gồm hai bước tách biệt rõ ràng: bước thứ nhất xác thực mật khẩu và kích hoạt gửi OTP qua email; bước thứ hai xác minh OTP để nhận JWT. Cơ chế giới hạn tốc độ được áp dụng tại cả hai bước nhằm chống tấn công brute-force. Toàn bộ thao tác đăng nhập, gửi OTP và xác minh OTP đều được ghi vào `audit_logs` trong PostgreSQL.

3.3 Thiết kế hệ thống

3.3.1 Kiến trúc tổng thể hệ thống

Hệ thống được tổ chức theo kiến trúc phân lớp, trong đó mỗi lớp đảm nhiệm một nhóm trách nhiệm riêng biệt và giao tiếp với lớp liền kề thông qua các giao diện được định nghĩa rõ ràng. Kiến trúc tổng thể gồm bốn lớp chính: lớp giao tiếp, lớp xử lý nghiệp vụ, lớp đa tác tử AI và lớp lưu trữ. Sơ đồ kiến trúc tổng thể của hệ thống được thể hiện trong Hình 3.16.



Hình 3.16 Sơ đồ kiến trúc tổng thể hệ thống

Lớp giao tiếp (Presentation Layer)

Lớp giao tiếp bao gồm giao diện web dành cho sinh viên và trang quản trị dành cho quản trị viên. Giao diện web được phục vụ trực tiếp bởi máy chủ backend dưới dạng các tệp tĩnh. Toàn bộ tương tác giữa giao diện và hệ thống backend diễn ra thông qua giao thức HTTP/REST, trong đó phản hồi chatbot được truyền tải theo cơ chế Server-Sent Events (SSE) nhằm đảm bảo trải nghiệm streaming thời gian thực.

Lớp xử lý nghiệp vụ (Application Layer)

Lớp này được hiện thực bởi framework FastAPI, chịu trách nhiệm tiếp nhận yêu cầu từ lớp giao tiếp, xác thực đầu vào, quản lý xác thực người dùng (JWT, OTP) và điều phối luồng xử lý đến lớp đa tác tử. Lớp ứng dụng cũng đảm nhiệm ghi nhật ký chat, theo dõi kết nối đồng thời và thu thập chỉ số giám sát hệ thống.

Lớp đa tác tử AI (Multi-Agent Layer)

Đây là lớp cốt lõi của hệ thống, bao gồm các thành phần điều phối (Guardrail, Query Rewriter, Planner, Supervisor, Aggregator) và bảy tác tử chuyên biệt tương ứng với bảy nghiệp vụ của Học viện. Mỗi tác tử chuyên biệt vận hành một pipeline RAG độc lập, được lựa chọn linh hoạt (Native, Hybrid hoặc Agentic) dựa trên mức độ phức tạp của truy vấn. Lớp này giao tiếp trực tiếp với lớp lưu trữ để thực hiện tìm kiếm ngữ nghĩa và quản lý ngữ cảnh hội thoại.

Lớp lưu trữ (Storage Layer)

Lớp lưu trữ bao gồm ba thành phần với vai trò bổ trợ nhau:

- **Qdrant (cơ sở dữ liệu vector):** lưu trữ toàn bộ tài liệu nghiệp vụ dưới dạng vector embedding, phục vụ tìm kiếm ngữ nghĩa cho các tác tử chuyên biệt. Tất cả tài liệu được tổ chức trong một collection duy nhất kma_docs, phân biệt theo trường metadata agent_id.
- **PostgreSQL (cơ sở dữ liệu quan hệ):** lưu trữ thông tin tài khoản quản trị viên, nhật ký chat, nhật ký kiểm toán, hồ sơ tài liệu đã nạp và phiên hội thoại ở trạng thái lạnh (cold storage).
- **Redis (bộ nhớ đệm tốc độ cao):** lưu trữ phiên hội thoại đang hoạt động (hot session) với thời gian sống hai giờ, đảm bảo độ trễ truy xuất lịch sử hội thoại ở mức tối thiểu trong quá trình tương tác.

3.3.2 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng hai loại cơ sở dữ liệu với vai trò bổ trợ nhau: cơ sở dữ liệu quan hệ PostgreSQL phục vụ lưu trữ có cấu trúc và cơ sở dữ liệu vector Qdrant phục vụ tìm kiếm ngữ nghĩa.

3.3.2.1. Cơ sở dữ liệu quan hệ (PostgreSQL)

Cơ sở dữ liệu quan hệ bao gồm sáu bảng với chức năng phân tách rõ ràng.

Bảng users – Lưu trữ tài khoản quản trị viên

Bảng 3.9 Cấu trúc bảng users

STT	Tên trường	Kiểu dữ liệu	Ý nghĩa
1	id	Integer (PK)	Khóa chính tự tăng
2	username	Varchar, unique	Tên đăng nhập
3	email	Varchar, unique	Địa chỉ email nhận OTP
4	hashed_password	Varchar	Mật khẩu đã băm (bcrypt)
5	role	Enum	Vai trò người dùng (DEAN)
6	full_name	Varchar	Họ và tên
7	phone_number	Varchar	Số điện thoại
8	is_active	Boolean	Trạng thái tài khoản
9	created_at	DateTime	Thời điểm tạo tài khoản

Bảng chat_session_archives – Lưu trữ lạnh phiên hội thoại

Bảng 3.10 Cấu trúc bảng chat_session_archives

STT	Tên trường	Kiểu dữ liệu	Ý nghĩa
1	id	Integer (PK)	Khóa chính tự tăng
2	session_id	Varchar(64), unique	Định danh phiên hội thoại
3	summary	Text	Tóm tắt nội dung hội thoại (sinh bởi LLM)
4	turn_count	Integer	Số lượt hỏi đáp trong phiên
5	last_agents	Varchar(500)	Danh sách tác tử đã xử lý gần nhất
6	messages_json	Text	Toàn bộ lịch sử tin nhắn (JSON)
7	created_at	DateTime	Thời điểm tạo phiên
8	updated_at	DateTime	Thời điểm cập nhật gần nhất

Bảng này lưu dữ liệu phiên hội thoại sau khi được flush từ Redis (cold storage). Khi người dùng quay lại, hệ thống khôi phục phiên từ bảng này lên Redis để tiếp tục xử lý.

Bảng chat_logs – Nhật ký mỗi lượt hội thoại

Bảng 3.11 Cấu trúc bảng chat_logs

STT	Tên trường	Kiểu dữ liệu	Ý nghĩa
1	id	Integer (PK)	Khóa chính tự tăng
2	session_id	Varchar(64)	Định danh phiên hội thoại
3	question_preview	Varchar(500)	Nội dung tóm tắt câu hỏi
4	primary_agent	Varchar(32)	Tác tử chính xử lý câu hỏi
5	agents_used	Varchar(200)	Tất cả tác tử đã tham gia
6	pipeline	Varchar(32)	Pipeline đã sử dụng (native/hybrid/agentive)
7	in_scope	Boolean	Câu hỏi có trong phạm vi nghiệp vụ
8	qc	Float	Điểm phức tạp truy vấn (Q_c)

9	t_total	Float	Tổng thời gian xử lý (giây)
10	source_count	Integer	Số lượng tài liệu nguồn trả về
11	client_ip	Varchar(64)	Địa chỉ IP của người dùng
12	stream	Boolean	Phản hồi theo dạng streaming hay không
13	created_at	DateTime	Thời điểm ghi nhận

Bảng document_records – Theo dõi tài liệu đã nạp

Bảng 3.12 Cấu trúc bảng document_records

STT	Tên trường	Kiểu dữ liệu	Ý nghĩa
1	id	Integer (PK)	Khóa chính tự tăng
2	agent_id	Varchar(32)	Tác tử chuyên biệt sở hữu tài liệu
3	filename	Varchar(255)	Tên file tài liệu
4	source_key	Varchar(512), unique	Khóa định danh duy nhất của tài liệu
5	file_size_bytes	Integer	Kích thước file (byte)
6	chunk_count	Integer	Số đoạn văn bản sau phân đoạn
7	status	Varchar(32)	Trạng thái nạp (indexed / failed / deleted)
8	uploaded_by	Varchar(64)	Tên tài khoản quản trị viên đã upload
9	created_at	DateTime	Thời điểm nạp tài liệu
10	updated_at	DateTime	Thời điểm cập nhật gần nhất

Bảng audit_logs – Nhật ký kiểm toán thao tác quản trị

Bảng 3.13 Cấu trúc bảng audit_logs

STT	Tên trường	Kiểu dữ liệu	Ý nghĩa
1	id	Integer (PK)	Khóa chính tự tăng

2	username	Varchar(64)	Tên tài khoản thực hiện thao tác
3	ip	Varchar(64)	Địa chỉ IP của quản trị viên
4	action	Varchar(64)	Loại thao tác (login, document_upload, document_delete, ...)
5	detail	Text	Chi tiết bổ sung của thao tác
6	success	Boolean	Thao tác thành công hay thất bại
7	created_at	DateTime	Thời điểm ghi nhận

Bảng token_sessions – Quản lý phiên JWT

Bảng 3.14 Cấu trúc bảng token_sessions

STT	Tên trường	Kiểu dữ liệu	Ý nghĩa
1	jti	Varchar(36) (PK)	JWT ID – định danh duy nhất của token
2	user_id	Integer	Liên kết đến tài khoản quản trị viên
3	family_id	Varchar(36)	Nhóm token (dùng cho refresh rotation)
4	token_type	Varchar(16)	Loại token (access hoặc refresh)
5	expires_at	DateTime	Thời điểm hết hạn
6	revoked	Boolean	Token đã bị thu hồi hay chưa
7	created_at	DateTime	Thời điểm cấp phát

Bảng này hỗ trợ cơ chế refresh token rotation và thu hồi token, đảm bảo phiên đăng nhập quản trị không thể bị tái sử dụng sau khi đăng xuất hoặc bị thu hồi.

3.3.2.2. Cơ sở dữ liệu vector (Qdrant)

Toàn bộ tài liệu nghiệp vụ của hệ thống được lưu trữ trong một collection duy nhất trên Qdrant, sử dụng mô hình embedding text-embedding-3-small của OpenAI để tạo vector 1536 chiều cho mỗi đoạn văn bản.

Cấu trúc một vector point trong collection kma_docs:

Bảng 3.15 Cấu trúc vector point trong collection kma_docs

Trường	Kiểu	Ý nghĩa
id	UUID	Định danh duy nhất, sinh tất định từ agent_id + source + chunk_index
vector	float[1536]	Vector embedding của đoạn văn bản
agent_id	string	Tác tử chuyên biệt sở hữu tài liệu (ví dụ: tuyen_sinh, lich_thi)
source	string	Tên file tài liệu gốc
page	integer	Số trang trong tài liệu gốc
text	string	Nội dung văn bản của đoạn
chunk_type	string	Loại đoạn (flat, parent, child) theo chiến lược phân đoạn

Trường agent_id là thành phần then chốt trong chiến lược phân tách tri thức: mỗi truy vấn vector đều kèm theo điều kiện lọc agent_id == <id_tac_tu>, đảm bảo mỗi tác tử chuyên biệt chỉ tìm kiếm trong phạm vi tài liệu thuộc nghiệp vụ của mình. Cách tiếp cận này vừa tăng độ chính xác vừa giảm nhiễu từ các nghiệp vụ không liên quan.

3.3.3 Thiết kế API

Hệ thống cung cấp hai nhóm API chính: nhóm API công khai phục vụ sinh viên và nhóm API quản trị yêu cầu xác thực JWT. Toàn bộ được xây dựng theo chuẩn RESTful, phản hồi định dạng JSON, riêng endpoint chat hỗ trợ thêm cơ chế SSE.

3.3.3.1. Nhóm 1: API quản lý phiên hội thoại

Bảng 3.16 API tạo phiên hội thoại mới

Thuộc tính	Mô tả
Endpoint	/session/new
Method	POST
Tham số	Không có
Kết quả	session_id (string) – định danh phiên hội thoại mới

Bảng 3.17 API đóng phiên hội thoại

Thuộc tính	Mô tả
Endpoint	/session/close
Method	POST
Tham số	session_id (string) – định danh phiên cần đóng
Kết quả	ok (boolean), session_id (string) – xác nhận đóng và flush phiên xuống PostgreSQL

3.3.3.2. Nhóm 2: API hội thoại chatbot

Bảng 3.18 API gửi câu hỏi với phản hồi đầy đủ

Thuộc tính	Mô tả
Endpoint	/chat
Method	POST
Tham số	question (string) – nội dung câu hỏi; history (array) – lịch sử hội thoại; session_id (string) – định danh phiên
Kết quả	answer (string) – câu trả lời; agents_used (array) – danh sách tác tử đã xử lý; pipeline (string) – pipeline đã dùng; qc (float) – điểm phức tạp; sources (array) – tài liệu tham chiếu; t_total (float) – thời gian xử lý; was_rewritten (boolean); planner_used (boolean); sub_questions (array)

Bảng 3.19 API gửi câu hỏi với phản hồi streaming SSE

Thuộc tính	Mô tả
Endpoint	/chat/stream
Method	POST
Tham số	question (string); history (array); session_id (string)
Kết quả	Luồng SSE phát từng event: {"type": "token", "content": "..."} trong quá trình sinh; {"type": "done", ...} kèm metadata đầy đủ khi hoàn thành; kết thúc bằng [DONE]

Bảng 3.20 API tải biểu mẫu đã điền

Thuộc tính	Mô tả
Endpoint	/forms/filled/{download_id}/{filename}
Method	GET
Tham số	download_id (path) – mã tải xuống; filename (path) – tên file
Kết quả	File biểu mẫu dạng application/octet-stream

3.3.3.3. Nhóm 3: API xác thực quản trị viên

Bảng 3.21 API đăng nhập bước 1 – xác thực mật khẩu

Thuộc tính	Mô tả
Endpoint	/auth/login
Method	POST
Tham số	username (string); password (string)
Kết quả	requires_otp (boolean); message (string); email_hint (string) – địa chỉ email đã che một phần

Bảng 3.22 API xác nhận OTP bước 2 – nhận JWT

Thuộc tính	Mô tả
Endpoint	/auth/verify-otp
Method	POST
Tham số	username (string); otp (string) – mã OTP 6 chữ số
Kết quả	access_token (string); refresh_token (string); role (string)

Bảng 3.23 API đổi mật khẩu

Thuộc tính	Mô tả
Endpoint	/auth/change-password
Method	POST
Yêu cầu	JWT hợp lệ trong header Authorization
Tham số	old_password (string); new_password (string)
Kết quả	message (string) – xác nhận đổi mật khẩu thành công

3.3.3.4. Nhóm 4: API quản lý tài liệu nghiệp vụ

Bảng 3.24 API lấy danh sách tài liệu

Thuộc tính	Mô tả
Endpoint	/admin/documents
Method	GET
Yêu cầu	JWT hợp lệ (role: DEAN)
Tham số	agent_id (string, tùy chọn) – lọc theo tác tử
Kết quả	documents (array) – danh sách tài liệu gồm: agent_id, filename, chunk_count, status, uploaded_by, updated_at; total (integer)

Bảng 3.25 API upload tài liệu

Thuộc tính	Mô tả
Endpoint	/admin/documents/upload
Method	POST
Yêu cầu	JWT hợp lệ (role: DEAN); multipart/form-data
Tham số	agent_id (string) – tác tử đích; file (file) – file PDF; overwrite (string) – cho phép ghi đè
Kết quả	ok (boolean); chunk_count (integer); download_url (string)

Bảng 3.26 API xóa tài liệu

Thuộc tính	Mô tả
Endpoint	/admin/documents/{agent_id}/{filename}
Method	DELETE
Yêu cầu	JWT hợp lệ (role: DEAN)
Tham số	agent_id (path); filename (path)
Kết quả	ok (boolean); deleted (string) – source_key của tài liệu đã xóa

Bảng 3.27 API re-index tài liệu theo tác tử

Thuộc tính	Mô tả
Endpoint	/admin/documents/reindex/{agent_id}
Method	POST
Yêu cầu	JWT hợp lệ (role: DEAN)

Tham số	agent_id (path) – tác tử cần re-index
Kết quả	ok (boolean); chunks_indexed (integer)

3.3.3.5. Nhóm 5: API quản lý tin tức thông báo

Bảng 3.28 API upload tin tức

Thuộc tính	Mô tả
Endpoint	/admin/news/upload
Method	POST
Yêu cầu	JWT hợp lệ (role: DEAN); multipart/form-data
Tham số	file (file) – file PDF tin tức; title (string); summary (string); overwrite (string)
Kết quả	ok (boolean); item (object) – thông tin tin tức vừa đăng

Bảng 3.29 API xóa tin tức

Thuộc tính	Mô tả
Endpoint	/admin/news/{filename}
Method	DELETE
Yêu cầu	JWT hợp lệ (role: DEAN)
Tham số	filename (path) – tên file tin tức
Kết quả	ok (boolean); deleted (string)

3.4 Tổng kết chương 3

Chương 3 đã trình bày toàn bộ quá trình phân tích và thiết kế hệ thống trợ lý ảo AI đa tác tử hỗ trợ sinh viên Học viện Kỹ thuật Mật mã. Phần yêu cầu hệ thống xác định đầy đủ các yêu cầu về hình thức sản phẩm, yêu cầu chức năng cho hai nhóm tác nhân là sinh viên và quản trị viên, cùng các yêu cầu phi chức năng về hiệu năng, độ chính xác, bảo mật và tính ổn định. Phần phân tích hệ thống xây dựng biểu đồ usecase tổng quát, đặc tả chi tiết tám usecase chính và năm biểu đồ tuần tự thể hiện các luồng xử lý quan trọng của hệ thống. Phần thiết kế hệ thống đề xuất kiến trúc phân lớp gồm bốn lớp, thiết kế sáu bảng cơ sở dữ liệu quan hệ PostgreSQL, cấu trúc collection vector Qdrant và mười bốn API endpoint được phân nhóm theo chức năng. Các kết quả phân tích và thiết kế trong chương này là cơ sở trực tiếp cho quá trình xây dựng và triển khai hệ thống được trình bày trong chương tiếp theo.

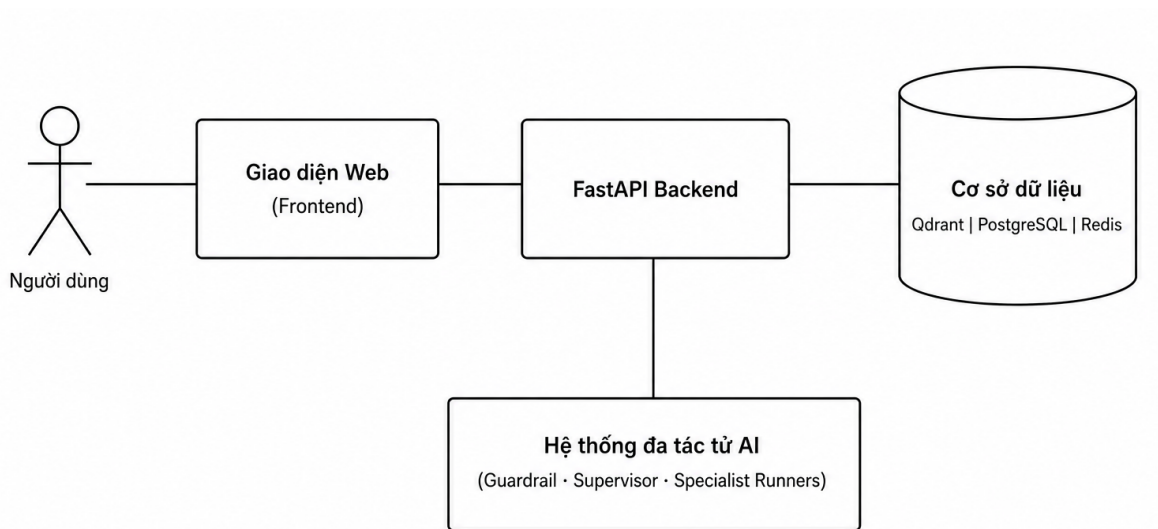
CHƯƠNG 4: XÂY DỰNG HỆ THỐNG VÀ THỰC NGHIỆM

4.1 Mô hình tổng quan của hệ thống

Hệ thống có kiến trúc phân tán bao gồm các thành phần:

- Giao diện web.
- Hệ thống Backend.
- Hệ thống đa tác tử AI.
- Cơ sở dữ liệu.

Tổ chức của các thành phần được mô tả như Hình 4.1.



Hình 4.1 Kiến trúc hệ thống

Kiến trúc của hệ thống cho phép tách biệt các thành phần Backend phục vụ, cơ sở dữ liệu, hệ thống đa tác tử AI và giao diện frontend. Kết nối các thành phần của hệ thống là các API theo chuẩn RESTful và cơ chế phát trực tuyến Server-Sent Events.

4.2 Các chức năng của hệ thống

4.2.1 Trang chủ và tin tức thông báo

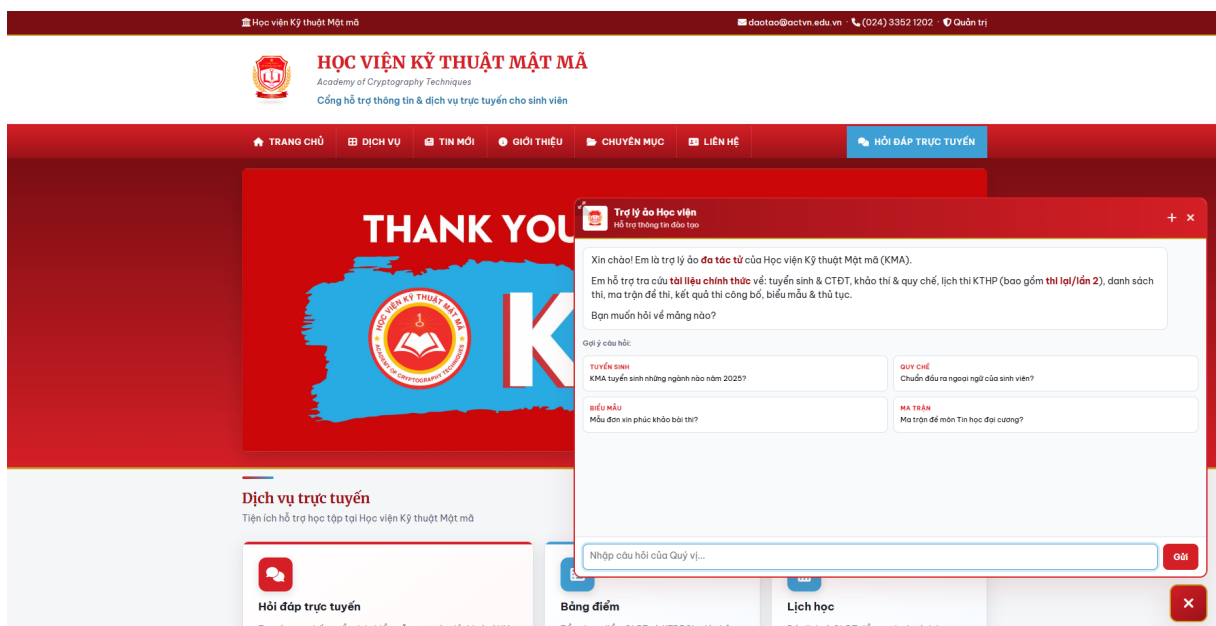
Khi người dùng truy cập vào trang chủ của hệ thống, giao diện hiển thị thanh điều hướng, khu vực giới thiệu hệ thống và danh sách tin tức thông báo mới nhất được cập nhật bởi quản trị viên. Góc dưới bên phải trang chủ hiển thị nút mở cửa sổ hội thoại với chatbot AI.



Hình 4.2 Trang chủ và danh sách tin tức thông báo

4.2.2 Giao diện hội thoại với chatbot AI

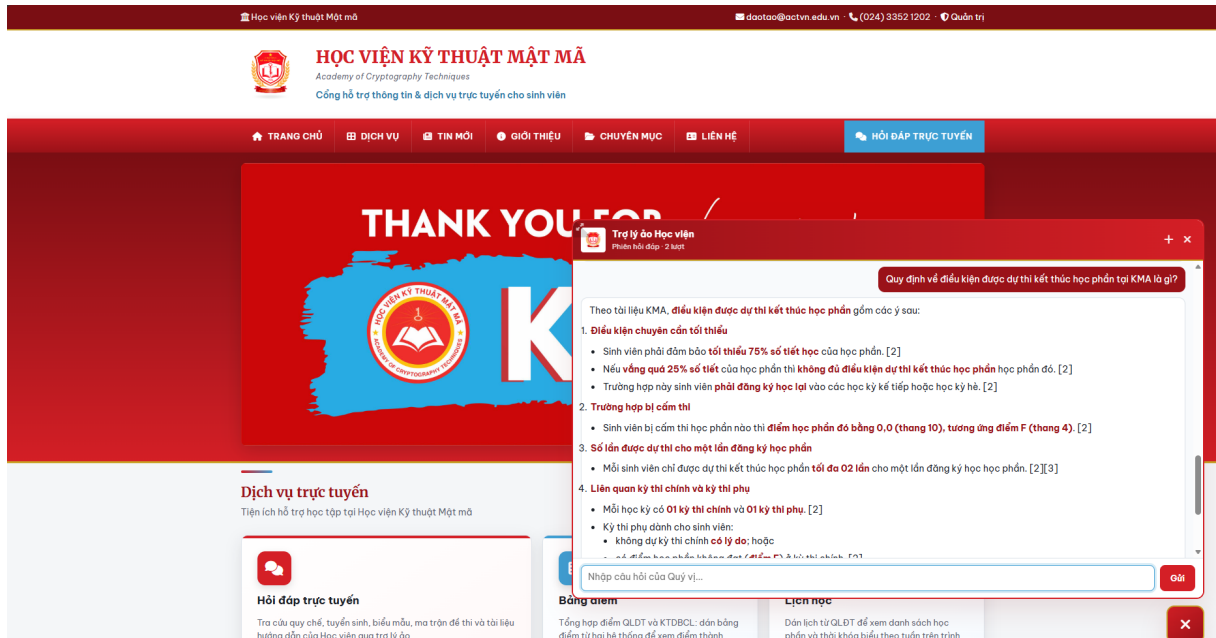
Khi người dùng bấm vào nút mở chatbot, giao diện hội thoại hiển thị cho phép người dùng nhập câu hỏi và nhận phản hồi theo thời gian thực. Câu trả lời được phát trực tuyến từng token, tạo trải nghiệm tự nhiên tương tự các hệ thống chatbot hiện đại. Phía dưới mỗi câu trả lời, hệ thống hiển thị danh sách tài liệu nguồn được truy xuất kèm tên file và số trang, giúp người dùng kiểm chứng thông tin.



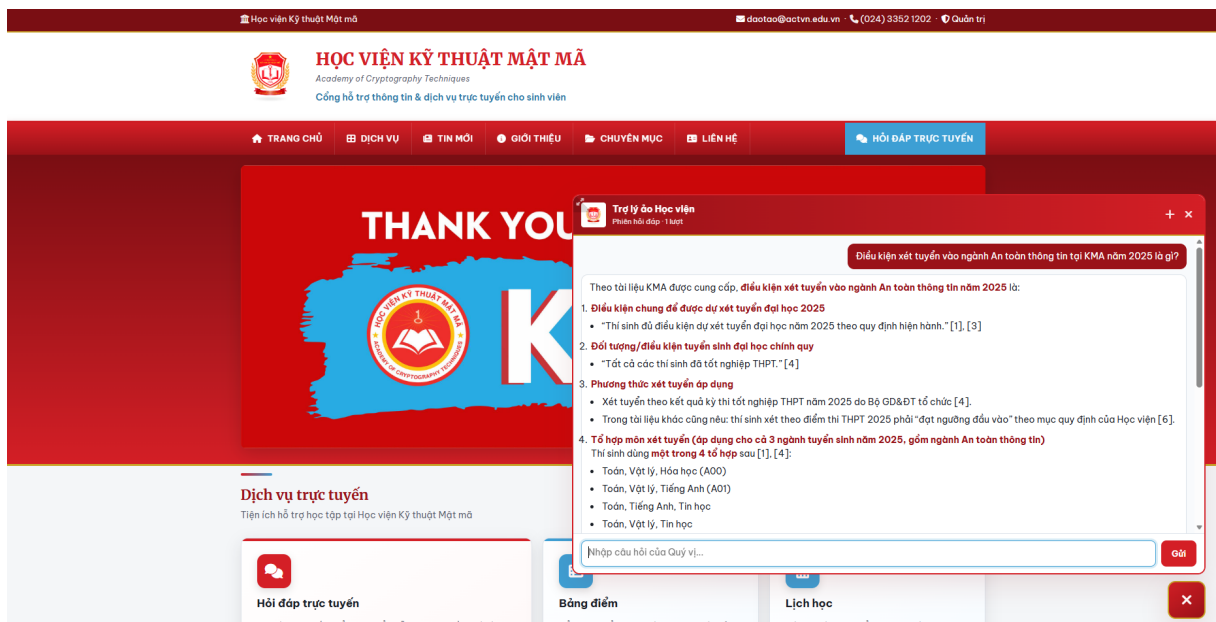
Hình 4.3 Giao diện hội thoại với chatbot AI

4.2.2.1. Chức năng hỏi đáp thông tin nghiệp vụ

Người dùng có thể đặt câu hỏi liên quan đến các nghiệp vụ của Học viện như tuyển sinh, chương trình đào tạo, quy chế đào tạo, ma trận đề thi. Hệ thống tự động phân tích câu hỏi, định tuyến đến tác tử chuyên biệt phù hợp và trả về câu trả lời có căn cứ từ tài liệu chính thức.



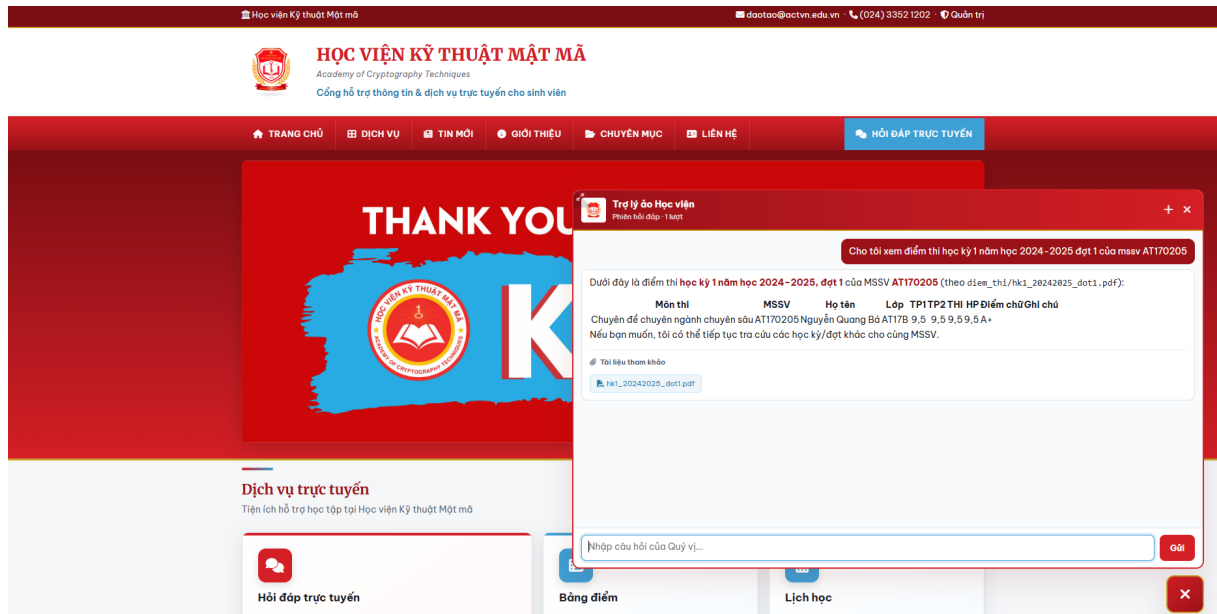
Hình 4.4 Hỏi đáp thông tin nghiệp vụ trên giao diện chatbot



Hình 4.5 Kết quả trả lời có kèm tài liệu nguồn tham chiếu

4.2.2.2. Chức năng tra cứu điểm thi theo mã số sinh viên

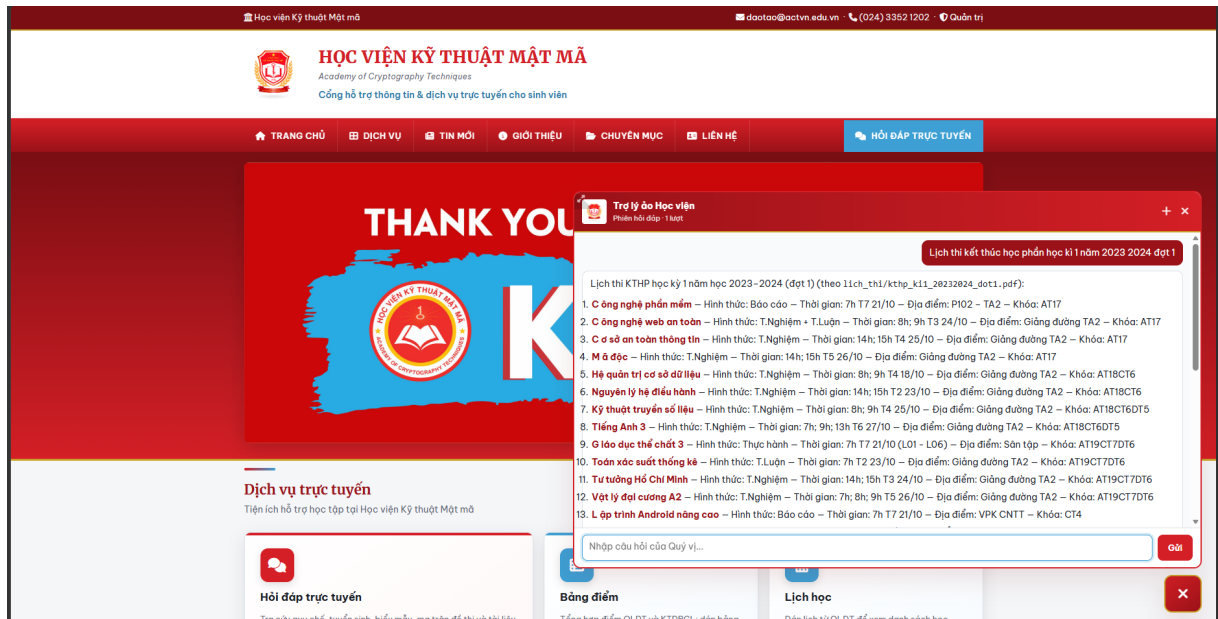
Khi sinh viên cung cấp mã số sinh viên kèm yêu cầu tra cứu điểm, hệ thống tự động trích xuất MSSV, tìm kiếm trong tập bảng điểm đã công bố và tổng hợp kết quả bao gồm tên môn học, điểm thành phần và điểm tổng kết. Sinh viên có thể chỉ định học kỳ cụ thể hoặc yêu cầu tra cứu toàn bộ.



Hình 4.6 Tra cứu điểm thi theo mã số sinh viên

4.2.2.3. Chức năng tra cứu lịch thi và danh sách dự thi

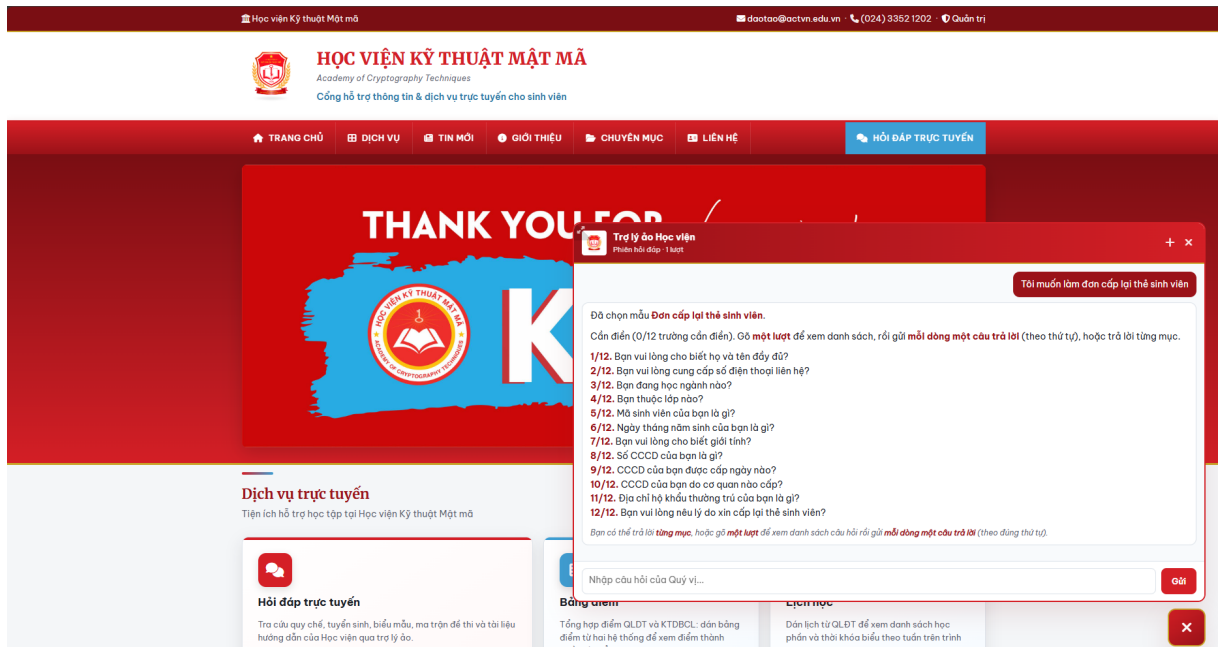
Sinh viên có thể hỏi về lịch thi học phần theo học kỳ, đợt thi hoặc tên môn học. Hệ thống trả về thông tin ngày thi, ca thi và danh sách sinh viên dự thi theo phòng thi, buổi thi được công bố.



Hình 4.7 Tra cứu lịch thi và danh sách dự thi

4.2.2.4. Chức năng hỏi biểu mẫu và thủ tục hành chính

Sinh viên có thể hỏi về các biểu mẫu và thủ tục hành chính như đơn phúc khảo, nghỉ học, thực tập, ra trường. Hệ thống hướng dẫn thủ tục và cung cấp đường dẫn tải biểu mẫu tương ứng.



Hình 4.8 Hỏi biểu mẫu và thủ tục hành chính

4.2.2.5. Chức năng xử lý câu hỏi đa ý định

Khi sinh viên đặt một câu hỏi chứa nhiều nội dung liên quan đến các nghiệp vụ khác nhau, hệ thống tự động phân tách thành các câu hỏi con, xử lý song song qua các

tác tử chuyên biệt và tổng hợp thành một câu trả lời đầy đủ, thống nhất.

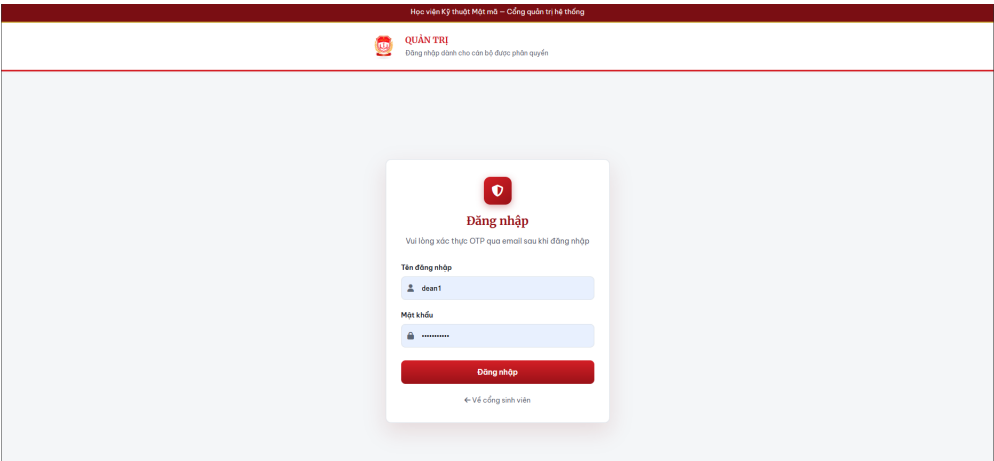


Hình 4.9 Xử lý câu hỏi đa ý định

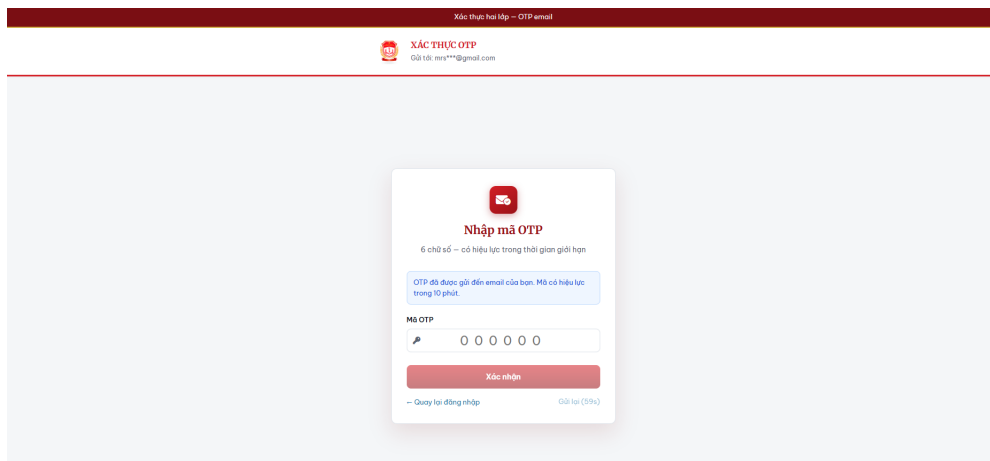
4.2.3 Giao diện quản trị

4.2.3.1. Chức năng đăng nhập hệ thống quản trị

Quản trị viên truy cập trang đăng nhập, nhập tên đăng nhập và mật khẩu. Sau khi xác thực mật khẩu thành công, hệ thống gửi mã OTP đến địa chỉ email đã đăng ký. Quản trị viên nhập mã OTP để hoàn tất đăng nhập và được chuyển vào trang dashboard quản trị.



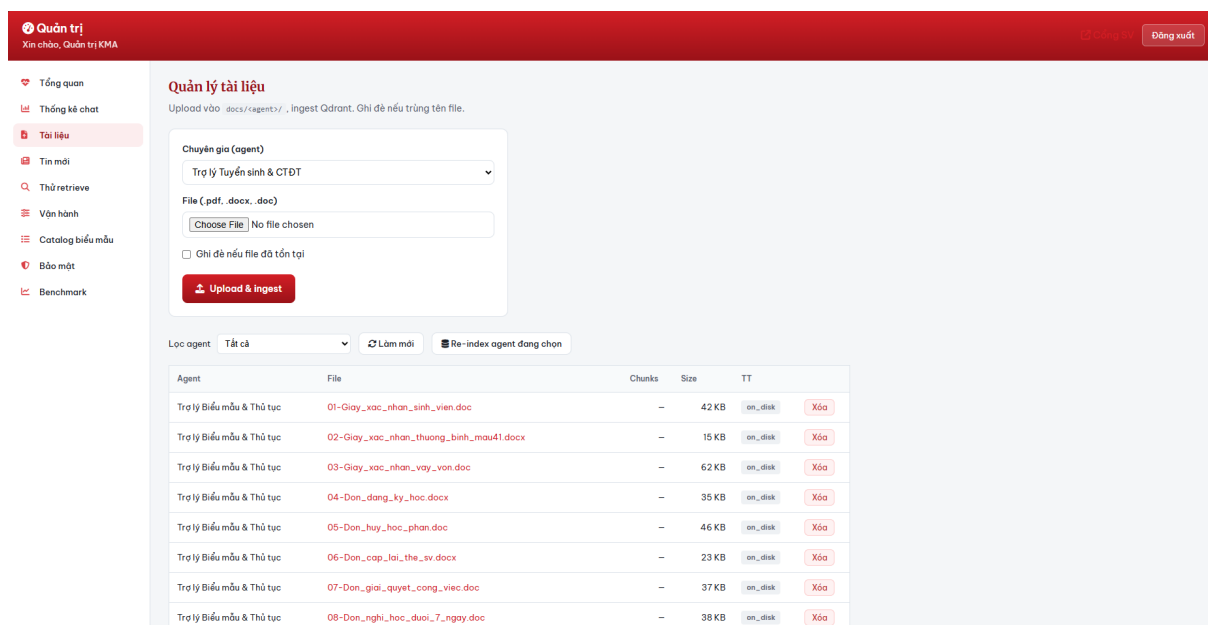
Hình 4.10 Giao diện đăng nhập hệ thống quản trị



Hình 4.11 Giao diện xác thực OTP khi đăng nhập quản trị

4.2.3.2. Chức năng quản lý tài liệu nghiệp vụ

Sau khi đăng nhập, quản trị viên truy cập trang quản lý tài liệu. Giao diện hiển thị danh sách toàn bộ tài liệu đã nạp phân theo từng tác tử chuyên biệt, kèm thông tin tên file, số chunk, trạng thái nạp và người upload. Quản trị viên có thể tải lên tài liệu mới, xóa tài liệu hoặc yêu cầu re-index lại toàn bộ tài liệu của một tác tử.



Hình 4.12 Giao diện quản lý tài liệu nghiệp vụ

4.2.3.3. Chức năng quản lý tin tức thông báo

Quản trị viên có thể đăng tải tin tức thông báo mới dưới dạng file PDF với tiêu đề và mô tả ngắn. Tin tức sau khi đăng sẽ hiển thị công khai trên trang chủ dành cho sinh viên. Quản trị viên cũng có thể xóa các tin tức đã đăng khi không còn cần thiết.

Quản trị

Xin chào, Quản trị KMA

Đăng xuất

Tổng quan

Thống kê chat

Tài liệu

Tin mới

Thủ retrieve

Vấn hành

Catalog biểu mẫu

Báo một

Benchmark

Tin mới trang chủ

Upload PDF thông báo từ Phòng Đào tạo để hiển thị danh sách ở trang chủ.

Tiêu đề tin

Ví dụ: Lịch thi KTHP học kỳ 1 năm học 2024-2025 đợt 1

Mô tả ngắn

Tóm tắt 1-2 dòng để sinh viên dễ nhận biết

File PDF

Choose File | No file chosen

☐ Ghi đè nếu file đã tồn tại

Đăng tin

Làm mới

Tiêu đề	File	Cập nhật	
Quyết định tốt nghiệp ĐHCQ Xét tháng 3 năm 2026	3. QĐ TỐT NGHIỆP ĐHCQ hệ ĐT phục vụ KT-XH (ĐỢT XÉT T3.26)-BAN HÀNH.pdf	2026-05-27 15:06	Xóa
Thông báo thi sát hạch Tiếng Anh đầu ra đợt 2 năm 2026 Dành cho sinh viên sắp ra trường, có hạn 24 tháng	Kế hoạch thi SHDR ngoại ngữ 2026. Đợt 2.pdf	2026-05-27 15:04	Xóa

Hình 4.13 Giao diện quản lý tin tức thông báo

4.2.4 Giao diện tra cứu bảng điểm

Ngoài kênh hỏi đáp tự nhiên, hệ thống cung cấp một trang tra cứu bảng điểm riêng biệt nhằm hỗ trợ người dùng tổng hợp và theo dõi kết quả học tập một cách trực quan. Giao diện được thiết kế theo phong cách nhận diện thương hiệu của Học viện, tích hợp dữ liệu từ hai nguồn là hệ thống Quản lý đào tạo (QLĐT) và hệ thống Khảo thí – Đảm bảo chất lượng (KTDBCL).

Sau khi dữ liệu điểm được nạp, giao diện tự động tổng hợp và hiển thị các thông tin bao gồm:

- Bảng điểm tổng hợp: liệt kê từng học phần theo học kỳ, hiển thị điểm quá trình, điểm thi, điểm tổng kết và số tín chỉ tương ứng; áp dụng màu sắc để phân biệt trạng thái đạt hoặc không đạt.
- Chỉ số GPA: tính toán GPA theo thang 10 và thang 4 dựa trên toàn bộ kết quả đã có, với chỉ số được làm nổi bật để người dùng dễ nhận biết.
- Công cụ mô phỏng cải thiện điểm: cho phép người dùng điều chỉnh giả định điểm của một số học phần thông qua thanh trượt, hệ thống lập tức tính toán lại GPA kỳ vọng và gợi ý các phương án cải thiện khả thi theo hai mức độ tích cực và vừa phải.

Giao diện hướng đến mục tiêu hỗ trợ người dùng lập kế hoạch học tập chủ động, thay vì chỉ phản ánh thụ động kết quả hiện có.

GPA dự kiến

3.21

Tổng tín chỉ tích lũy

110

Chỉ tính GPA cho các môn đã chọn

GDTC và TH Vật lý không tính vào GPA

Bảng điểm tổng hợp

Xuất Excel

+ Thêm môn

	MÔN HỌC	TÍN CHỈ	LẦN THI	GK (TP1)	CC (TP2)	ĐIỂM CK	ĐIỂM TK	CHỮ	XÓA
	An toàn và bảo mật trong phát triển phần	3	1	9.5	7	9.2	9.1	A+	
	Công nghệ phần mềm nhúng	3	1	7.5	9	7.3	7.5	B	
	Cơ sở an toàn và bảo mật thông tin	3	1	8.5	9	8.5	8.6	A	
	Hệ điều hành nhúng thời gian thực	3	1	8.4	8	8.3	8.3	B+	
	Hệ thống thông tin di động	2	1	10	9.5	8.8	9.1	A+	
	Kiểm thử phần mềm nhúng	2	2	7.5	10	5	6	C	
	Lập trình Android cơ bản	3	1	8	8	8	8	B+	

↓ Cuộn trong khung bảng điểm

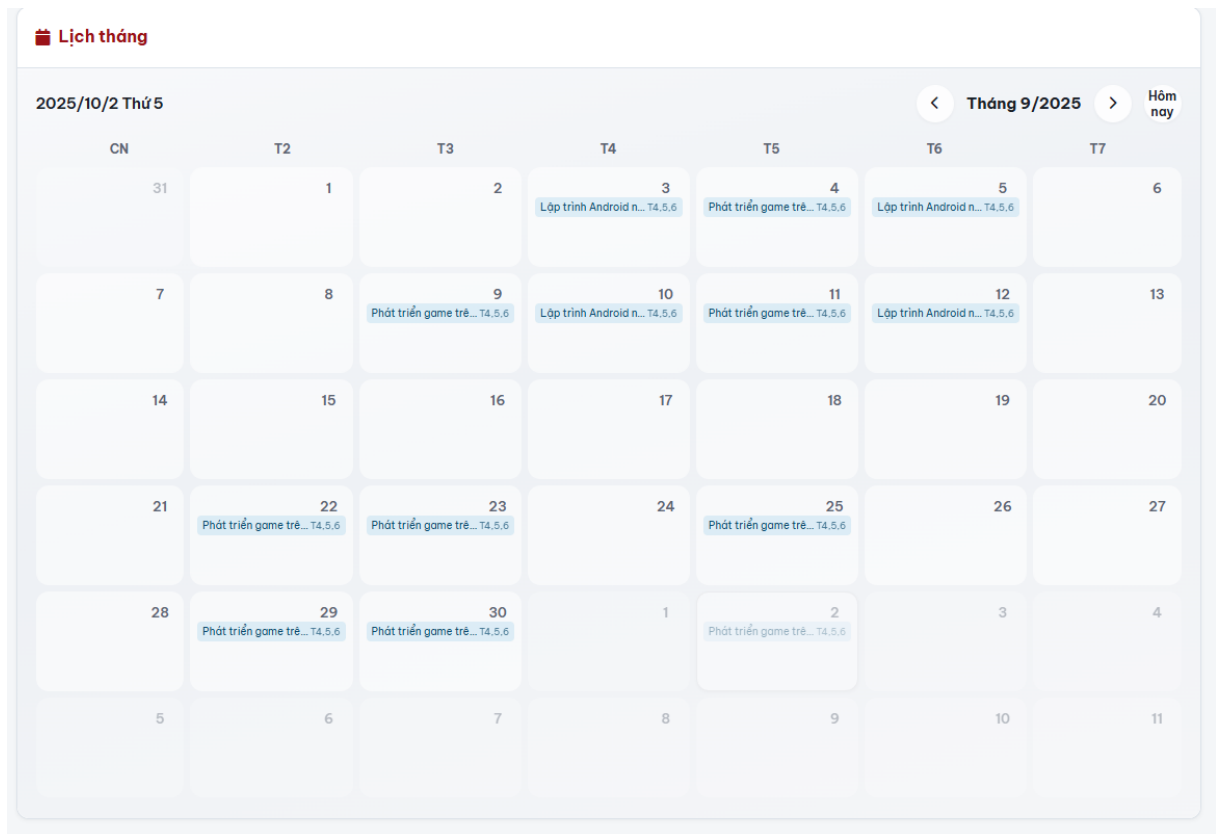
Hình 4.14 Giao diện tra cứu bảng điểm và mô phỏng cải thiện GPA

4.2.5 Giao diện theo dõi thời khóa biểu

Hệ thống cung cấp trang theo dõi thời khóa biểu học kỳ, giúp người dùng nắm bắt lịch học một cách hệ thống và thuận tiện. Giao diện kết nối với dữ liệu thời khóa biểu từ hệ thống QLĐT và trình bày theo hai chế độ song song:

- Danh sách học phần: trình bày dạng bảng với các cột học phần, lịch học tóm tắt, địa điểm, giảng viên và số tín chỉ; kèm thống kê tổng số học phần và tổng tín chỉ đăng ký trong học kỳ.
- Lịch tháng tương tác: hiển thị toàn bộ buổi học theo dạng lịch tháng; mỗi ô ngày liệt kê các môn học trong ngày đó kèm giờ bắt đầu; người dùng có thể điều hướng qua lại giữa các tháng và nhấn vào từng ô để xem chi tiết buổi học.

Hai chế độ hiển thị trên được thiết kế để bổ trợ nhau: chế độ danh sách phục vụ tra cứu nhanh theo tên môn hoặc giảng viên, trong khi chế độ lịch tháng cung cấp cái nhìn tổng thể về phân bổ thời gian học trong tuần và tháng.



Hình 4.15 Giao diện theo dõi thời khóa biểu học kỳ

4.3 Đánh giá

4.3.1 Mô tả bộ dữ liệu thực nghiệm

4.3.1.1. Cấu trúc bộ dữ liệu

Nhằm đánh giá hệ thống đa tác tử đã xây dựng, quá trình thực nghiệm được tiến hành trên một tập dữ liệu được chuẩn bị trước. Bộ dữ liệu sử dụng trong quá trình thực nghiệm bao gồm các truy vấn mẫu được xây dựng dựa trên các tình huống thực tế mà người dùng có thể gặp khi tìm kiếm thông tin học vụ tại Học viện Kỹ thuật Mật mã. Do đặc trưng của hệ thống đa tác tử, bộ dữ liệu được chia thành hai nhóm chính:

- Nhóm câu hỏi có đầu ra xác định (N1): câu hỏi tra cứu thông tin cụ thể, có kết quả duy nhất và kiểm chứng được bằng đối chiếu trực tiếp. Ví dụ câu hỏi về điểm thi theo mã sinh viên, điểm chuẩn tuyển sinh, tên file biểu mẫu, số điện thoại liên hệ, điểm TOEIC theo quy định.
- Nhóm câu hỏi có đầu ra mở (N2): câu hỏi yêu cầu tổng hợp, giải thích hoặc so sánh, kết quả đầu ra có thể được diễn đạt theo nhiều văn phong khác nhau. Ví dụ câu hỏi về quy trình thủ tục hành chính, điều kiện được dự thi, so sánh hai loại đơn từ, hướng dẫn lộ trình chuẩn tiếng Anh.

Đối với nhóm N1, phép đo Accuracy có thể thực hiện tự động bằng đối chiếu chuỗi. Đối với nhóm N2, cần bổ sung phép đo Context Relevance để đánh giá mức độ bám sát

tài liệu gốc của câu trả lời.

Mỗi dòng trong bộ dữ liệu thực nghiệm bao gồm các trường thông tin được biểu diễn như Bảng 4.1.

Bảng 4.1 Bảng mô tả ý nghĩa các trường của bộ dữ liệu thực nghiệm

Tên trường	Ý nghĩa
Query	Câu hỏi đầu vào cho hệ thống.
Expect	Thông tin bắt buộc phải xuất hiện trong câu trả lời đúng.
Context	Đoạn văn bản gốc trích từ tài liệu KMA liên quan đến câu hỏi. Đối với nhóm N2, trường này bắt buộc để thực hiện phép đo Context Relevance. Đối với nhóm N1, trường này không bắt buộc.

4.3.1.2. Chuẩn bị dữ liệu

Để thực hiện đánh giá hệ thống, bộ dữ liệu thực nghiệm được chuẩn bị gồm 100 câu hỏi, trong đó 55 câu thuộc nhóm N1 và 45 câu thuộc nhóm N2. Các câu hỏi được thiết kế với độ phức tạp tăng dần, từ những câu hỏi đơn giản có thể thực hiện với một bước truy xuất như câu hỏi về điểm thi theo mã sinh viên, tên file biểu mẫu, cho đến những câu hỏi cần Planner phân rã thành nhiều truy vấn con và kích hoạt đồng thời nhiều tác tử chuyên biệt.

Bảng 4.2 và Bảng 4.3 thể hiện ví dụ về một số câu hỏi trong bộ dữ liệu thực nghiệm.

Bảng 4.2 Một số câu hỏi nhóm N1 trong bộ dữ liệu thực nghiệm

STT	Query	Expect	Context
1	Mã trường KMA trong đề án tuyển sinh 2025 là gì?	KMA	
2	Sinh viên cần đạt tối thiểu bao nhiêu điểm TOEIC khi kết thúc Tiếng Anh 1?	300	
3	Môn Tin học đại cương có bao nhiêu câu trắc nghiệm theo ma trận đề thi?	50	
4	Tổng số tiền phải nộp khi làm thủ tục nhập học KMA 2024?	10.896.940	
5	CT060310 điểm học kỳ 2 năm 2024–2025 đợt 1	CT060310, có điểm từng môn	
6	Sinh viên AT200201 có đạt phân loại tiếng Anh đầu vào A20C8D7 2024 lần 2 không?	ĐẠT, AT200201	

Bảng 4.3 Một số câu hỏi nhóm N2 trong bộ dữ liệu thực nghiệm

Câu hỏi số 1	
Query	Quy định về điều kiện được dự thi kết thúc học phần tại KMA.
Expect	Sinh viên phải tham dự ít nhất 80% số giờ học lý thuyết; không bị đình chỉ học tập hoặc đình chỉ thi; hoàn thành các yêu cầu thực hành, thí nghiệm của học phần.
Context	Trích điều khoản trong Quy chế đào tạo đại học KMA 2025 về điều kiện dự thi kết thúc học phần.
Câu hỏi số 2	
Query	So sánh điểm trúng tuyển ngành An toàn thông tin cơ sở Hà Nội năm 2023 và 2024.
Expect	25,60 (năm 2023); 25,90 (năm 2024); tăng 0,30 điểm.
Context	Trích bảng số liệu Đề án tuyển sinh 2024, dòng An toàn thông tin cơ sở Hà Nội.

4.3.2 Kịch bản thực nghiệm

Việc đo lường các chỉ số là quan trọng để đánh giá mức độ hiệu quả của hệ thống đa tác tử. Việc đánh giá tập trung vào chất lượng và độ chính xác của hệ thống AI cốt lõi, không tính đến các yếu tố giao diện người dùng. Phần thực nghiệm tập trung đo lường nhằm xác định các tiêu chí đánh giá sau đây:

- Accuracy.
- Recall.
- Context Relevance.
- Latency.

4.3.2.1. Accuracy

Accuracy hay độ chính xác là tiêu chí cốt lõi phản ánh tính đúng đắn của câu trả lời. Với nhóm N1, độ chính xác được đo bằng việc kiểm tra sự xuất hiện của các thông tin bắt buộc trong trường Expect ở câu trả lời của hệ thống. Độ chính xác tổng thể là tỉ lệ phần trăm giữa số câu trả lời đúng trên tổng số câu truy vấn đã thực hiện, được tính theo công thức:

$$Accuracy = \frac{\text{Số câu trả lời đúng}}{\text{Tổng số câu truy vấn}} \times 100\%$$

Với mỗi câu truy vấn, kết quả đánh giá nhị phân chỉ gồm hai trạng thái: Pass khi câu trả lời chứa đầy đủ thông tin bắt buộc và Fail khi câu trả lời thiếu hoặc sai thông tin. Quá trình kiểm tra được thực hiện bằng hai phương pháp: đối chiếu chuỗi tự động, tức

kiểm tra sự xuất hiện của các gold fact trong văn bản trả về, và đánh giá thủ công. Trong phạm vi đề tài, cả hai phương pháp đều được áp dụng để so sánh kết quả.

4.3.2.2. Recall

Recall hay độ bao phủ là chỉ số đo lường khả năng thu thập đủ các thông tin cần thiết của hệ thống. Với câu hỏi có nhiều ý, ví dụ như “Liệt kê các tổ hợp môn xét tuyển KMA 2025 và điều kiện dự thi kết thúc học phần”, cần kiểm tra hệ thống có trả lời đầy đủ các điểm quan trọng hay bỏ sót không.

Recall được tính bằng tỷ số lượng thông tin bắt buộc xuất hiện trong câu trả lời so với tổng lượng thông tin bắt buộc cần có, theo công thức:

$$Recall = \frac{|Expect \cap Output|}{|Expect|} \times 100\%$$

4.3.2.3. Context Relevance

Context Relevance hay mức độ liên quan đến ngữ cảnh là chỉ số phản ánh mức độ bám sát tài liệu gốc của câu trả lời, áp dụng cho nhóm N2. Chỉ số này được tính bằng tỷ số lượng thông tin chung giữa câu trả lời của hệ thống và đoạn context thực tế so với lượng thông tin trong đáp án mong muốn.

$$Context\ Relevance = \frac{|Context \cap Output|}{|Expect|} \times 100\%$$

4.3.2.4. Latency

Latency là chỉ số thể hiện tốc độ phản hồi của hệ thống. Latency được tính bằng tổng thời gian hệ thống cần để xử lý một truy vấn và trả về kết quả hoàn chỉnh, tính từ thời điểm nhận câu hỏi đến khi phản hồi kết thúc. Chỉ số này được tính bằng đơn vị mili giây (ms).

4.3.3 Tiến hành thực nghiệm

Thực nghiệm được tiến hành thông qua một chương trình đánh giá tự động viết bằng ngôn ngữ lập trình Python. Mỗi câu hỏi được gửi tuần tự đến endpoint /chat của hệ thống, thu thập câu trả lời và tính toán các chỉ số. Trong đó, chỉ số Accuracy được đo đồng thời bằng chương trình tự động và đánh giá thủ công nhằm đảm bảo kết quả chính xác nhất. Chương trình thực nghiệm được xây dựng theo các bước sau:

- Bước 1: Nạp bộ dữ liệu thực nghiệm từ file benchmark.json.
- Bước 2: Với mỗi câu hỏi trong bộ dữ liệu, gửi truy vấn đến hệ thống và đo lường các chỉ số.

- Bước 3: Ghi nhận kết quả Pass/Fail, pipeline thực tế, danh sách tác tử được kích hoạt và Latency, sau đó chuyển sang câu hỏi tiếp theo.
- Bước 4: Tổng hợp kết quả đo lường trên toàn bộ tập dữ liệu.

4.3.4 Kết quả thực nghiệm

4.3.4.1. Kết quả thực nghiệm

Kết quả thực nghiệm là giá trị của các phép đo, trong đó phép đo Accuracy là phép đo tổng hợp trên toàn tập dữ liệu nên chỉ có một giá trị, các phép đo còn lại ghi nhận thêm giá trị tối thiểu và tối đa. Sau quá trình thực nghiệm, thu được các kết quả như Bảng 4.4 và Bảng 4.5.

Bảng 4.4 Kết quả thực nghiệm bộ dữ liệu N1

Chỉ số	Giá trị trung bình	Giá trị tối thiểu	Giá trị tối đa
Accuracy (đo tự động)	87%	–	–
Accuracy (đo thủ công)	93%	–	–
Recall	82,4%	45,3%	100%
Latency	35,2 giây	4,5 giây	85,0 giây

Bảng 4.5 Kết quả thực nghiệm bộ dữ liệu N2

Chỉ số	Giá trị trung bình	Giá trị tối thiểu	Giá trị tối đa
Accuracy (đo tự động)	68%	–	–
Accuracy (đo thủ công)	82%	–	–
Recall	67,8%	19,5%	94,3%
Context Relevance	73,6%	28,4%	91,5%
Latency	187,4 giây	38,5 giây	698,4 giây

4.3.4.2. Đánh giá

Từ kết quả thực nghiệm, có thể rút ra các nhận xét sau:

- Độ chính xác của nhóm N2 thấp hơn nhóm N1 do đặc tính câu trả lời có thể diễn đạt theo nhiều văn phong khác nhau, khiến phép đo tự động dựa trên đối chiếu chuỗi không bao phủ hết các trường hợp đúng.
- Sự chênh lệch đáng kể giữa Accuracy đo tự động và đo thủ công ở nhóm N2 phản ánh thực tế rằng câu trả lời đúng về nội dung nhưng được diễn đạt theo cách khác với Expect được ghi trong bộ dữ liệu.

- Tỷ lệ bao phủ (Recall) của nhóm N1 cao hơn nhóm N2 do câu trả lời nhóm N1 thường ngắn gọn và tập trung vào các con số, tên file hoặc từ khóa cụ thể để kiểm chứng.
- Context Relevance đạt 73,6% trung bình và cao nhất đạt 91,5%, cho thấy cơ chế RAG kết hợp với định tuyến theo nghiệp vụ có tác dụng cải thiện đáng kể mức độ bám sát tài liệu gốc trong câu trả lời.
- Latency của nhóm N2 cao hơn đáng kể so với N1, do các câu hỏi mở thường kích hoạt pipeline Agentic RAG hoặc cơ chế đa tác tử với nhiều vòng lặp truy xuất và tổng hợp. Các truy vấn đơn giản như Guardrail, lịch thi và danh sách thi đạt thời gian phản hồi thấp nhờ các fast-path chuyên biệt.

4.4 Tổng kết chương 4

Trong chương 4, báo cáo đã trình bày mô hình tổng quan của hệ thống trợ lý ảo đa tác tử, đồng thời giới thiệu các chức năng cụ thể của hệ thống sau khi xây dựng, bao gồm giao diện hỏi đáp hội thoại, tra cứu điểm thi theo mã sinh viên, tra cứu lịch thi và danh sách dự thi, hỗ trợ biểu mẫu thủ tục, theo dõi bảng điểm và thời khóa biểu, cùng các chức năng quản trị dành cho quản trị viên. Phần sau của chương đã trình bày phương pháp thực nghiệm với bộ dữ liệu 100 câu hỏi chia theo hai nhóm N1 và N2, kết quả đạt được trên các tiêu chí Accuracy, Recall, Context Relevance và Latency. Từ các kết quả thực nghiệm đó, chương 4 cũng rút ra các nhận xét về điểm mạnh và hạn chế của hệ thống, làm cơ sở cho phần kết luận và hướng phát triển được trình bày ở phần cuối báo cáo.

KẾT LUẬN

Đề tài “Xây dựng nhóm trợ lý ảo AI đa tác tử hỗ trợ các trường đại học” đã hoàn thành các mục tiêu cốt lõi đặt ra ở phần mở đầu: nghiên cứu, phân tích, thiết kế và triển khai thử nghiệm một hệ thống trợ lý ảo thông minh phục vụ nhu cầu tra cứu thông tin học vụ tại Học viện Kỹ thuật Mật mã, cho phép người dùng đặt câu hỏi bằng ngôn ngữ tự nhiên và nhận lại câu trả lời chính xác dựa trên tài liệu chính thức của Học viện mà không cần thao tác tìm kiếm thủ công. Tuy nhiên, do giới hạn về thời gian và phạm vi của một đồ án tốt nghiệp, hệ thống vẫn còn những điểm cần tiếp tục hoàn thiện. Phần này tổng kết một cách khách quan những kết quả đã đạt được, những hạn chế còn tồn tại và đề xuất hướng phát triển trong tương lai.

Kết quả đạt được

Về mặt nghiên cứu, đề tài đã hệ thống hóa các kiến thức nền tảng về mô hình ngôn ngữ lớn (LLM), kỹ thuật Retrieval-Augmented Generation (RAG), kiến trúc AI Agent và hệ thống đa tác tử, làm cơ sở lý thuyết cho toàn bộ quá trình thiết kế và xây dựng. Đặc biệt, đề tài đã đề xuất và triển khai cơ chế đánh giá độ phức tạp truy vấn (Query Complexity – QC) với công thức kết hợp bốn thành phần Entity, Relation, Length và Semantic Ambiguity, cho phép mỗi tác tử chuyên biệt tự động lựa chọn pipeline xử lý phù hợp thay vì áp dụng một pipeline cố định cho mọi loại câu hỏi.

Về mặt sản phẩm, hệ thống đã được xây dựng hoàn chỉnh với kiến trúc đa tác tử, gồm các thành phần phối hợp chặt chẽ. Các kết quả nổi bật bao gồm:

- Cơ chế điều phối sáu tầng: Guardrail lọc câu hỏi ngoài phạm vi, Query Rewriter làm rõ ngữ cảnh hội thoại, Planner phân rã câu hỏi đa ý định, Supervisor định tuyến theo nghiệp vụ, Router lựa chọn pipeline theo QC và Aggregator tổng hợp kết quả từ nhiều tác tử, tạo thành luồng xử lý tự động, linh hoạt và chính xác.
- Bảy tác tử chuyên biệt: tuyển sinh, khảo thí, ma trận đề thi, kết quả học tập, biểu mẫu, danh sách thi và lịch thi. Mỗi tác tử vận hành độc lập trên kho tài liệu riêng và có thể được kích hoạt đồng thời song song khi câu hỏi thuộc nhiều nghiệp vụ.
- Ba pipeline RAG phân tầng: Native RAG cho câu hỏi đơn giản, Hybrid RAG cho câu hỏi trung bình và Agentic RAG với cơ chế lặp truy xuất, đánh giá, mở rộng truy vấn cho các câu hỏi phức tạp; kết hợp với các fast-path chuyên biệt cho tra cứu điểm thi theo MSSV, lịch thi và danh sách dự thi.
- Quản lý phiên hội thoại hai tầng: Redis lưu trữ hot session cho các phiên đang hoạt động, PostgreSQL lưu trữ cold archive cho lịch sử hội thoại dài hạn; kết hợp cơ chế tóm tắt phiên và cache kết quả truy vấn nhằm giảm thiểu chi phí gọi API.

- Giao diện đa chức năng: cổng hỏi đáp hội thoại tích hợp streaming, trang tra cứu bảng điểm với mô phỏng cải thiện GPA, trang theo dõi thời khóa biểu dạng lịch tháng và hệ thống quản trị nội dung cho quản trị viên.
- Bộ kiểm thử 100 case: phân tầng từ L0 đến L6 theo độ phức tạp tăng dần, với kết quả Accuracy đo thử công đạt 93% trên nhóm câu hỏi xác định và 82% trên nhóm câu hỏi mở.

Hệ thống đã chứng minh tính khả thi của việc kết hợp kiến trúc đa tác tử, cơ chế đánh giá độ phức tạp truy vấn và kho tài liệu nghiệp vụ chuyên biệt cho bài toán hỗ trợ thông tin học vụ đại học.

Hạn chế còn tồn tại

Qua quá trình thử nghiệm, hệ thống vẫn bộc lộ một số hạn chế cần được nhìn nhận khách quan:

- Độ trễ còn cao ở truy vấn phức tạp: do pipeline Agentic RAG thực hiện nhiều vòng lặp truy xuất và tác tử đa nghiệp vụ gọi API LLM nhiều lần, thời gian phản hồi cho các câu hỏi phức tạp có thể lên tới vài trăm giây, ảnh hưởng đến trải nghiệm người dùng.
- Phạm vi dữ liệu giới hạn trong tài liệu tĩnh: hệ thống hoạt động dựa trên tập tài liệu được nạp trước; khi Học viện cập nhật quy chế, lịch thi hoặc danh sách điểm mới, quản trị viên phải thực hiện thao tác nạp lại thủ công, chưa có cơ chế đồng bộ tự động.
- Chất lượng câu trả lời phụ thuộc vào chất lượng tài liệu gốc: nếu tài liệu được nạp vào có lỗi định dạng hoặc thông tin không nhất quán, câu trả lời của hệ thống cũng bị ảnh hưởng tương ứng.
- Hội thoại nhiều lượt phức tạp còn sai sót: độ chính xác ở tier L5, tức hội thoại nhiều lượt, thấp hơn các tier khác, đặc biệt khi người dùng đổi chủ đề đột ngột hoặc sử dụng đại từ chỉ thị mơ hồ giữa các lượt hội thoại.
- Chưa có cơ chế phản hồi người dùng để cải thiện liên tục: hệ thống chưa thu thập đánh giá của người dùng về chất lượng câu trả lời, do đó chưa có vòng lặp cải thiện tự động dựa trên phản hồi thực tế.

Hướng phát triển

Trên cơ sở các hạn chế nêu trên, đề tài đề xuất các hướng phát triển trong tương lai:

- Tối ưu hiệu năng và độ trễ: áp dụng cache kết quả ở tầng Supervisor, rút gọn số vòng lặp Agentic RAG cho các câu hỏi ở ngưỡng biên, streaming câu trả lời theo từng token để giảm thời gian chờ cảm nhận của người dùng.
- Tích hợp nguồn dữ liệu động: xây dựng cơ chế webhook hoặc cronjob tự động nạp

lại khi tài liệu được cập nhật, đồng thời kết nối trực tiếp với hệ thống quản lý đào tạo để lấy dữ liệu điểm thi và lịch thi theo thời gian thực.

- Mở rộng sang các trường đại học khác: kiến trúc tác tử và cơ chế định tuyến đã được thiết kế theo hướng có thể cấu hình; việc triển khai cho trường mới chủ yếu chỉ cần cung cấp tập tài liệu nghiệp vụ tương ứng và điều chỉnh danh sách tác tử trong file cấu hình.
- Nâng cao chất lượng hội thoại nhiều lượt: cải tiến cơ chế Query Rewriter với ngữ cảnh hội thoại sâu hơn, bổ sung phát hiện chủ đề để xử lý chính xác hơn các tình huống chuyển chủ đề giữa các lượt.
- Xây dựng vòng lặp cải thiện liên tục: tích hợp tính năng đánh giá câu trả lời của người dùng, sử dụng dữ liệu phản hồi để định kỳ tinh chỉnh ngưỡng định tuyến, cập nhật bộ câu hỏi kiểm thử và cải thiện chất lượng tài liệu nền.
- Phát triển giao diện đa nền tảng: mở rộng sang ứng dụng di động và tích hợp vào các nền tảng nhắn tin phổ biến như Zalo, Telegram để tăng khả năng tiếp cận của người dùng.

TÀI LIỆU THAM KHẢO

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, và I. Polosukhin, “Attention Is All You Need,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017. Online: <https://arxiv.org/abs/1706.03762>
- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Tau Yih, T. Rocktäschel, S. Riedel, và D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020. Online: <https://arxiv.org/abs/2005.11401>
- [3] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, và H. Wang, “Retrieval-Augmented Generation for Large Language Models: A Survey,” 2023. Online: <https://arxiv.org/abs/2312.10997>
- [4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, và Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” *International Conference on Learning Representations (ICLR)*, 2023. Online: <https://arxiv.org/abs/2210.03629>
- [5] S. G. Patil, T. Zhang, X. Wang, và J. E. Gonzalez, “Gorilla: Large Language Model Connected with Massive APIs,” 2023. Online: <https://arxiv.org/abs/2305.15334>
- [6] Q. S. Lim, “Corrective Retrieval Augmented Generation,” 2024. Online: <https://arxiv.org/abs/2401.15884>
- [7] Y.-H. Jiang, R. Chi, Z. Zhou, C. Qi, và H. Hu, “AI Agent for Education: von Neumann Multi-Agent System Framework,” 2025. Online: <https://arxiv.org/html/2501.00083v1>
- [8] OpenAI, “OpenAI API Documentation – GPT-4o mini,” 2024. Online: <https://platform.openai.com/docs>
- [9] OpenAI, “Text Embeddings – text-embedding-3-small,” 2024. Online: <https://platform.openai.com/docs/guides/embeddings>
- [10] Qdrant Team, “Qdrant Vector Database Documentation,” 2024. Online: <https://qdrant.tech/documentation>

- [11] S. Ramírez, “FastAPI Documentation,” 2024. Online: <https://fastapi.tiangolo.com>
- [12] Explosion AI, “spaCy: Industrial-Strength Natural Language Processing,” 2024. Online: <https://spacy.io/usage>
- [13] C. Huyen, “Agents,” 2025. Online: <https://huyenchip.com/2025/01/07/agents.html>
- [14] M. Allahyari và A. Yang, “A Practical Approach to Retrieval Augmented Generation Systems,” 2024. Online: <https://mallahyari.github.io/rag-ebook/>